Universidad Escuela Colombiana de Ingeniería Julio Garavito

Desarrollo Orientado por Objetos

DOPO

Presentado por:

Wilson Steven Mendivelso Sanabria

David Santiago Garzon Rodriguez

Docente:

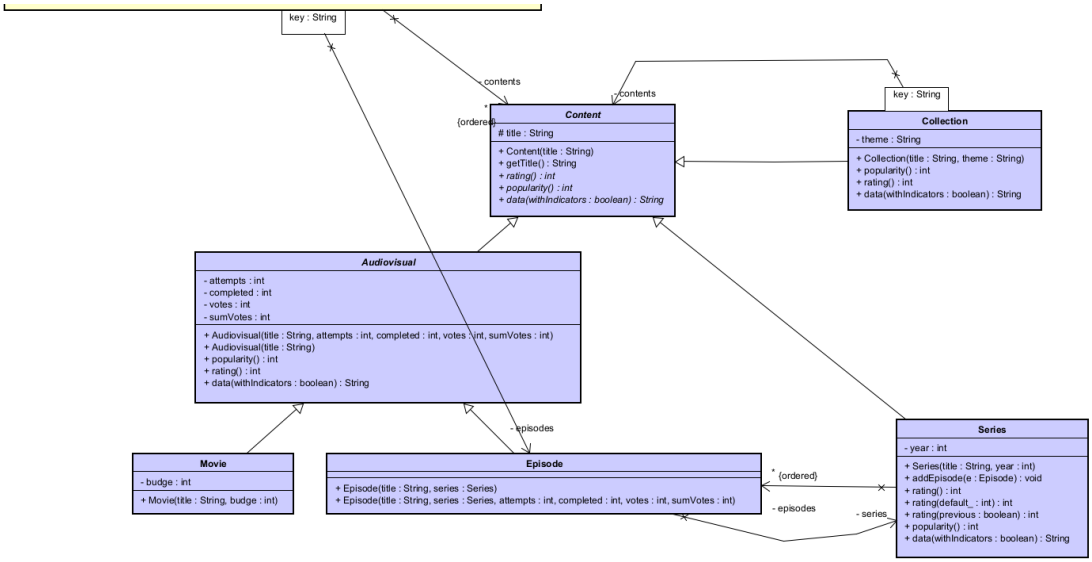
María Irma Diaz Rozo

PARTE I. Practicando con excepciones.

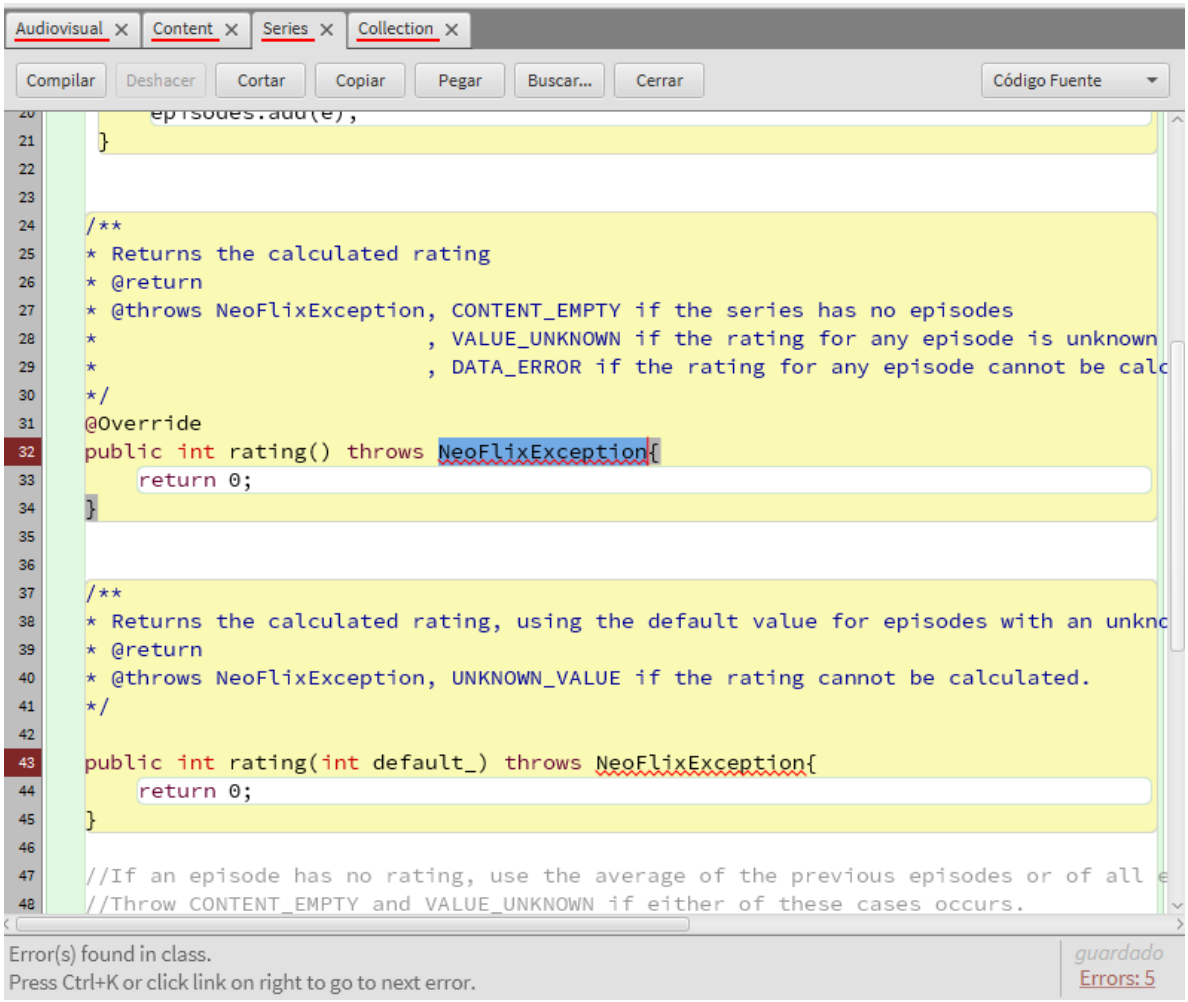
A. MDD y BDD con neoFlix

En este punto vamos a aprender a diseñar, codificar y probar usando excepciones. Para esto se van a trabajar algunos métodos de la clase Serie.

1. En su directorio descarguen los archivos contenidos en neoFlix.zip, revisen el contenido y estudien el diseño estructural de la aplicación (únicamente la zona en azul).



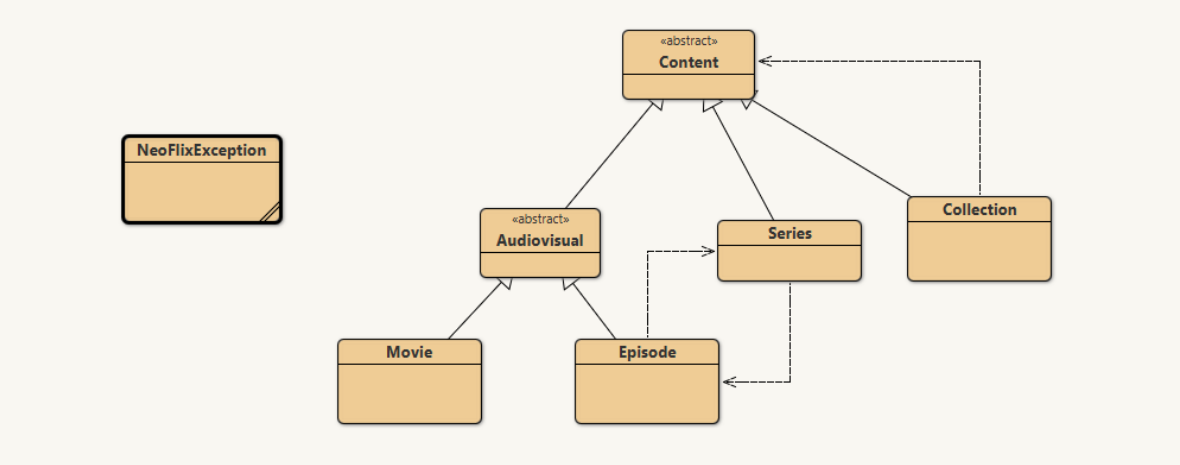
2. Expliquen por qué el proyecto no compila. Realicen las adiciones necesarias para lograrlo.

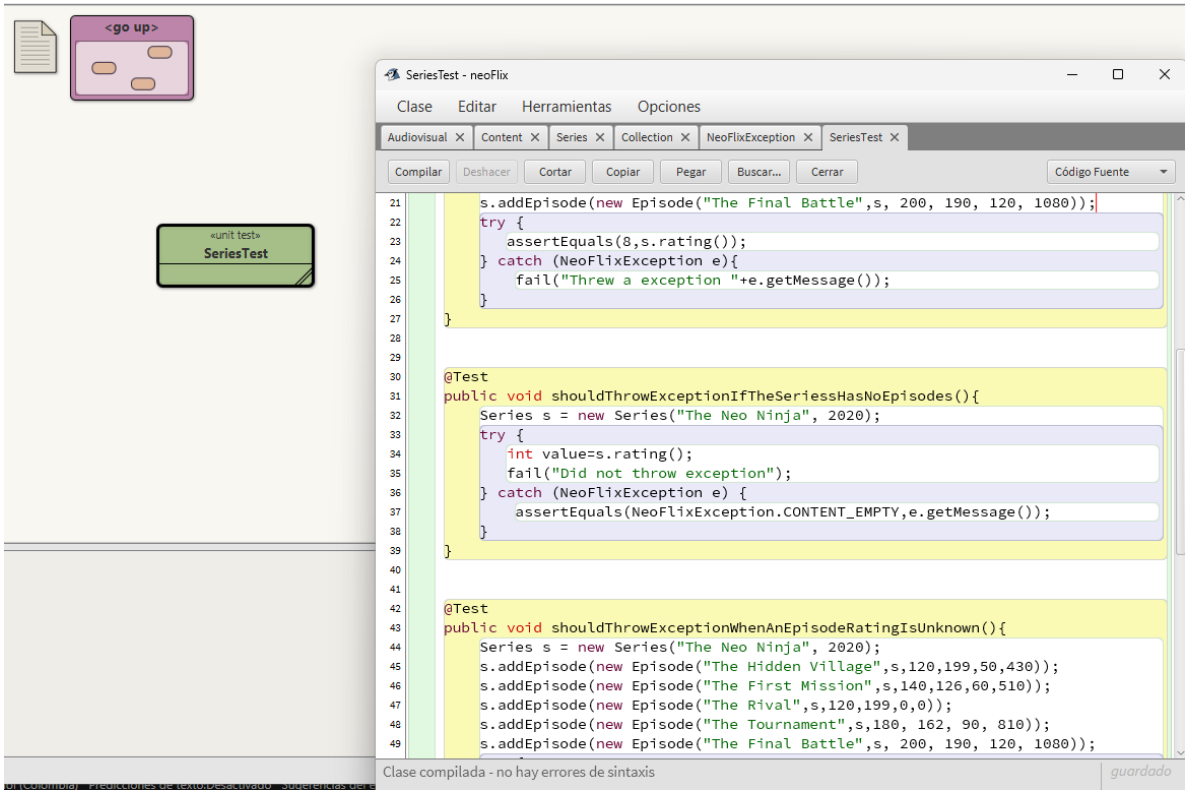


El proyecto no compila por hace falta crear la clase de excepciones NeoFlixException:

```java
package domain;

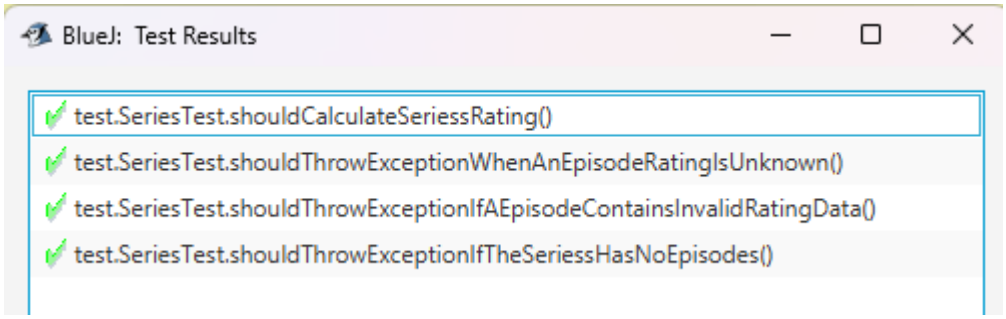

/**
 * Represent the NeoFlixException Class
 *
 * @author Wilson Mendivelso - David Garzon
 * @version 2/10/2026
 */
public class NeoFlixException extends Exception
{
    public static String TO_BE_IMPLEMENTED = "To be implemented";
    public static String VALUE_UNKNOWN = "Value unknown";
    public static String DATA_ERROR = "Data error";
    public static String CONTENT_EMPTY="Content empty";
    /**
     * Constructor for objects of class NeoFlixException
     * @param message of Exception
     */
    public NeoFlixException(String message)
    {
       super(message);
    }
}
```





3. Dado el diseño y las pruebas, documenten (si es necesario) y codifiquen el método raiting()

```java
/**
* Returns the calculated rating
* @return
* @throws NeoFlixException, CONTENT_EMPTY if the series has no episodes
*                         , VALUE_UNKNOWN if the rating for any episode is unknown
*                         , DATA_ERROR if the rating for any episode cannot be calculated due to a data error
*/
@Override
public int rating() throws NeoFlixException{
    if(episodes.size() == 0){
        throw new NeoFlixException(NeoFlixException.CONTENT_EMPTY);
    }
    int sum = 0;
    for(int i = 0; i< episodes.size(); i++){
        int rating = episodes.get(i).rating();
        sum= sum + rating;
    }
    return sum/episodes.size();
}
```



4. Dada la documentación y el diseño, codifiquen y prueben el método raiting(default).

Las pruebas son las siguientes:

```java
/**
 * If we give a default value for rating, but all episodes have rating, the result shouldn't change
 */
@Test
public void shouldCountAllNormallyIfADefaultValueIsGiven(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 400));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 20, 120));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

    try{
        assertEquals(7, s.rating(10));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
```

```java
/**
 * If we give a default value for rating, and there's at least one with a possible UNKOWN_VALUE, we change it's rating value for the default value.
 */
@Test
public void shoulfChangeIfThereIsAnUnkownValueAndADefaultValueIsGiven(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 400));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 0, 0));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

    try{
        assertEquals(8, s.rating(10));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
```

```
/**
 * If we give a default value for rating a there's a Data Error it should ignore it.
 */
@Test
public void shouldIgnoreIfThereIsDataErrorAndADefaultValueIsGiven(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 540));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 20, 120));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

    try{
        assertEquals(20/3, s.rating(10));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
```

```
/**
 * If there is an unkown value and there is a data error, it should ignore the data error and should
 * change the unkown value.
 */
@Test
public void shouldIgnoreDataErrorAndChangeUnkownValueIfADefaultValueIsGiven(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 540));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 0, 0));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

    try{
        assertEquals(8, s.rating(10));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
```

El método fue codificado de la siguiente manera:

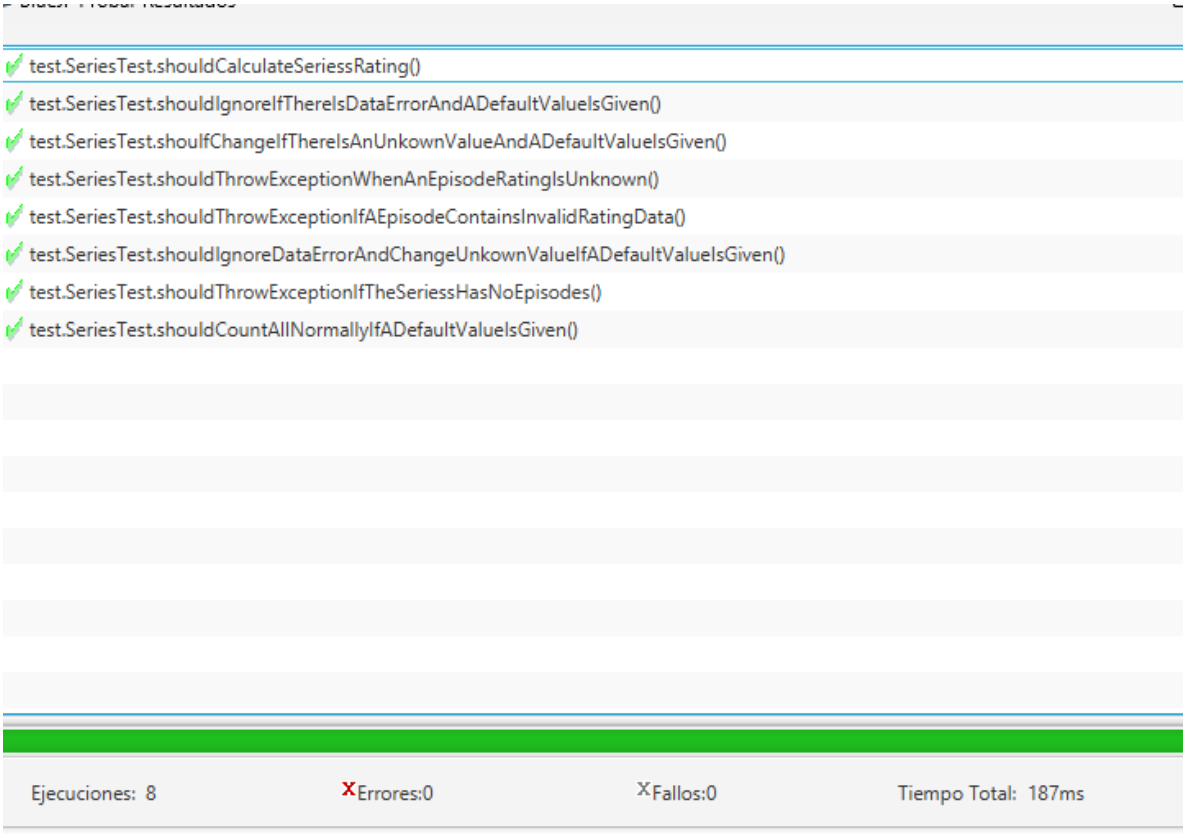
```
/**
* Returns the calculated rating, using the default value for episodes with an unknown rating and ignoring those with errors.
* @param default_ is the default value if rating has no value
* @return the series rating
* @throws NeoFlixException, UNKNOWN_VALUE if the rating cannot be calculated.
*/

public int rating(int default_) throws NeoFlixException{
    if(episodes.size() == 0){
        throw new NeoFlixException(NeoFlixException.CONTENT_EMPTY);
    }
    int sum = 0;
    int cont = 0;
    for(Episode e: episodes){
        int rating = 0;
        try{
            rating = e.rating();
            cont++;
        }
        catch(NeoFlixException ex){
            if(ex.getMessage().equals(NeoFlixException.VALUE_UNKNOWN)){
                rating = default_;
                cont++;
            }

        }

        sum= sum + rating;
    }
    return sum/cont;
}
```

Al ejecutar las pruebas:



5. Documenten, diseñen, codifiquen y prueben el método raiting(previous).
   Para previous se codificaron las siguientes pruebas:
   - Si es true:

```java
/**
 * If previous is true, but all episodes have rating, it should count rating normally
 */
@Test
public void shouldCountNormallyIfPreviousIsTrue(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 400));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 20, 120));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

    try{
        assertEquals(7, s.rating(true));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
/**
 * If previous is true and there's an unkown value it should calculate with its rating with the previous episodes.
 */
@Test
public void shouldCountPreviousIfValueIsUnknown(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 400));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 0, 0));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

    try{
        assertEquals(31/4, s.rating(true));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
```

```java
/**
 * If previous is true and there's a data error it should calculate with its rating with the previous episodes.
 */
@Test
public void shouldCountPreviousIfValueIsDataError(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 540));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 20, 120));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

    try{
        assertEquals(15/2, s.rating(true));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
/**
 *If previous is true and there's a data error or an unkown value it should calculate with its rating with the previous episodes.
 */
@Test
public void shouldCountPreviousIfValueIsDataErrorOrAnUnkownValue(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 540));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 0, 0));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

    try{
        assertEquals(17/2, s.rating(true));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
```

Si es false:

```java
/**
 * If previous is false, but all episodes have rating it should calculate with its original rating
 */
@Test
public void shouldCountNormallyIfPreviousIsFalse(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 400));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 20, 120));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

    try{
        assertEquals(7, s.rating(false));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
/**
 * If previous is false, and there is an unkown value, first calculate the other rating values and then it put that average to the unkown value
 */
@Test
public void shouldCountAllIfValueIsUnknown(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 400));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 0, 0));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

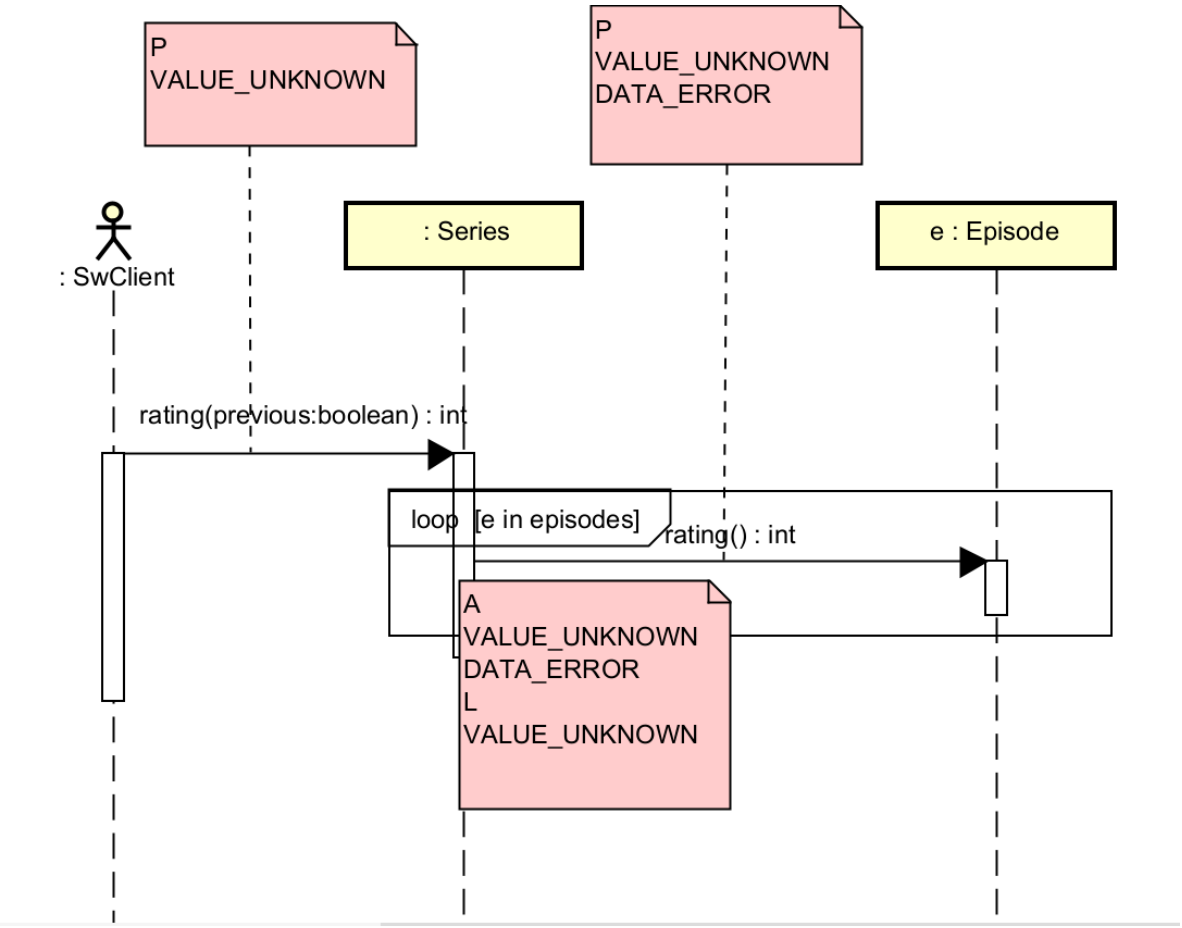
    try{
        assertEquals(22/3, s.rating(false));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
```

```java
/**
 * If previous is false, and there is a data error, first calculate the other rating values and then it put that average to the data
 */
@Test
public void shouldCountAllIfThereIsADataError(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 540));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 20, 120));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

    try{
        assertEquals(6, s.rating(false));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
/**
 * If previous is false, and there's at least one data error and an unkown value, it will put the rating of all the others to those.
 */
@Test
public void shouldCountAllIfValueIsDataErrorOrAnUnkownValue(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 120, 199, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 110, 180, 50, 540));
    s.addEpisode(new Episode("Heisenberg", s, 120, 199, 0, 0));
    s.addEpisode(new Episode("the fly", s, 120, 199, 10, 40));

    try{
        assertEquals(7, s.rating(false));
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
```

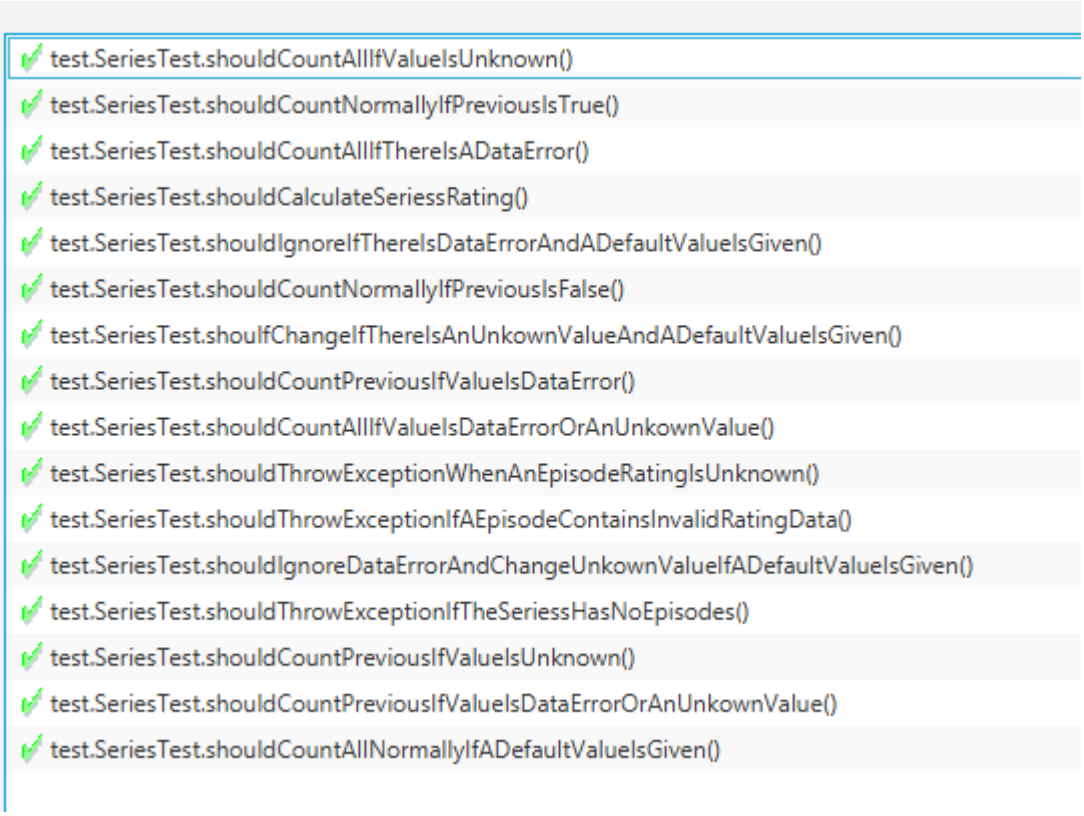
El diseño es el ya dado:



El código sería el siguiente:

```java
/**
 * If an episode has no rating, use the average of the previous episodes or of all episodes, depending on the value of the previous parameter.
 * @param previous says if it will take previous rating for the unkown or data error values, or if it will take all the values
 * @return the serie's rating
 * @Throw CONTENT_EMPTY and VALUE_UNKNOWN if either of these cases occurs.
 */
public int rating(boolean previous) throws NeoFlixException{
    if(episodes.size() == 0){
        throw new NeoFlixException(NeoFlixException.CONTENT_EMPTY);
    }
    int sum = 0;
    int unkownOrDataError = 0;
    for(Episode e: episodes){
        int rating = 0;
        try{
            rating = e.rating();
        }
        catch(NeoFlixException ex){
            if(previous){
                if(episodes.indexOf(e) != 0){
                    rating = sum/episodes.indexOf(e);
                }
                else{
                    throw new NeoFlixException(NeoFlixException.VALUE_UNKNOWN);
                }
            }
            else{
                unkownOrDataError++;
            }
        }
        sum = sum + rating;
    }
    while(unkownOrDataError > 0){
        sum+= sum/(episodes.size()-unkownOrDataError);
        unkownOrDataError--;
    }
    return sum/episodes.size();
}
```

Y se puede ver como pasan todas las pruebas:



6. Propongan el cálculo, documenten, diseñen, codifiquen y prueben el método popularity(). (Definan reglas que obliguen a utilizar excepciones en sus cuatro momentos: propagar, lanzar y atender)

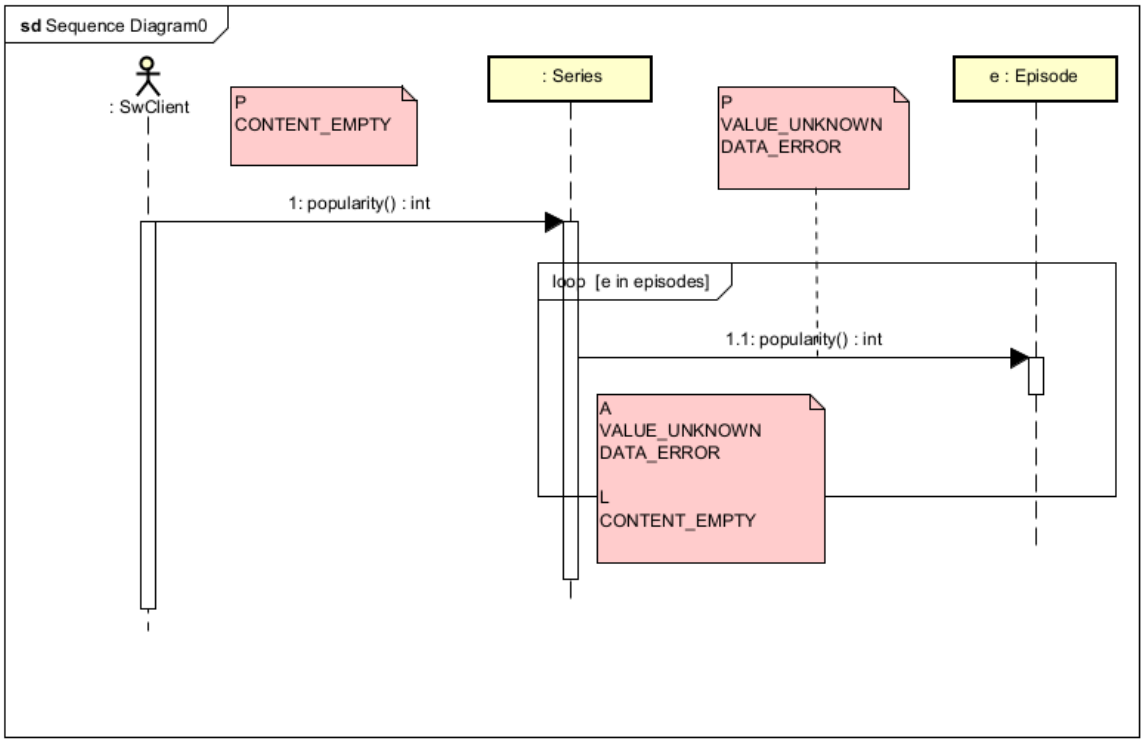
   Para las reglas que definiremos para usar el método de popularity, es necesario recordar que en las unidades audiovisuales, ya está definida como debe ser la popularidad:

   - Popularidad: (Terminados / intentos)[0..100]

   Por tanto la popularidad de la serie funcionará igual que con el rating, en plan, sumar y dar un promedio. Eso sí, es necesario tener en cuenta que dentro de audiovisual ya está, aunque se le agregó otra excepción.

```java
/**
 * Gives the popularity of an Audiovisual content.
 * @return Audiovisual's popularity.
 */
@Override
public int popularity() throws NeoFlixException{
    if (attempts == 0) throw new NeoFlixException(NeoFlixException.VALUE_UNKNOWN);
    else if(completed > attempts ) throw new NeoFlixException(NeoFlixException.DATA_ERROR);
    return (completed*100)/attempts;
}
```

Teniendo en cuenta lo anterior se diseñó del siguiente modo:



Las pruebas diseñadas son las siguientes:

```java
/**
 * Should count the popularity if all the values are known.
 */
@Test
public void shouldCalculatePopularityIfAllValuesAreKnown(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 199, 120, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 180, 110, 50, 400));
    s.addEpisode(new Episode("Heisenberg", s, 199, 120, 20, 120));
    s.addEpisode(new Episode("the fly", s, 199, 120, 10, 40));

    try{
        assertEquals(60, s.popularity());
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
/**
 * If a serie has no episodes it should launch an exception.
 */
@Test
public void shouldLaunchAnExceptionIfSerieHasNoEpisodes(){
    Series s = new Series("Breaking Bad", 2008);
    try{
        int popularity = s.popularity();
        fail("It didn't throw an exception");
    }
    catch (NeoFlixException e){
        assertEquals(NeoFlixException.CONTENT_EMPTY,e.getMessage());
    }
}
```

```java
/**
 * Should put 0 in popularity, if episode has unkown values.
 */
@Test
public void shouldPutZeroInPopularityForEpisodesWithUnkownValues(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 199, 120, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 0, 180, 50, 400));
    s.addEpisode(new Episode("Heisenberg", s, 199, 120, 20, 120));
    s.addEpisode(new Episode("the fly", s, 0, 199, 10, 40));

    try{
        assertEquals(30, s.popularity());
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
/**
 * Should put 0 in popularity, if episodes has data error values.
 */
@Test
public void shouldPutZeroInPopularityIfHasEpisodesWithDataError(){
    Series s = new Series("Breaking Bad", 2008);
    s.addEpisode(new Episode("Ozymandias", s, 199, 120, 100, 1000));
    s.addEpisode(new Episode("Felina", s, 20, 180, 50, 400));
    s.addEpisode(new Episode("Heisenberg", s, 199, 120, 20, 120));
    s.addEpisode(new Episode("the fly", s, 20, 199, 10, 40));

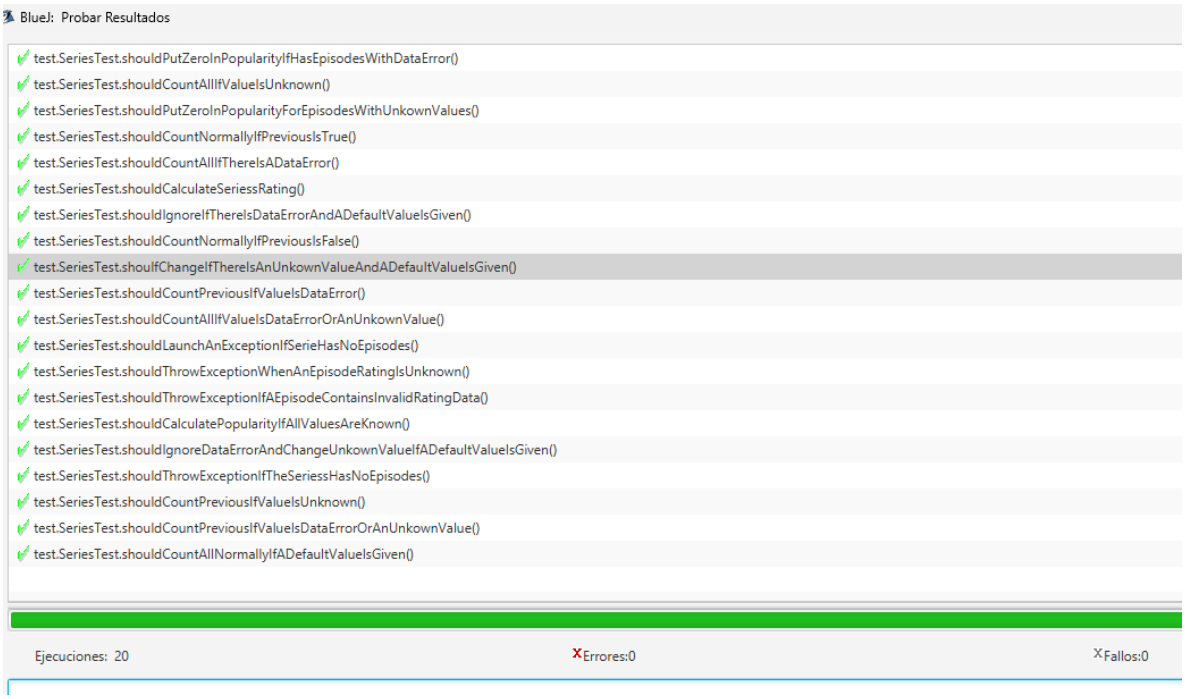
    try{
        assertEquals(30, s.popularity());
    }
    catch(NeoFlixException e){
        fail("It throw an exception");
    }
}
```

El código es el siguiente:

```java
/**
 * Calculates the popularity of a serie, if it has episodes with unkown values, it will take them as popularity of 0.
 * @throws NeoFlixException CONTENT_EMPTY if the series has no episodes
 */
public int popularity() throws NeoFlixException{
    int sum = 0;
    if(episodes.size() == 0){
        throw new NeoFlixException(NeoFlixException.CONTENT_EMPTY);
     }
    for(Episode e: episodes){
        try{
            sum += e.popularity();
        }
        catch(NeoFlixException ex){
            sum += 0;
        }
    }

    return sum/episodes.size();
}
```

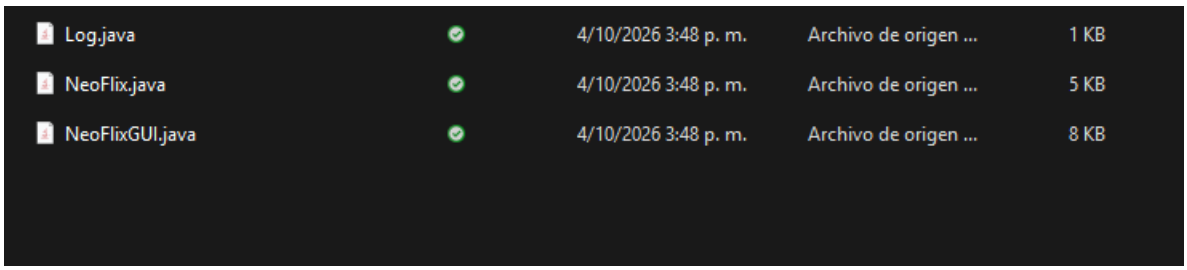
Al ejecutar todas las pruebas:



PARTE II. Desarrollando

A Revisando línea base

1. En su directorio descarguen los archivos contenidos en neoFlixMore.zip, revisen el contenido.

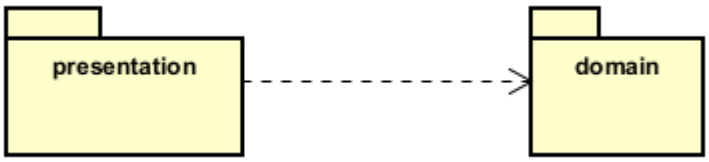


(a) ¿Cuántos archivos se tienen?
Tiene 3 archivos, Log, NeoFlix, y NeoFlixGUI

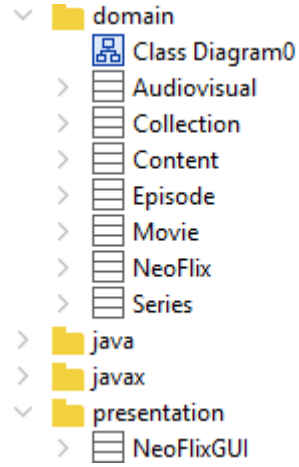
(b) ¿Cómo están organizados?
Todos están en una misma carpeta.

(c) ¿Cómo deberían estar organizados?
Debería haber una carpeta donde estuviera el domain(con Log y NeoFlix) y otra para presentation (donde iría NeoFlixGUI).

2. Estudien el diseño del programa: diagramas de paquetes y de clases.
   (a) ¿Cuántos paquetes tenemos?
   Tenemos dos paquetes: presentation y domain.

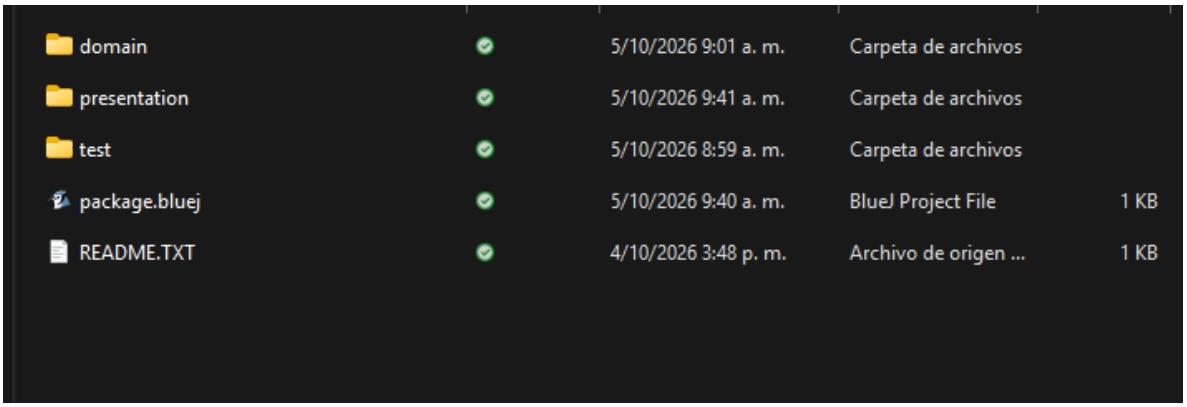


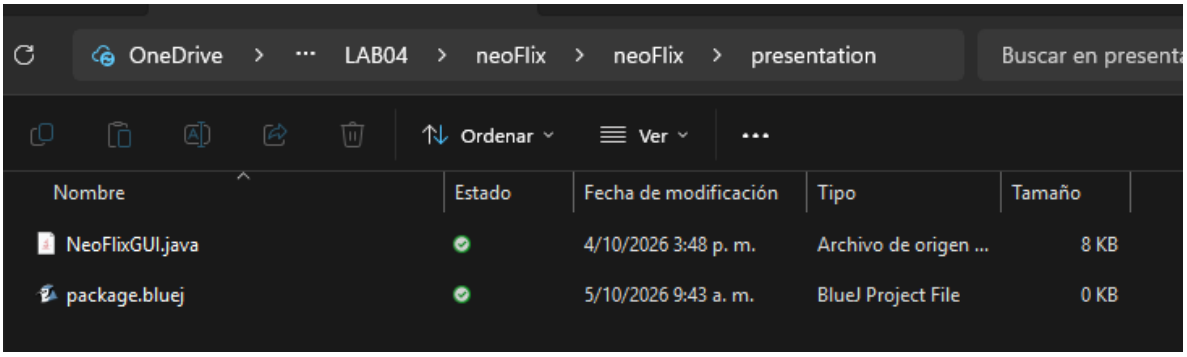
   (b) ¿Cuántas clases tiene el sistema?
   Tiene 8 clases.
   (c) ¿Cómo están organizadas?
   7 clases se encuentran en el domain y la otra se encuentra en presentation:



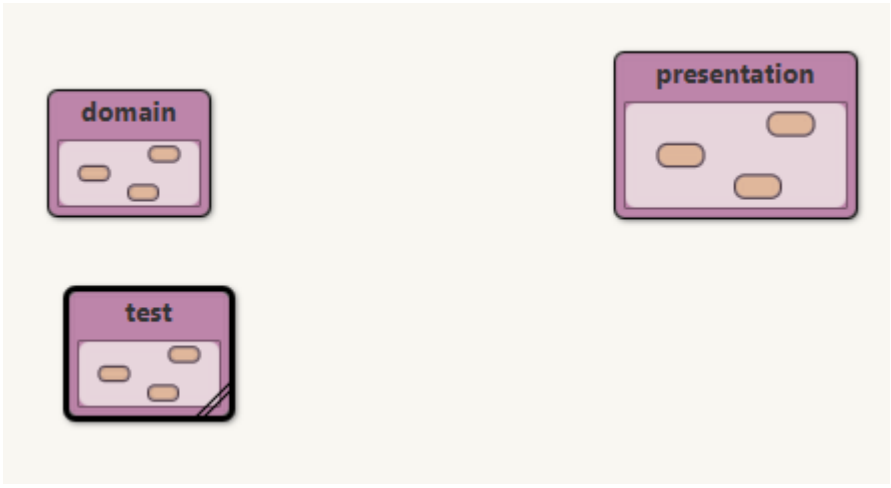
   (c) ¿Cuál es la clase ejecutiva?
   La clase ejecutiva es NeoFlixGUI, puesto que es aquella que tiene el main y permite ejecutarlo.

3. Incluya los nuevos archivos en los directorios correspondientes para ejecutar el proyecto,
   (a) ¿Qué clases deben tener cada directorio?
   El directorio de domain debe tener NeoFlix y Log como añadidos.
   El directorio de presentation debe crearse para añadir NeoFlixGUI.
   (b) ¿Cómo los incluyeron?
   Se creó la nueva carpeta de presentation y se añadió ahí.
   Además se añadió en domain NeoFlix y Log.

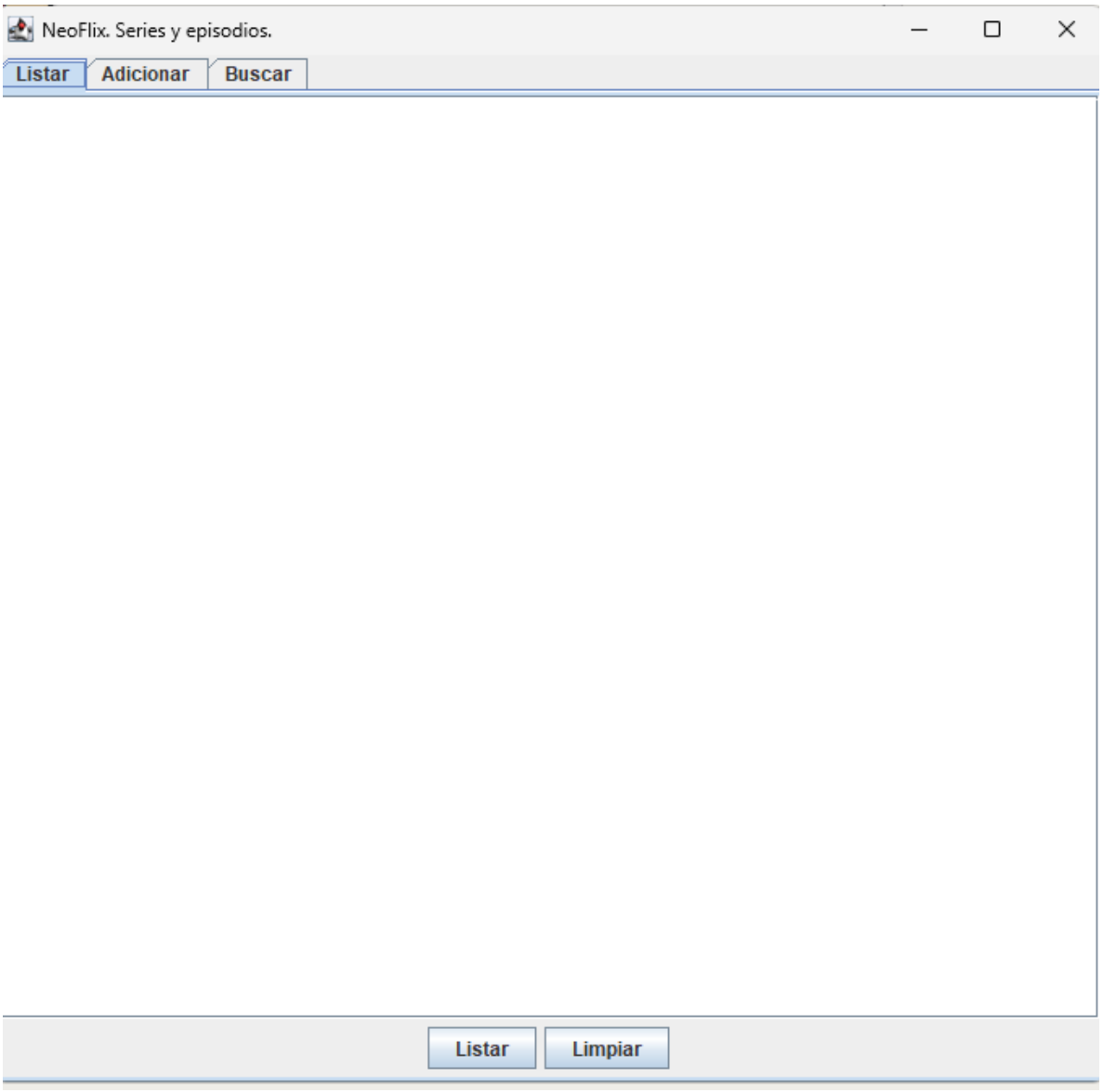




Dentro de BlueJ:



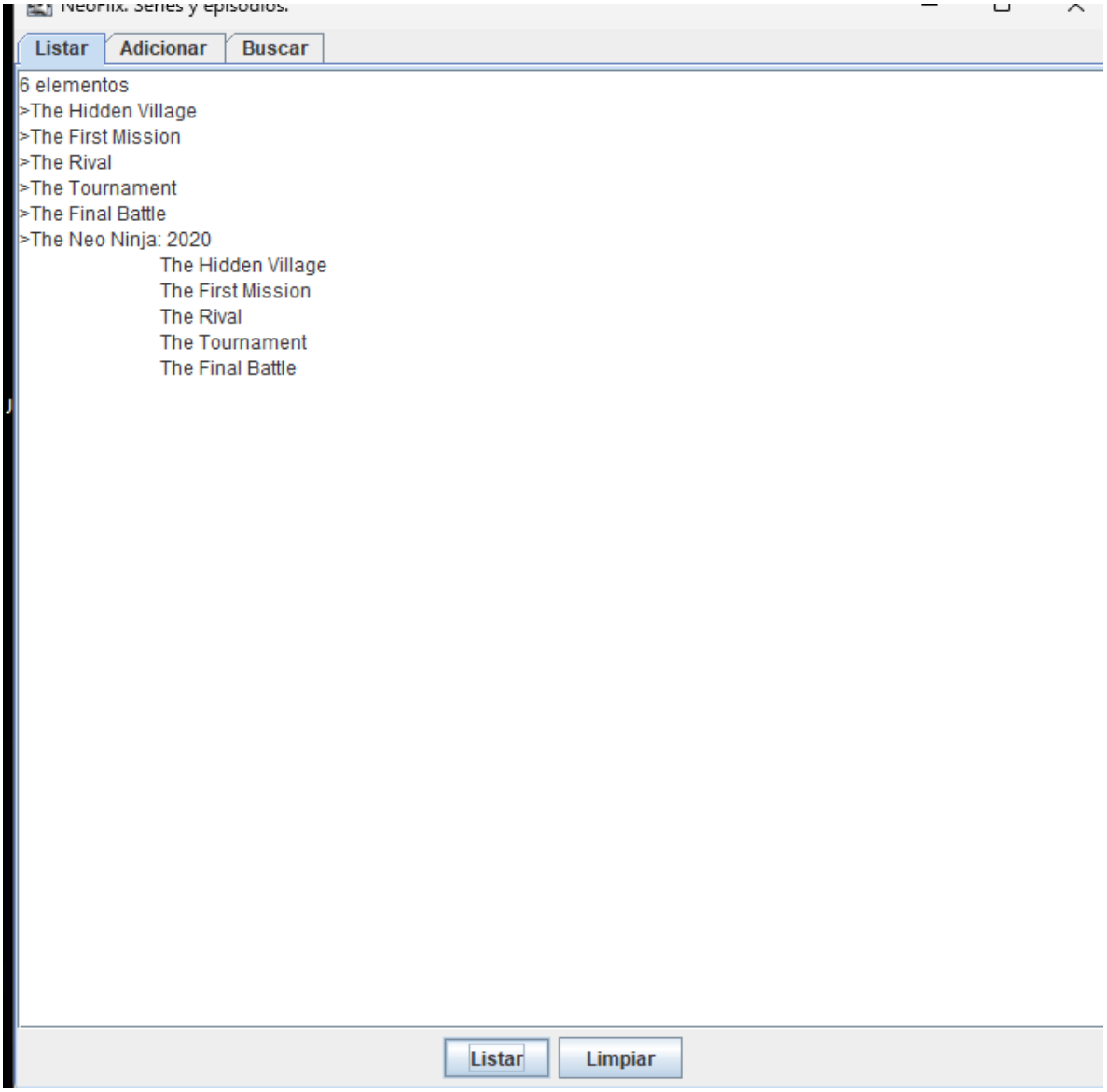
4. Ejecute el proyecto,
   (a) ¿Qué funcionalidades ofrece?



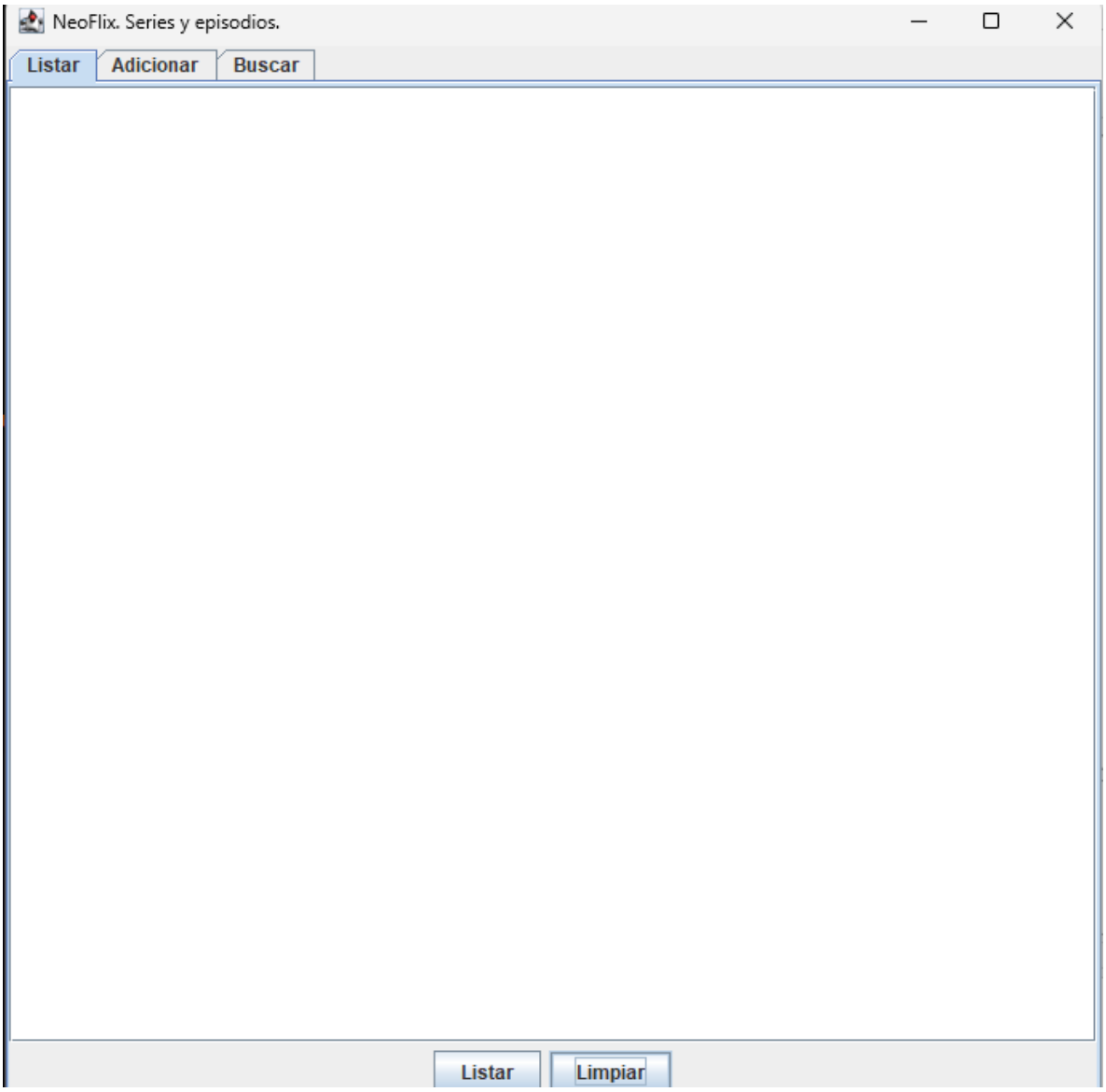
Ofrece Listar, Adicionar y Buscar

(b) ¿Cuáles funcionan?

Funciona Listar:



Y también funciona Limpiar:



Por otro lado, al usar adicionar no se añade nada y al usar buscar sale un error:

```
Can only enter input while your program is running
Exception in thread "AWT-EventQueue-0" java.lang.IndexOutOfBoundsException: Index 6 out of bounds for length 6
        at java.base/jdk.internal.util.Preconditions.outOfBounds(Preconditions.java:100)
        at java.base/jdk.internal.util.Preconditions.outOfBoundsCheckIndex(Preconditions.java:106)
        at java.base/jdk.internal.util.Preconditions.checkIndex(Preconditions.java:302)
        at java.base/java.util.Objects.checkIndex(Objects.java:385)
        at java.base/java.util.ArrayList.get(ArrayList.java:427)
        at domain.NeoFlix.select(NeoFlix.java:108)
        at domain.NeoFlix.search(NeoFlix.java:144)
        at presentation.NeoFlixGUI.actionSearch(NeoFlixGUI.java:235)
        at presentation.NeoFlixGUI$6.insertUpdate(NeoFlixGUI.java:209)
        at java.desktop/javax.swing.text.AbstractDocument.fireInsertUpdate(AbstractDocument.java:227)
```

5. Revisen el código y la documentación del proyecto.
   (a) ¿De dónde salen los datos iniciales?
   Salen de addSomeSerie():

```java
/**
* Adds sample series and episodes to NeoFlix.
*/
private void addSomeSeries(){

    String [][] episodes= {{"The Hidden Village","199","120","50","430"},
                           {"The First Mission","126","140","60","510"},
                           {"The Rival","199","120","50","430"},
                           {"The Tournament","162","180", "90", "810"},
                           {"The Final Battle","190", "200", "120", "1080"}};
    StringBuffer titles=new StringBuffer();
    for (String [] e : episodes){
        titles.append(e[0]+"\n");
    }

    for (String [] e: episodes){
        addEpisode(e[0],"?", e[1],e[2],e[3],e[4]);
    }
    String [][] series = {{"The Neo Ninja", "2020", titles.toString()}};
    for (String [] s: series){
        addSeries(s[0],s[1],s[2]);
    }


}
```

El cual se llama a través del constructor de NeoFlix:

```java
/**
 * Create a NeoFlix
 */
public NeoFlix(){
    contents = new ArrayList<Content>();
    episodes = new TreeMap<String,Episode>();
    addSomeSeries();
}
```

(b) ¿Qué clase pide que se adicionen?
La clase que pide que se adicionen es NeoFlix

```java
/**
 * Create a NeoFlix
 */
public NeoFlix(){
    contents = new ArrayList<Content>();
    episodes = new TreeMap<String,Episode>();
    addSomeSeries();
}
```

```java
/**
* Adds sample series and episodes to NeoFlix.
*/
private void addSomeSeries(){

    String [][] episodes= {{"The Hidden Village","199","120","50","430"},
                           {"The First Mission","126","140","60","510"},
                           {"The Rival","199","120","50","430"},
                           {"The Tournament","162","180", "90", "810"},
                           {"The Final Battle","190", "200", "120", "1080"}};
    StringBuffer titles=new StringBuffer();
    for (String [] e : episodes){
        titles.append(e[0]+"\n");
    }

    for (String [] e: episodes){
        addEpisode(e[0],"?", e[1],e[2],e[3],e[4]);
    }
    String [][] series = {{"The Neo Ninja", "2020", titles.toString()}};
    for (String [] s: series){
        addSeries(s[0],s[1],s[2]);
    }


}
```

(c) ¿Qué clase los adiciona?

Los adiciona NeoFlix

```java
/**
 * Create a NeoFlix
 */
public NeoFlix(){
    contents = new ArrayList<Content>();
    episodes = new TreeMap<String,Episode>();
    addSomeSeries();
}
```

```java
/**
* Adds sample series and episodes to NeoFlix.
*/
private void addSomeSeries(){

    String [][] episodes= {{"The Hidden Village","199","120","50","430"},
                           {"The First Mission","126","140","60","510"},
                           {"The Rival","199","120","50","430"},
                           {"The Tournament","162","180", "90", "810"},
                           {"The Final Battle","190", "200", "120", "1080"}};
    StringBuffer titles=new StringBuffer();
    for (String [] e : episodes){
        titles.append(e[0]+"\n");
    }

    for (String [] e: episodes){
        addEpisode(e[0],"?", e[1],e[2],e[3],e[4]);
    }
    String [][] series = {{"The Neo Ninja", "2020", titles.toString()}};
    for (String [] s: series){
        addSeries(s[0],s[1],s[2]);
    }


}
```

B Realizando inversa. Adicionar y listar. Todo OK.

(NO OLVIDEN BDD – MDD)

El objetivo es realizar ingeniería inversa a las funciones de adicionar y listar.

1. Adicionen un nuevo episodio de una nueva serie.

Serie

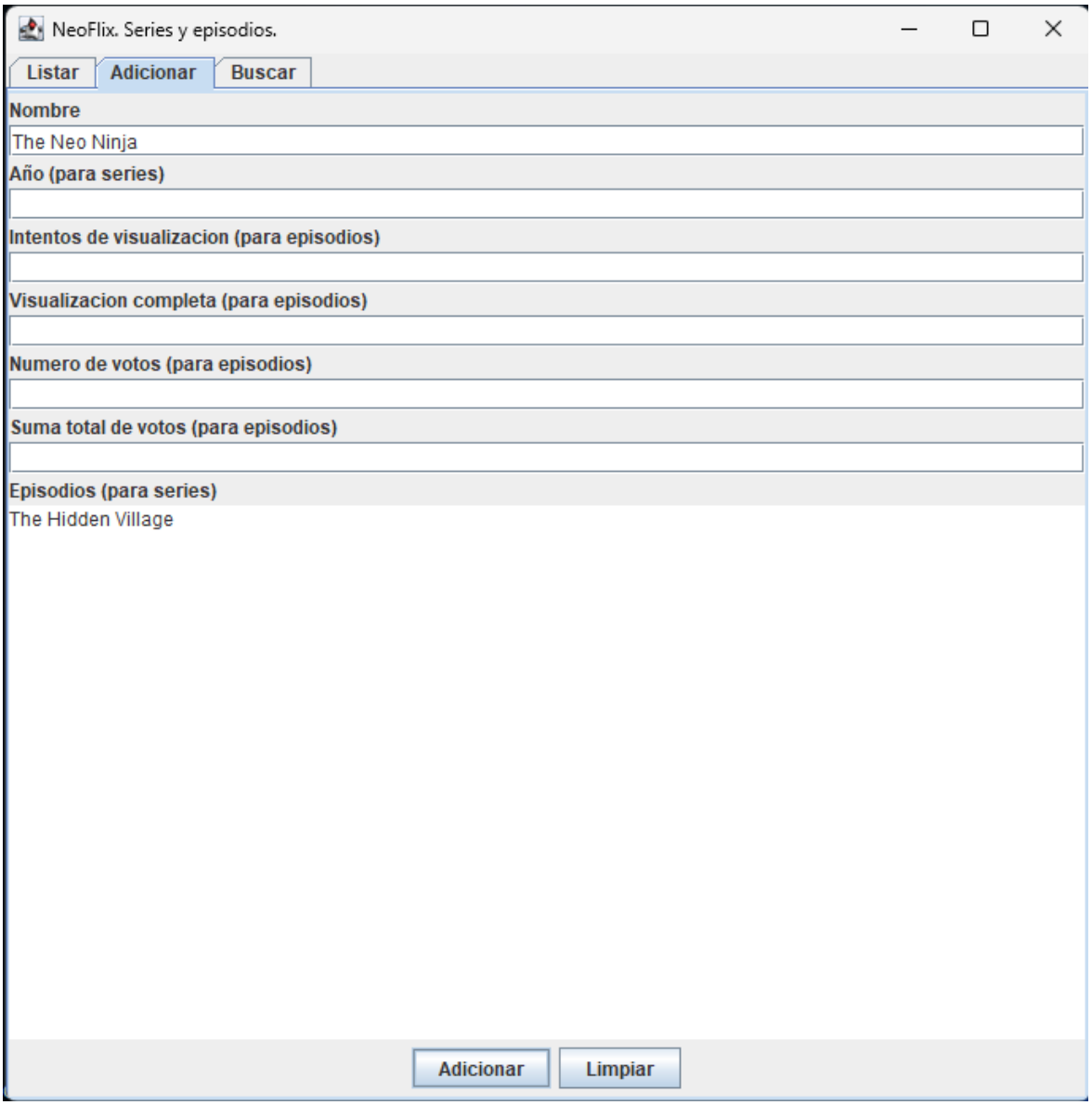
The Neo Ninja

Episodio

The Hidden Village

(a) ¿Qué ocurre?

No pasa nada, no se añade nada.



BlueJ: Ventana de Terminal - neoFlix

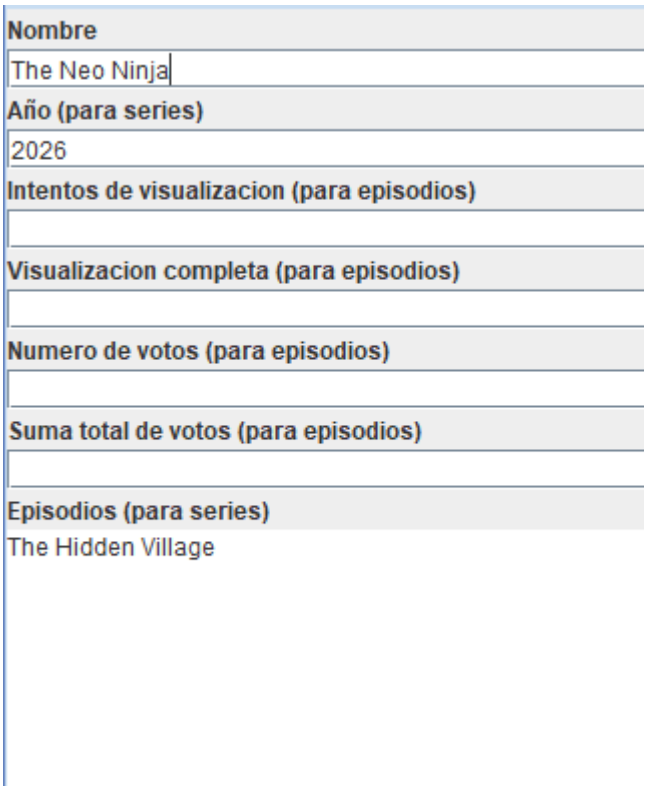
Opciones

Can only enter input while your program is running

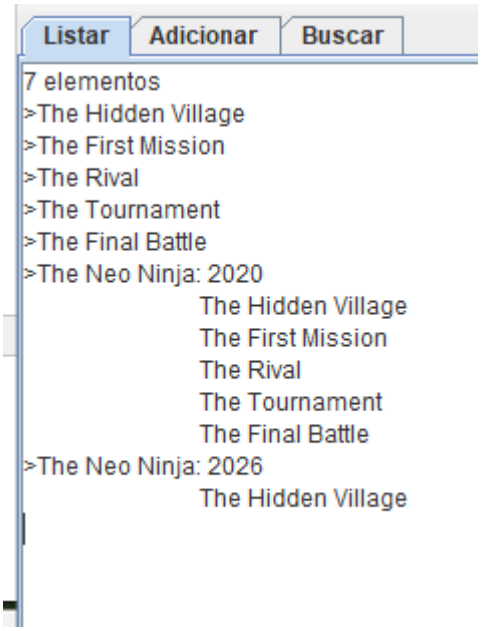
```
Exception in thread "AWT-EventQueue-0" java.lang.NumberFormatException: For input string: ""
        at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
        at java.base/java.lang.Integer.parseInt(Integer.java:672)
        at java.base/java.lang.Integer.parseInt(Integer.java:778)
        at domain.NeoFlix.addSeries(NeoFlix.java:91)
        at presentation.NeoFlixGUI.actionAdd(NeoFlixGUI.java:227)
        at presentation.NeoFlixGUI$4.actionPerformed(NeoFlixGUI.java:186)
        at java.desktop/javax.swing.AbstractButton.fireActionPerformed(AbstractButton.java:1972)
        at java.desktop/javax.swing.AbstractButton$Handler.actionPerformed(AbstractButton.java:2314)
        at java.desktop/javax.swing.DefaultButtonModel.fireActionPerformed(DefaultButtonModel.java:407)
        at java.desktop/javax.swing.DefaultButtonModel.setPressed(DefaultButtonModel.java:262)
        at java.desktop/javax.swing.plaf.basic.BasicButtonListener.mouseReleased(BasicButtonListener.java:279)
        at java.desktop/java.awt.Component.processMouseEvent(Component.java:6621)
        at java.desktop/javax.swing.JComponent.processMouseEvent(JComponent.java:3398)
        at java.desktop/java.awt.Component.processEvent(Component.java:6386)
        at java.desktop/java.awt.Container.processEvent(Container.java:2266)
        at java.desktop/java.awt.Component.dispatchEventImpl(Component.java:4996)
        at java.desktop/java.awt.Container.dispatchEventImpl(Container.java:2324)
        at java.desktop/java.awt.Component.dispatchEvent(Component.java:4828)
        at java.desktop/java.awt.LightweightDispatcher.retargetMouseEvent(Container.java:4948)
```

Inclusive ocurre un error.

Si le pongo un año cualquiera:

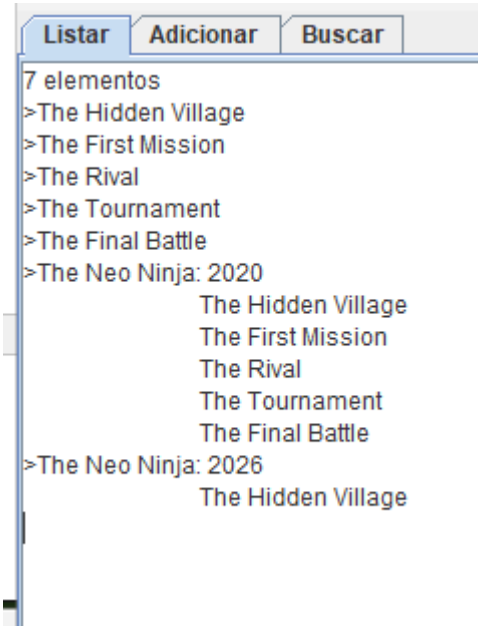


Ya el error no sale, y se pone en la lista:

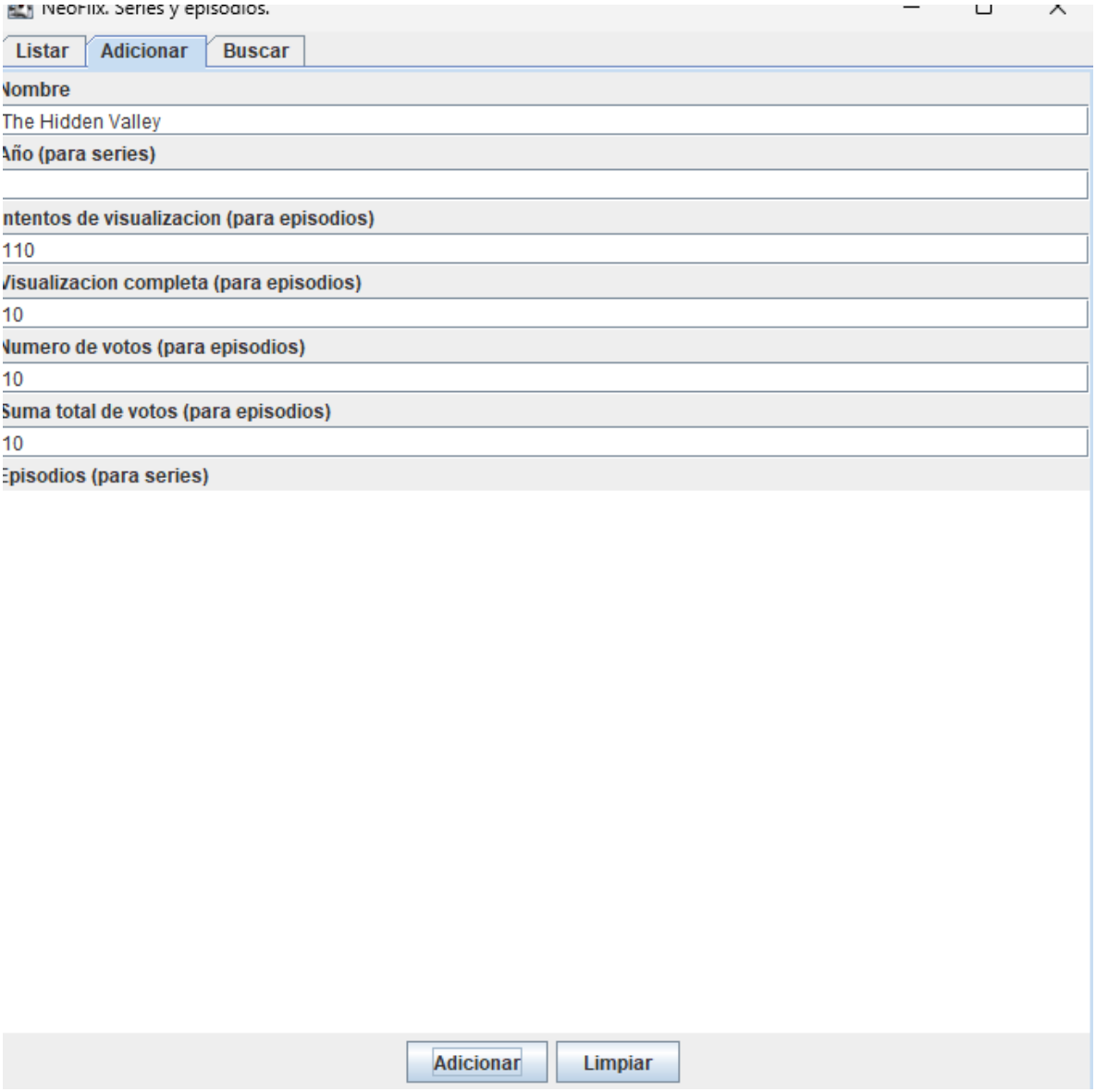


(b) ¿Cómo lo comprueban? Capturen la pantalla.

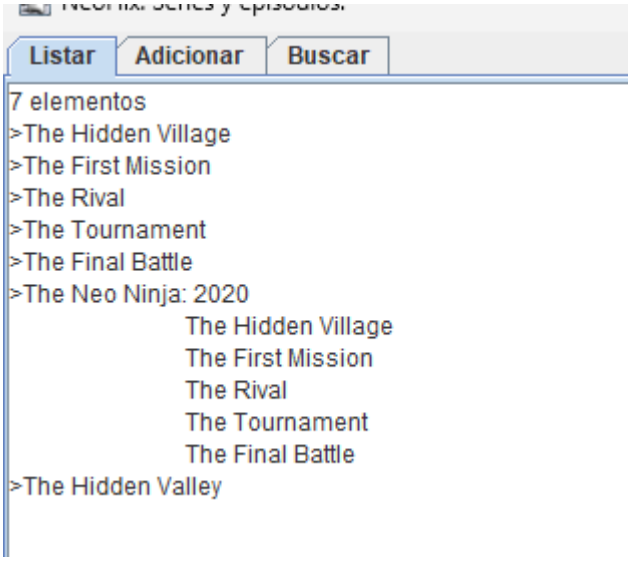
Al darle listar, aparece en la lista:



Por otro lado si solo añado el capítulo:



A la hora de listar:



(c) ¿Es adecuado este comportamiento?

Si, puesto que se está añadiendo una nueva serie con su capitulo. Eso sí, si trato de poner el que ya estaba o sea The Neo Ninja con año 2020 y con el capitulo The Hidden Village, me lo tomará como una serie diferente:



Lo único raro, es que teóricamente ya hay otro episodio llamado The Hidden Village en uno de los casos que presenté, al añadir el episodio vacio, lo cual es extraño que suceda, pero como en la vida real existen series con episodios llamados iguales, pues no hay demasiado problema.

2. Revisen el código asociado a adicionar en la capa de presentación y la capa de dominio.

(a) ¿Qué método es responsable en la capa de presentación?

El metodo es actionAdd()

(b) ¿Qué método en la capa de dominio?

Los metodos son addSeries(String title, String year, String theEpisodes):

```java
/**
 * Add a new series
 * @param title
 * @param year
 * @param theEpisodes the titles of the episodes separated by newlines
*/
public void addSeries(String title, String year, String theEpisodes){
    Series s = new Series(title,Integer.parseInt(year));
    String [] aEpisodes= theEpisodes.split("\n");
    for (String te : aEpisodes){
        s.addEpisode(episodes.get(te.toUpperCase()));
    }
    contents.add(s);
}
```

Y addEpisode(String title, String series, String attempts, String completed, String votes, String sumVotes):

```java
/**
* Adds a new episode.
* @param title
* @param series
* @param attempts
* @param completed
* @param votes
* @param sumVotes
*/
public void addEpisode(String title, String series, String attempts, String completed, String votes, String sumVotes){
    Series s = (Series) consult(series);
    Episode e=new Episode(title,s, Integer.parseInt(attempts), Integer.parseInt(completed), Integer.parseInt(votes), Integer.parseInt(sumVotes));
    contents.add(e);
    episodes.put(title.toUpperCase(),e);
}
```

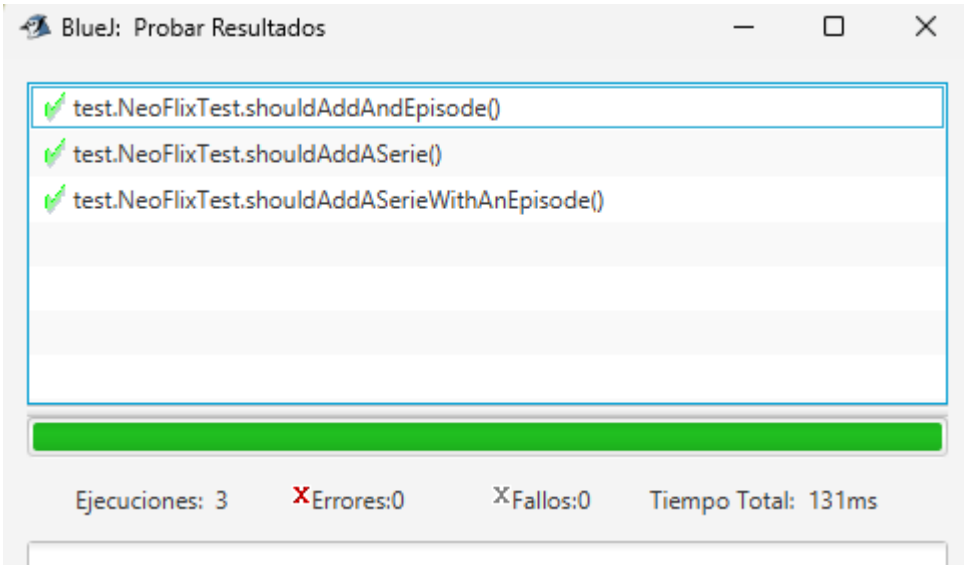
3. Realicen ingeniería inversa para la capa de dominio para adicionar. Capturen los resultados de las pruebas de unidad.

Como no encontramos pruebas de unidad para esta parte, decidimos hacer las nuestras propias, enfocadas en añadir episodios y series.

```java
public class NeoFlixTest{

    /**
     * Should add an episode without serie.
     */
    @Test
    public void shouldAddAndEpisode(){
        NeoFlix neoFlix = new NeoFlix();
        neoFlix.addEpisode("the fly", "?", "180", "170", "10", "40");
        assertTrue(neoFlix.consult("the fly") != null);

    }
    /**
     * Should add a Serie without episodes.
     */
    @Test
    public void shouldAddASerie(){
        NeoFlix neoFlix = new NeoFlix();
        neoFlix.addSeries("Breaking bad", "2008", "");
        assertTrue(neoFlix.consult("Breaking bad") != null);
    }
    /**
     * Should add a Serie with an Episode
     */
    @Test
    public void shouldAddASerieWithAnEpisode(){
        NeoFlix neoFlix = new NeoFlix();
        neoFlix.addEpisode("the fly", "?", "180", "170", "10", "40");
        neoFlix.addSeries("Breaking bad", "2008", "the fly");
        assertTrue(neoFlix.consult("Breaking bad") != null);
        assertTrue(neoFlix.consult("the fly") != null);
    }
```



4. Revisen el código asociado a listar en la capa de presentación y la capa de dominio.

(a) ¿Qué método es responsable en la capa de presentación?

El metodo es actionList():

```
private void actionList(){
    textDetails.setText(neoFlix.toString());
}
```

(b) ¿Qué método en la capa de dominio?

El metodo es toString(), puesto que cuando llama a actionList() se le está diciendo a neoFlix que use ese método para listar.

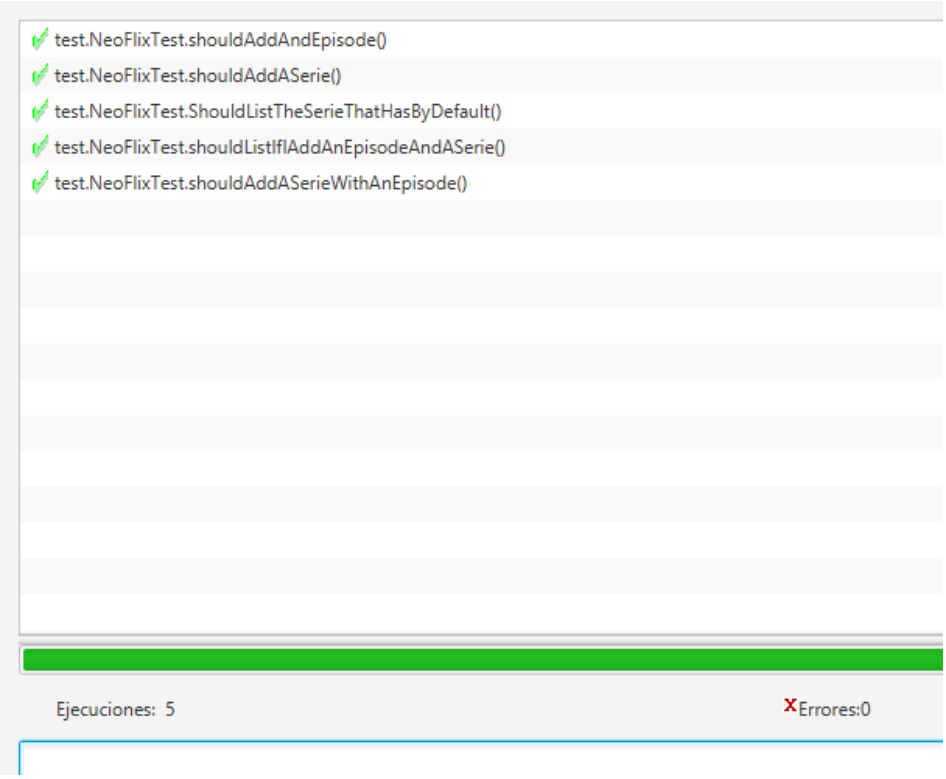
```
private void actionList(){
    textDetails.setText(neoFlix.toString());
}
```

```
/**
 * Return the data of all episodes
 * @return
 */
public String toString(){
    return data(contents,false);
}
```

5. Realicen ingeniería reversa para la capa de dominio para listar. Capturen los resultados de las pruebas de unidad.

Para este caso tampoco encontramos pruebas propuestas para probar, por tanto decidimos crear las nuestra propias:

```java
/**
 * Should list the serie that has by default.
 */
@Test
public void ShouldListTheSerieThatHasByDefault(){
    NeoFlix neoFlix = new NeoFlix();
    String esperado = "6 elementos\n"+
                      ">The Hidden Village\n"+
                      ">The First Mission\n"+
                      ">The Rival\n"+
                      ">The Tournament\n"+
                      ">The Final Battle\n"+
                      ">The Neo Ninja: 2020"+
                          "\n\tThe Hidden Village"+
                          "\n\tThe First Mission"+
                          "\n\tThe Rival"+
                          "\n\tThe Tournament"+
                          "\n\tThe Final Battle\n";
    assertEquals(esperado, neoFlix.toString());
}
/**
 * Should list if I add an episode and a Serie to neoFlix
 */
@Test
public void shouldListIfIAddAnEpisodeAndASerie(){
    NeoFlix neoFlix = new NeoFlix();
    neoFlix.addEpisode("the fly", "Breaking bad", "180", "170", "10", "40");
    neoFlix.addSeries("Breaking bad", "2008", "the fly");
    String esperado = "8 elementos\n"+
                      ">The Hidden Village\n"+
                      ">The First Mission\n"+
                      ">The Rival\n"+
                      ">The Tournament\n"+
                      ">The Final Battle\n"+
                      ">The Neo Ninja: 2020"+
                          "\n\tThe Hidden Village"+
                          "\n\tThe First Mission"+
                          "\n\tThe Rival"+
                          "\n\tThe Tournament"+
                          "\n\tThe Final Battle\n"+
                      ">the fly\n"+
                      ">Breaking bad: 2008"+
                          "\n\tthe fly\n";
    assertEquals(esperado, neoFlix.toString());
}
```



6. Propongan y ejecuten una prueba de aceptación. Capturen las pantallas correspondientes.

Teoricamente, con lo anterior ya se crearon varias pruebas de aceptación, puesto que no habían, pero si es necesario, se puede crear otra:

```
/**
 * Should fail if serie has an episode that does not exists yet.
 */
@Test
public void shouldFailIfSerieHasAnEpisodeThatDoesNotExistsYet(){
    NeoFlix neoFlix = new NeoFlix();
    neoFlix.addSeries("Breaking bad", "2008", "the fly");
    try{
        String resultado = neoFlix.toString();
        fail("Should throw an exception");
    }
    catch(NullPointerException e){

    }
}
```

Esta prueba simplemente busca mirar una posibilidad que fue encontrada, la cual es que si se busca listar una serie con un episodio que aún no existe como contenido, entonces va a lanzar un NullPointerException.

C. Fortaleciendo. Adicionar robusta

El objetivo es fortalecer la funcionalidad de adicionar para hacerla más robusta. Para cada uno de los siguientes casos realice los pasos del 1 al 4.

(i) ¿Y si el nombre ya existe?
(ii) ¿Y si los valores numéricos no son numéricos?
(iii) ¿Y si los valores no son los esperados? (suma de votos>10*votos o terminados>intentos)
(iv) Proponga una nueva condición

1. Propongan una prueba de aceptación que genere el fallo.

(a) ¿Se generó alguna excepción?

(b) ¿Por qué?

2. Analicen el diseño dinámico (diagrama de secuencia). Para hacer la funcionalidad robusta:

(a) ¿Qué método debería lanzar la excepción?

(b) ¿Qué métodos deberían propagarla?

(c) ¿Qué método debería atenderla? Expliquen claramente.

3. Construya la solución propuesta. Incluyan las excepciones en el diseño y presente evidencia de las pruebas de unidad: una ejecución normal y una para cada excepción definida.

4. Ejecuten nuevamente la aplicación con el caso de aceptación propuesto en 1.

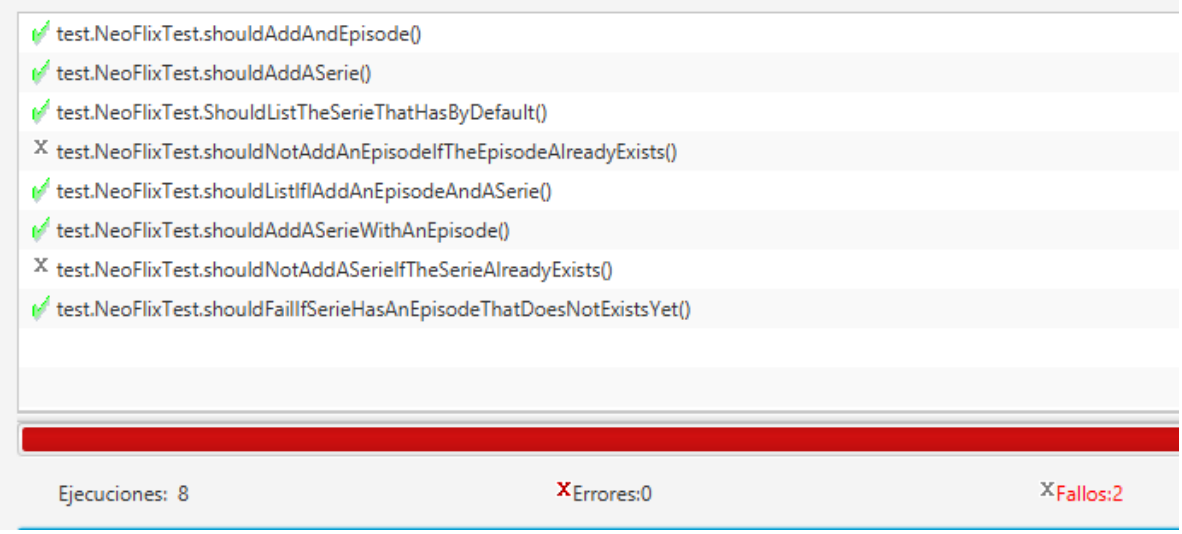
(a) ¿Qué sucede ahora? Capturen la pantalla.

(i) ¿Y si el nombre ya existe?

1. Propongan una prueba de aceptación que genere el fallo.

Se crearon dos pruebas unitarias:

```java
/**
 * Should not add a serie if the serie already exists.
 */
@Test
public void shouldNotAddASerieIfTheSerieAlreadyExists(){
    NeoFlix neoFlix = new NeoFlix();
    try{
        neoFlix.addSeries("The Neo Ninja", "2020", "The Hidden Village");
        fail("It didn't throw an exception");
    }
    catch(NeoFlixException e){
        assertEquals(NeoFlixException.CONTENT_ALREADY_EXISTS,e.getMessage());
    }
}
/**
 * Should not add a episode if the episode already exists
 */
@Test
public void shouldNotAddAnEpisodeIfTheEpisodeAlreadyExists(){
    NeoFlix neoFlix = new NeoFlix();
    try{
        neoFlix.addEpisode("The Hidden Village","?","199","120","50","430");
        fail("It didn't throw an exception");
    }
    catch(NeoFlixException e){
        assertEquals(NeoFlixException.CONTENT_ALREADY_EXISTS,e.getMessage());
    }
}
```

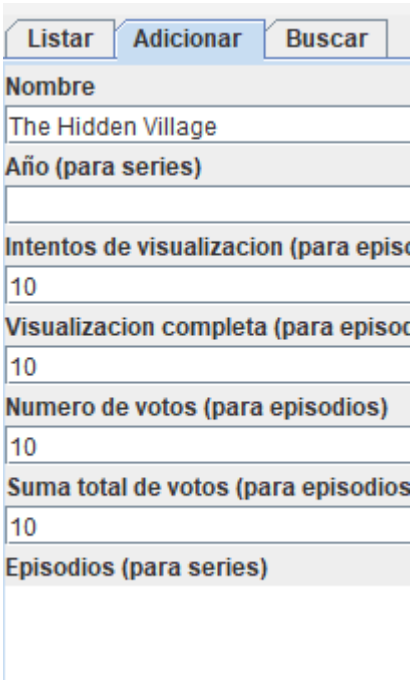


Además, se crea la prueba de aceptación:

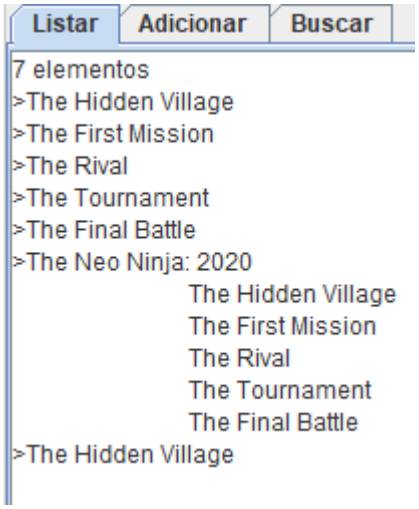
Originalmente empezamos así:



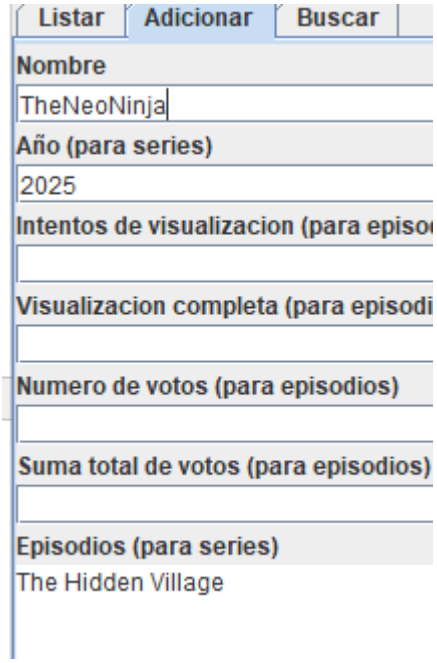
Si tratamos de añadir algo contenido repetido:



Se agrega:



Pero esto puede generar error, porque imaginemos que queremos crear una serie nueva:





Aunque ahí diga que sabe cuál es el episodio, no podemos estar del todo seguros que vaya a tomar el episodio de la serie antigua.

(a) ¿Se generó alguna excepción?

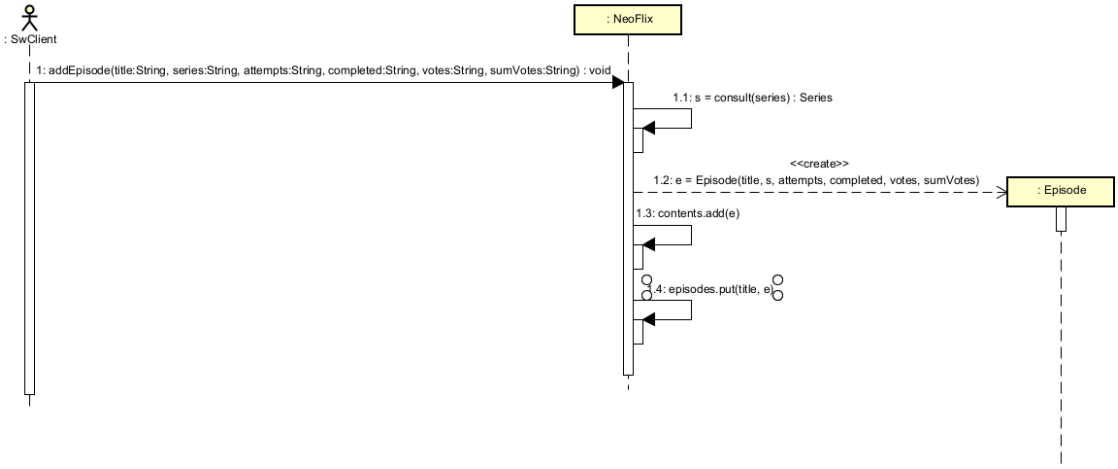
No.

(b) ¿Por qué?

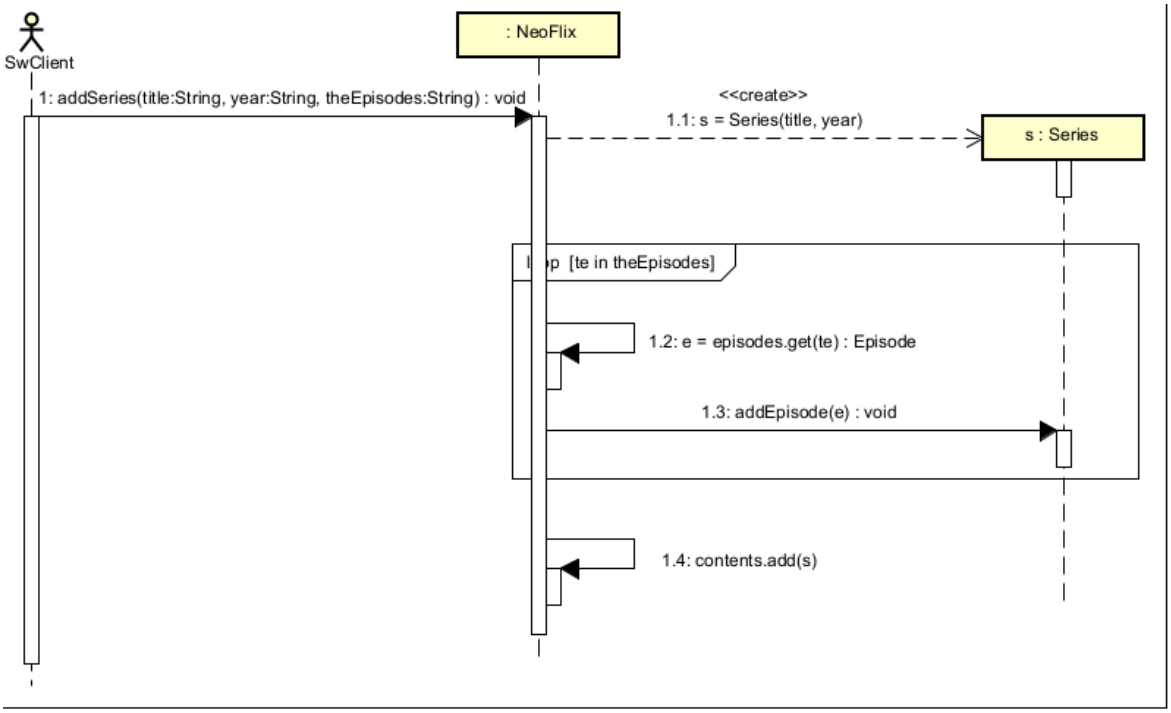
Porque como está implementado de momento, no debería generar excepciones, puesto que no tiene alguna forma de propagar.

2. Analicen el diseño dinámico (diagrama de secuencia). Para hacer la funcionalidad robusta:

El diagrama de secuencia inicial para añadir los episodios es el siguiente:



El diagrama de secuencia para añadir las series es el siguiente:



(a) ¿Qué método debería lanzar la excepción?

Para evitar romper el flujo de las pruebas de SeriesTest, es necesario desarrollar un nuevo método (por cada uno) que se ejecute dentro de addSeries y addEpisode en NeoFlix, el cuál se encargue de lanzar la excepción.

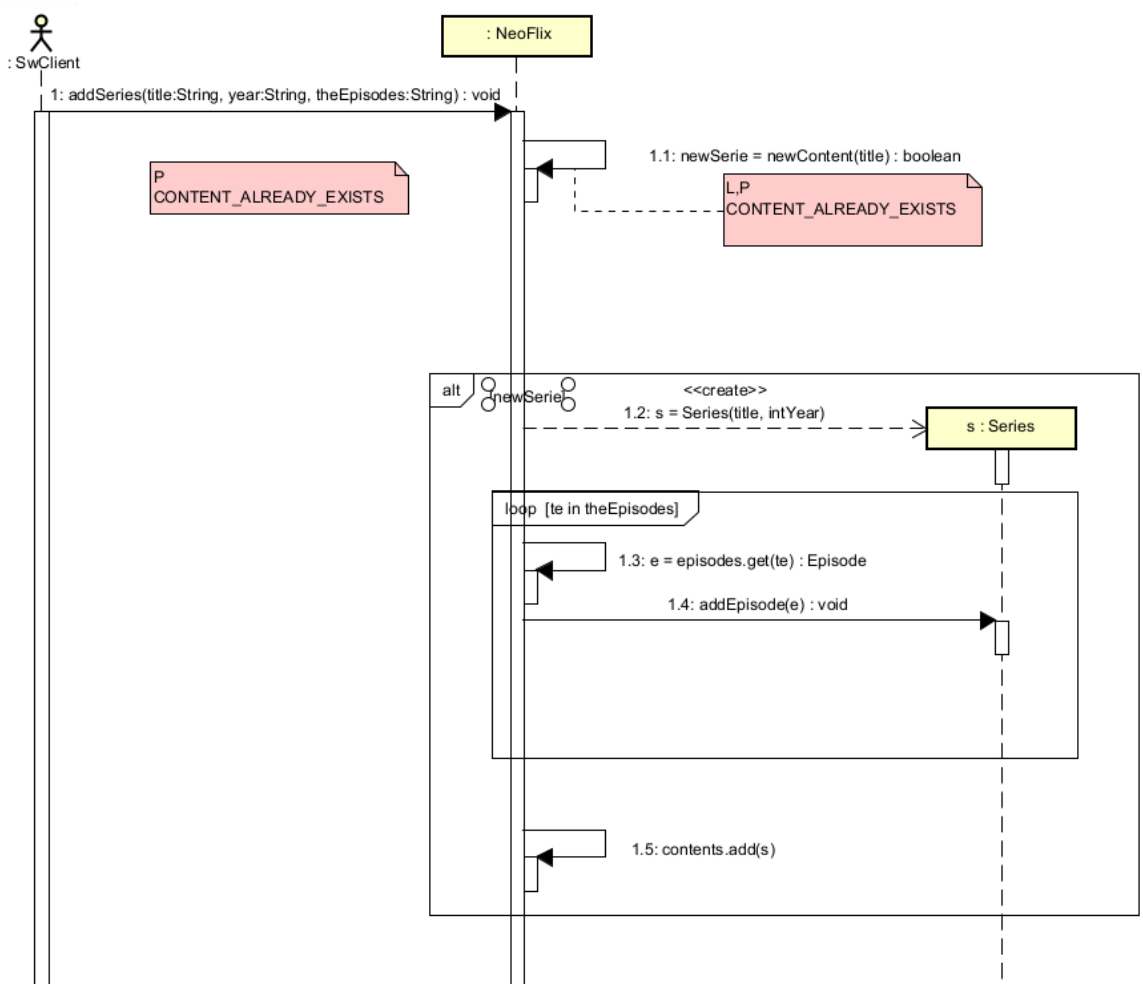
(b) ¿Qué métodos deberían propagarla?

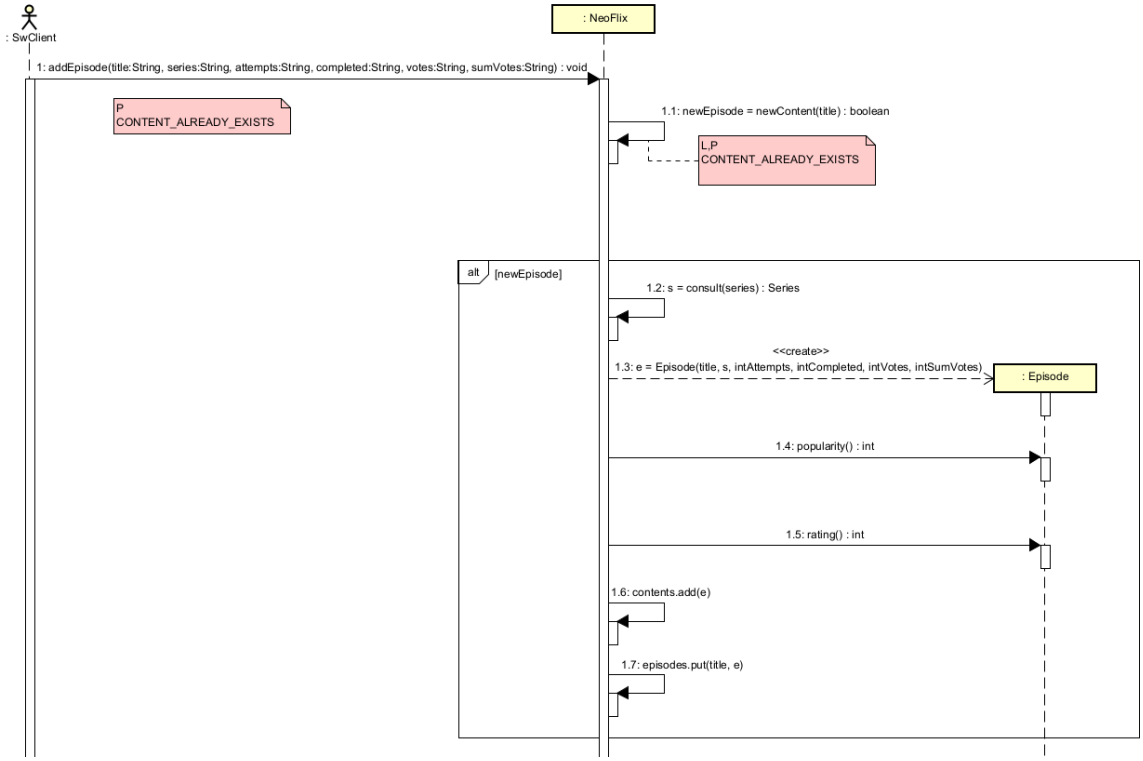
Esos mismos métodos que deseamos crear, van a encargarse de propagarla.

(c) ¿Qué método debería atenderla? Expliquen claramente.

La parte de NeoFlixGUI debería encargarse. Siendo que le debe mostrar al usuario algún tipo de mensaje si intenta ingresar alguna serie que ya existe.

3. Construya la solución propuesta. Incluyan las excepciones en el diseño y presente evidencia de las pruebas de unidad: una ejecución normal y una para cada excepción definida.





Además fue necesario modificar NeoFlixException para incluir esa nueva excepción:

```java
/**
 * Represent the NeoFlixException Class
 *
 * @author Wilson Mendivelso - David Garzon
 * @version 2/10/2026
 */
public class NeoFlixException extends Exception
{
    public static String TO_BE_IMPLEMENTED = "To be implemented";
    public static String VALUE_UNKNOWN = "Value unknown";
    public static String DATA_ERROR = "Data error";
    public static String CONTENT_EMPTY="Content empty";
    public static String CONTENT_ALREADY_EXISTS =  "Content already exists";
    /**
     * Constructor for objects of class NeoFlixException
     * @param message of Exception
     */
    public NeoFlixException(String message)
    {
       super(message);
    }
}
```

Además tocó añadir ese nuevo método, dentro de NeoFlix:

```java
/**
 * Consults if it is a new content
 * @param title is the title of the content
 * @return if content exists
 * @throws NeoFlixException - CONTENT_ALREADY_EXISTS
 */

private boolean newContent(String title) throws NeoFlixException{
    Content c = consult(title);
    if(c != null){
        throw new NeoFlixException(NeoFlixException.CONTENT_ALREADY_EXISTS);
    }
    return true;
}
```

Por tanto el método de addEpisode quedó como:

```java
/**
* Adds a new episode.
* @param title
* @param series
* @param attempts
* @param completed
* @param votes
* @param sumVotes
* @throws CONTENT_ALREADY_EXISTS, if content already exists
*/
public void addEpisode(String title, String series, String attempts, String completed, String votes, String sumVotes) throws NeoFlixException{
    boolean newEpisode = newContent(title);

    Series s = (Series) consult(series);
    Episode e=new Episode(title,s, Integer.parseInt(attempts), Integer.parseInt(completed), Integer.parseInt(votes), Integer.parseInt(sumVotes));
    contents.add(e);
    episodes.put(title.toUpperCase(),e);

}
```

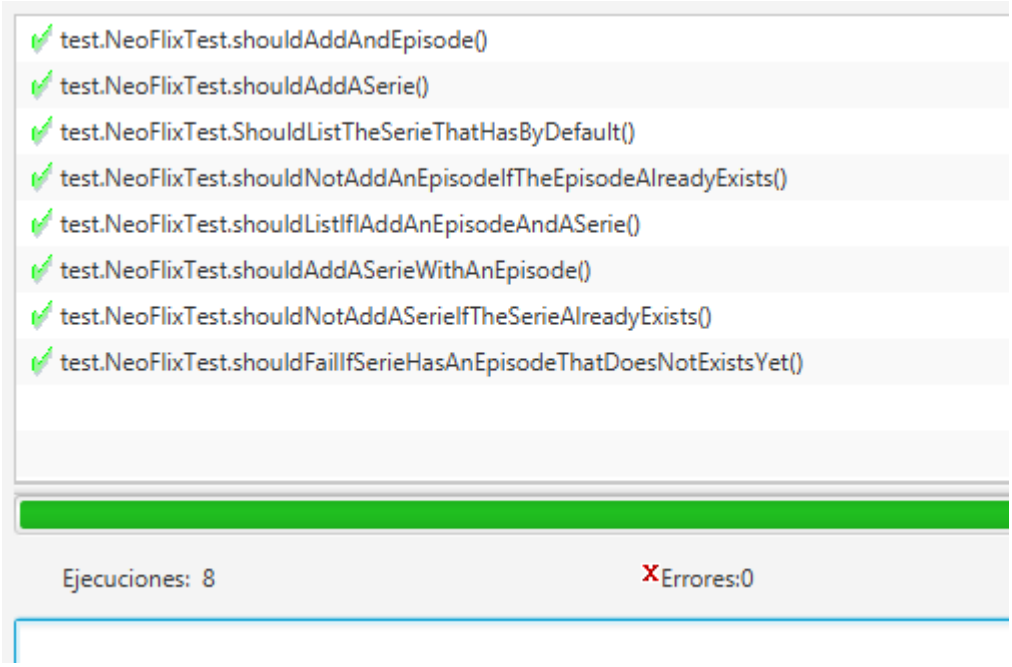
Mientras que el de addSeries quedó como:

```java
/**
 * Add a new series
 * @param title
 * @param year
 * @param theEpisodes the titles of the episodes separated by newlines
 * @throws CONTENT_ALREADY_EXISTS, if content already exists
*/
public void addSeries(String title, String year, String theEpisodes) throws NeoFlixException{
    boolean newSerie = false;
    newSerie = newContent(title);
    Series s = new Series(title,Integer.parseInt(year));
    String [] aEpisodes= theEpisodes.split("\n");
    for (String te : aEpisodes){
        s.addEpisode(episodes.get(te.toUpperCase()));
    }
    contents.add(s);


}
```

4. Ejecuten nuevamente la aplicación con el caso de aceptación propuesto en 1.

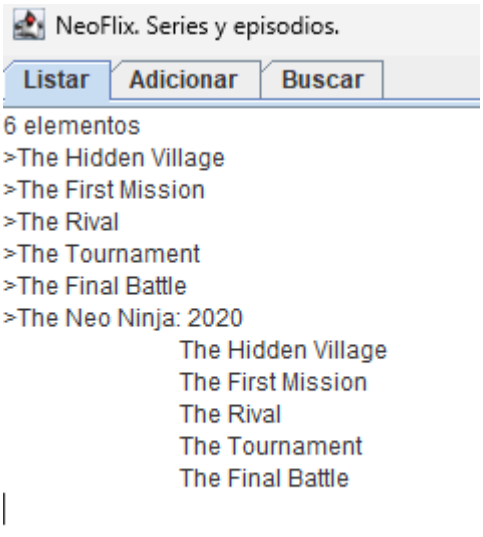
(a) ¿Qué sucede ahora? Capturen la pantalla.

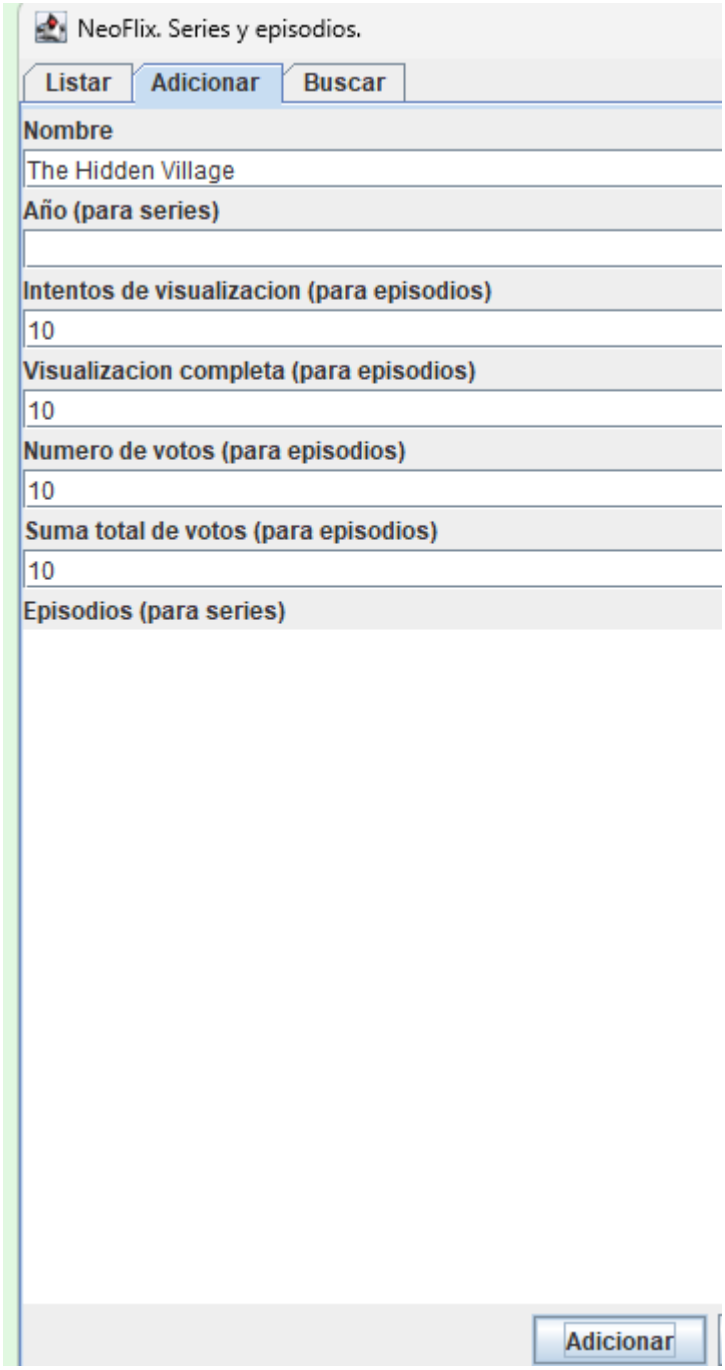


Ahora no deja añadir contenido repetido.

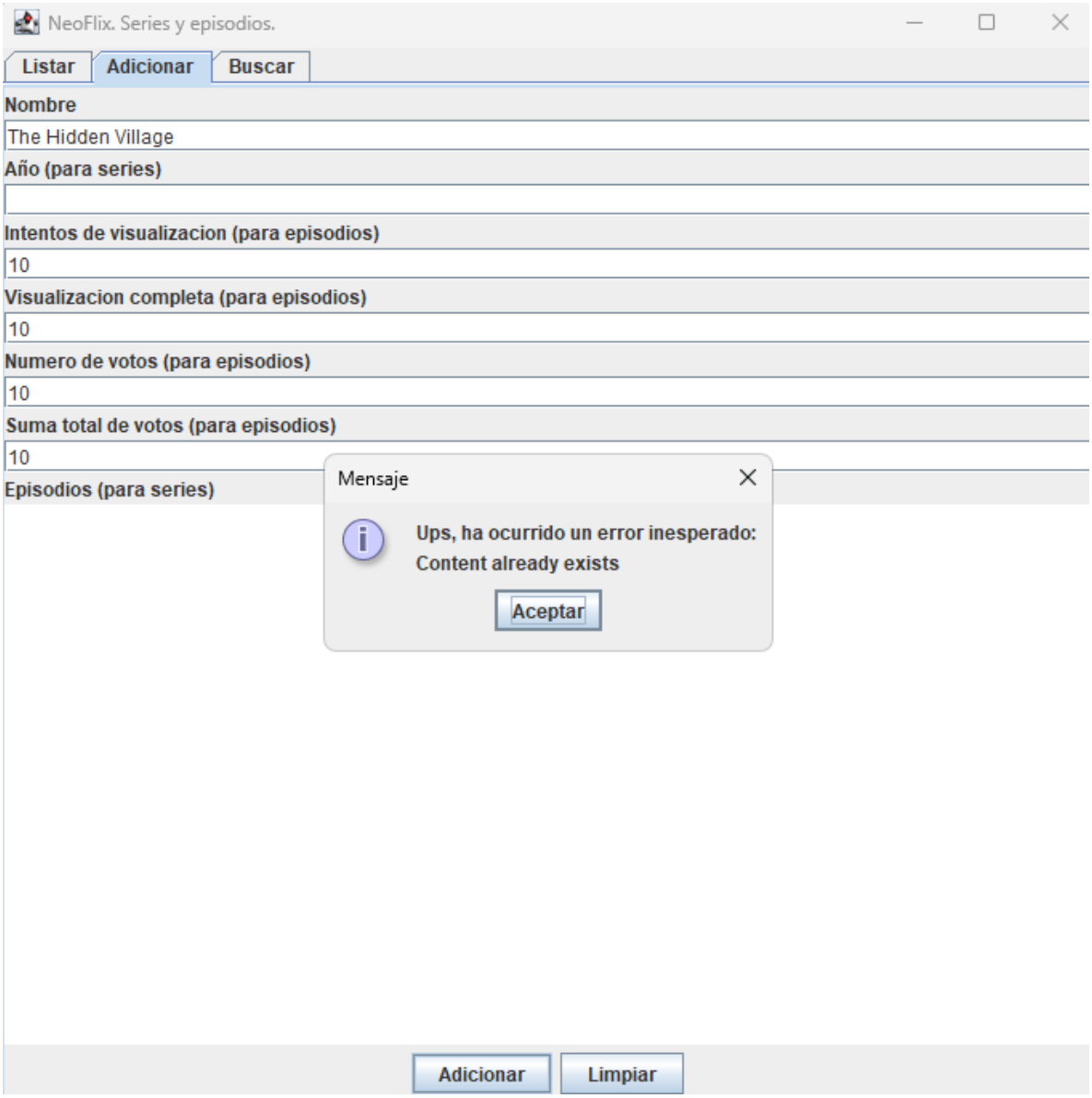
Y si lo intentamos manualmente:

Si se tiene originalmente:

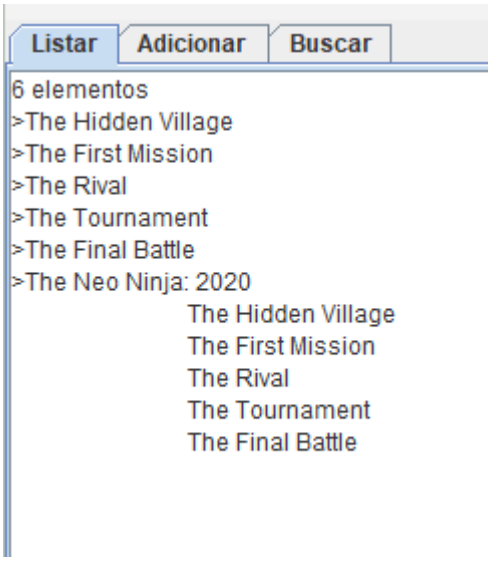




A la hora de darle añadir:



A la hora de listar:



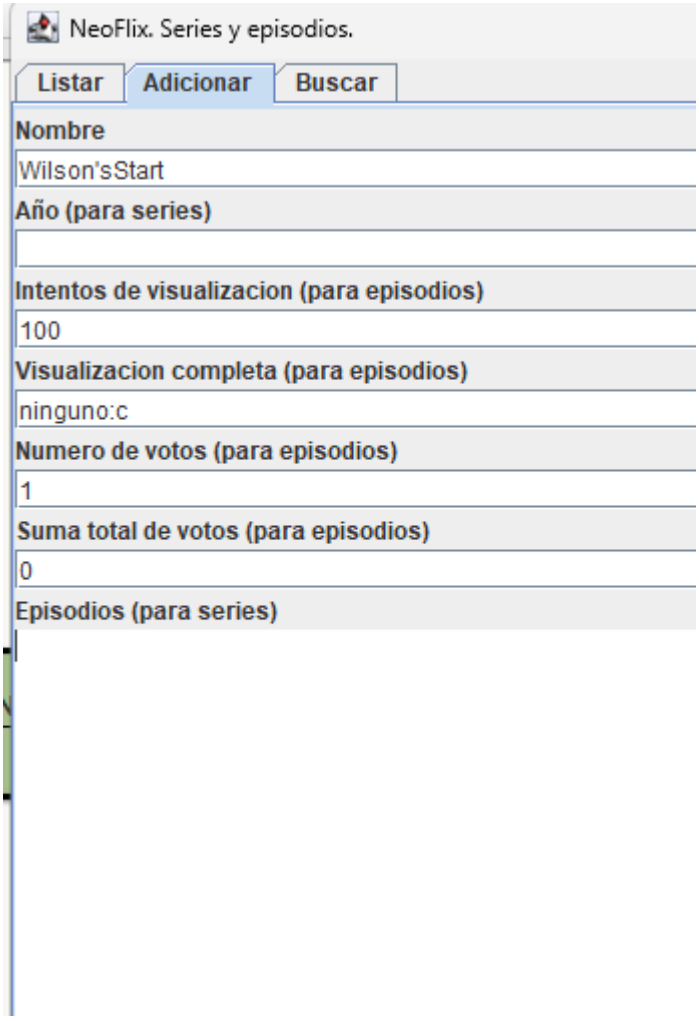
Queda exactamente igual

(ii) ¿Y si los valores numéricos no son numéricos?

1. Propongan una prueba de aceptación que genere el fallo.

La prueba de aceptación es la siguiente:

Se quiere crear un episodio:





(a) ¿Se generó alguna excepción?

Si

(b) ¿Por qué?

Porque no detecta que toda la cadena pueda ser transformada en un número cuando hace:

```
Integer.parseInt(attempts), Integer.parseInt(completed), Integer.parseInt(votes), Integer.parseInt(sumVotes));
```

2. Analicen el diseño dinámico (diagrama de secuencia). Para hacer la funcionalidad robusta:

(a) ¿Qué método debería lanzar la excepción?

Debería crearse un nuevo método dentro de NeoFlix que se encargue de transformarlos en enteros, y en dado caso lance una excepción si detecta que no son 100% numeros

(b) ¿Qué métodos deberían propagarla?

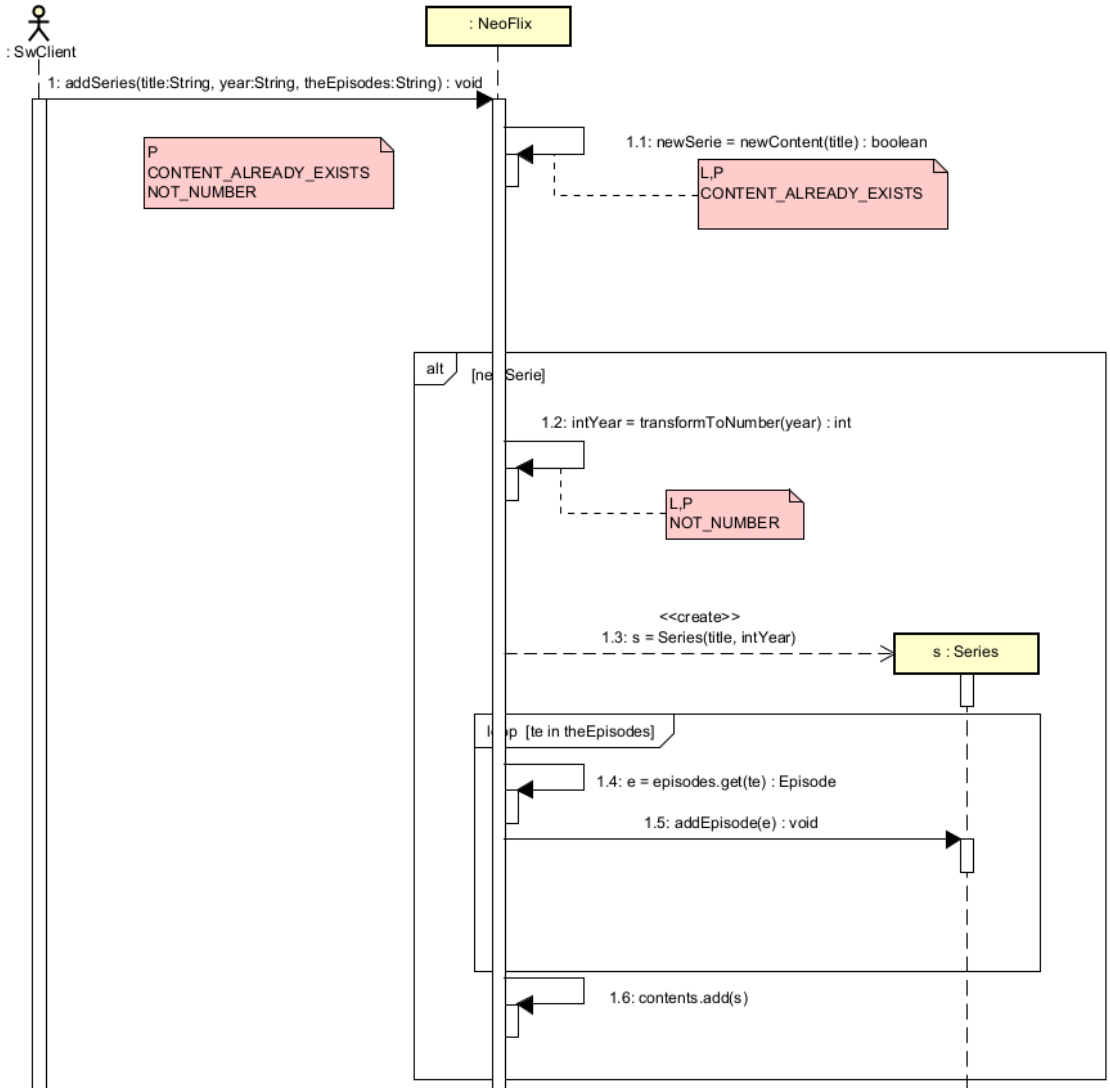
Ese mismo método que menciono en el inciso a.

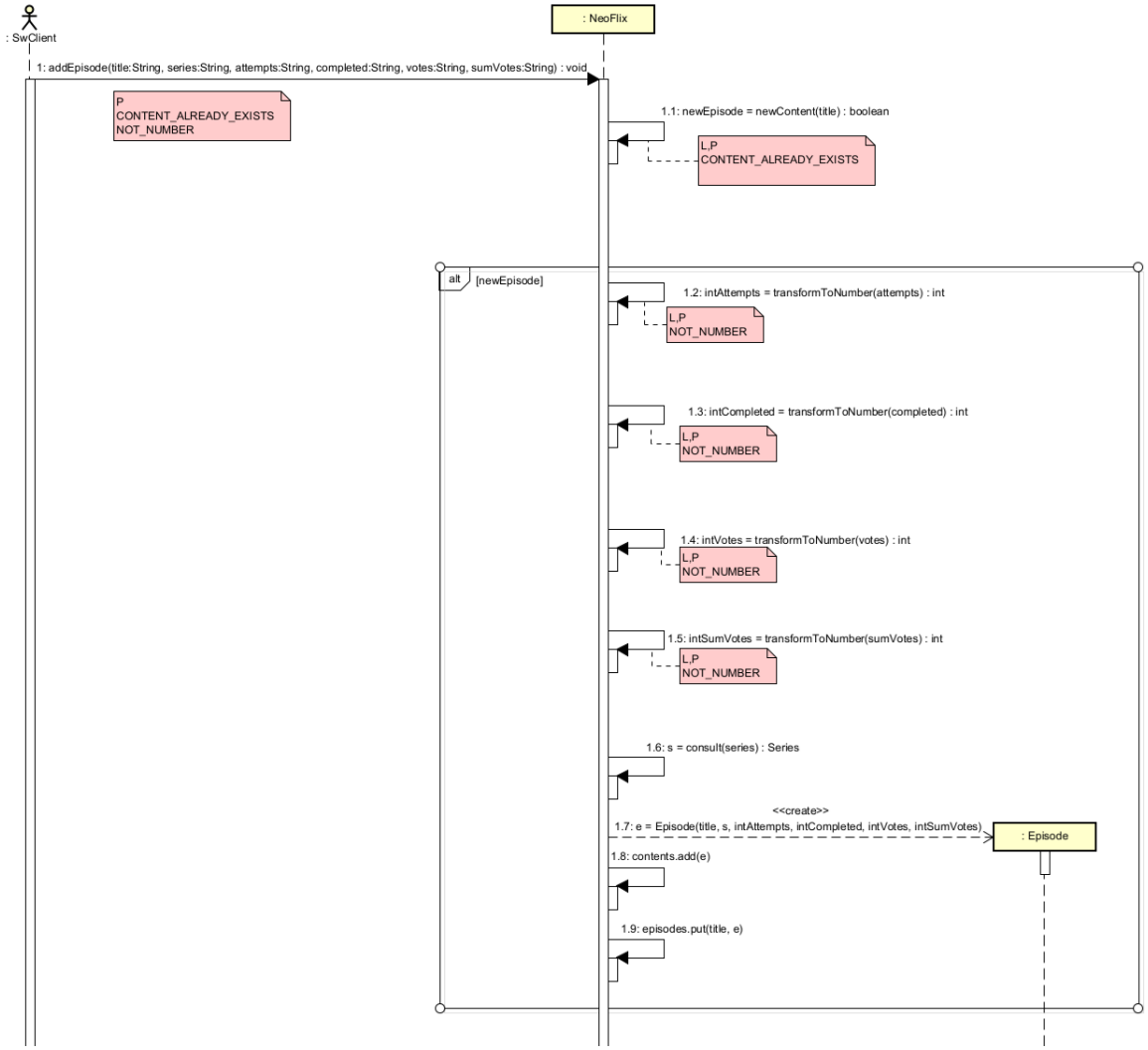
(c) ¿Qué método debería atenderla? Expliquen claramente.

Debería atenderla la parte de NeoFlixGUI y mostrarle al usuario que algo falló con respecto a la digitación de los npumeros

3. Construya la solución propuesta. Incluyan las excepciones en el diseño y presente evidencia de las pruebas de unidad: una ejecución normal y una para cada excepción definida.

Para los diagramas de secuencia:


: SwClient
: NeoFlix
1: addSeries(title:String, year:String, theEpisodes:String) : void
P
CONTENT_ALREADY_EXISTS
NOT_NUMBER
1.1: newSerie = newContent(title) : boolean
L,P
CONTENT_ALREADY_EXISTS
alt
[ne Serie]
1.2: intYear = transformToNumber(year) : int
L,P
NOT_NUMBER
<<create>>
1.3: s = Series(title, intYear)
s : Series
[te in theEpisodes]
1.4: e = episodes.get(te) : Episode
1.5: addEpisode(e) : void
1.6: contents.add(s)



Para el tema de NeoFlixException es necesario crear una nueva excepción:

```java
/**
 * Represent the NeoFlixException Class
 *
 * @author Wilson Mendivelso - David Garzon
 * @version 2/10/2026
 */
public class NeoFlixException extends Exception
{
    public static String TO_BE_IMPLEMENTED = "To be implemented";
    public static String VALUE_UNKNOWN = "Value unknown";
    public static String DATA_ERROR = "Data error";
    public static String CONTENT_EMPTY="Content empty";
    public static String CONTENT_ALREADY_EXISTS =  "Content already exists";
    public static String NOT_NUMBER = "Some of the content isn't a number";
    /**
     * Constructor for objects of class NeoFlixException
     * @param message of Exception
     */
    public NeoFlixException(String message)
    {
        super(message);
    }
}
```

Además fue necesario crear ese nuevo método que se nombra en el diagrama de secuencia:

```java
/**
 * Transforms a String to an int
 * @param transform is the String to transform
 * @return the int
 * @throws NeoFlixException - NOT_NUMBER
 */
private int transformToNumber(String transform) throws NeoFlixException{
    int return_ = 0;
    try{
        return_ = Integer.parseInt(transform);
    }
    catch(java.lang.NumberFormatException e){
        throw new NeoFlixException(NeoFlixException.NOT_NUMBER);
    }
    return return_;
}
```

Además de modificar addEpisode:

```java
/**
* Adds a new episode.
* @param title
* @param series
* @param attempts
* @param completed
* @param votes
* @param sumVotes
* @throws CONTENT_ALREADY_EXISTS, if content already exists
* @throws NOT_NUMBER if some of the content that must be a number isn't
*/
public void addEpisode(String title, String series, String attempts, String completed, String votes, String sumVotes) throws NeoFlixException{
    boolean newEpisode = newContent(title);
    int intAttempts = transformToNumber(attempts);
    int intCompleted = transformToNumber(completed);
    int intVotes = transformToNumber(votes);
    int intSumVotes = transformToNumber(sumVotes);
    Series s = (Series) consult(series);
    Episode e=new Episode(title,s, intAttempts, intCompleted, intVotes, intSumVotes);
    contents.add(e);
    episodes.put(title.toUpperCase(),e);

}
```

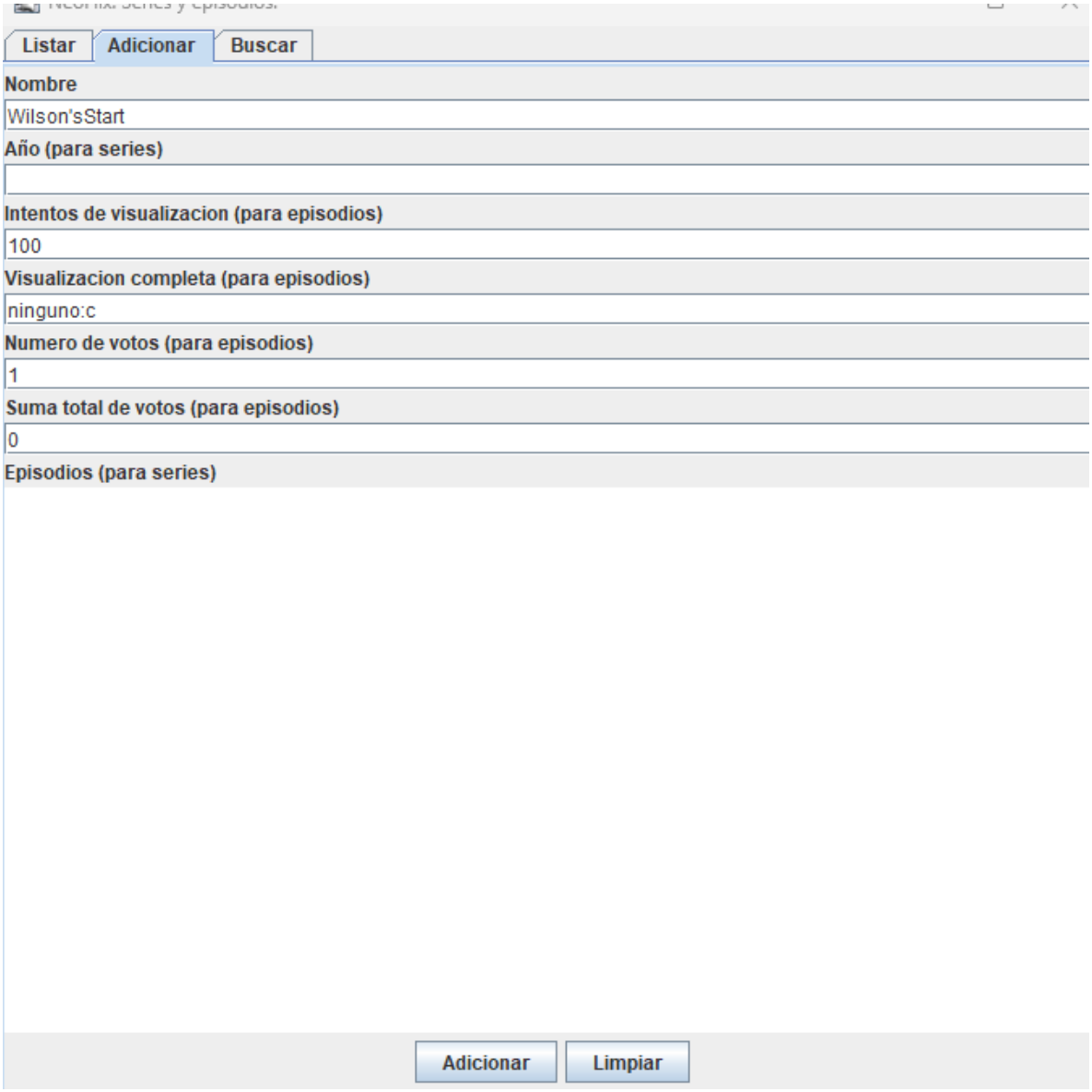
Para addSeries:

```java
/**
 * Add a new series
 * @param title
 * @param year
 * @param theEpisodes the titles of the episodes separated by newlines
 * @throws CONTENT_ALREADY_EXISTS, if content already exists
 * @throws NOT_NUMBER if some of the content that must be a number isn't
*/
public void addSeries(String title, String year, String theEpisodes) throws NeoFlixException{
    boolean newSerie = false;
    newSerie = newContent(title);
    int intYear = transformToNumber(year);
    Series s = new Series(title,intYear);
    String [] aEpisodes= theEpisodes.split("\n");
    for (String te : aEpisodes){
        s.addEpisode(episodes.get(te.toUpperCase()));
    }
    contents.add(s);
}
```

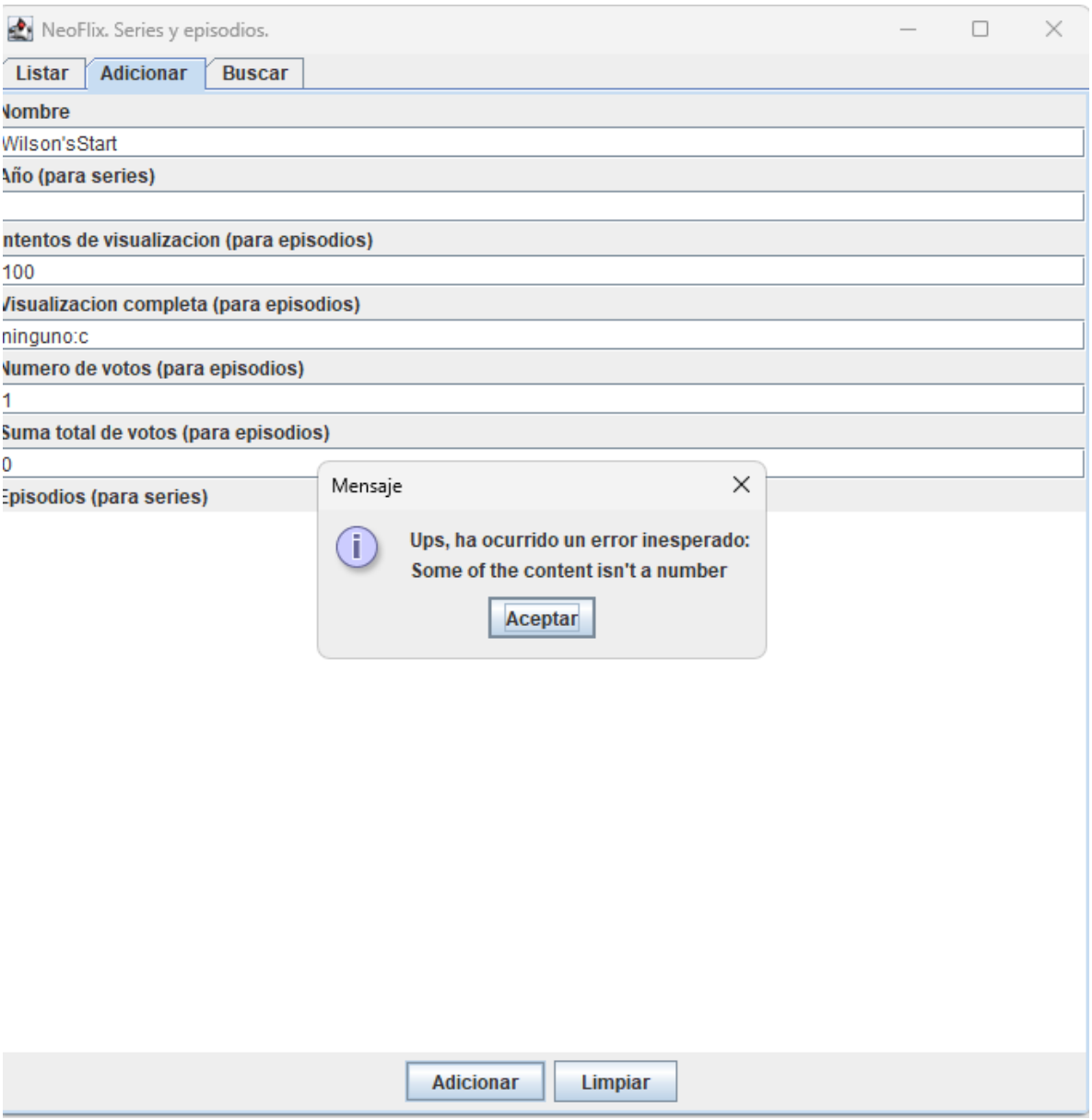
4. Ejecuten nuevamente la aplicación con el caso de aceptación propuesto en 1.

(a) ¿Qué sucede ahora? Capturen la pantalla.

Si ahora hago:

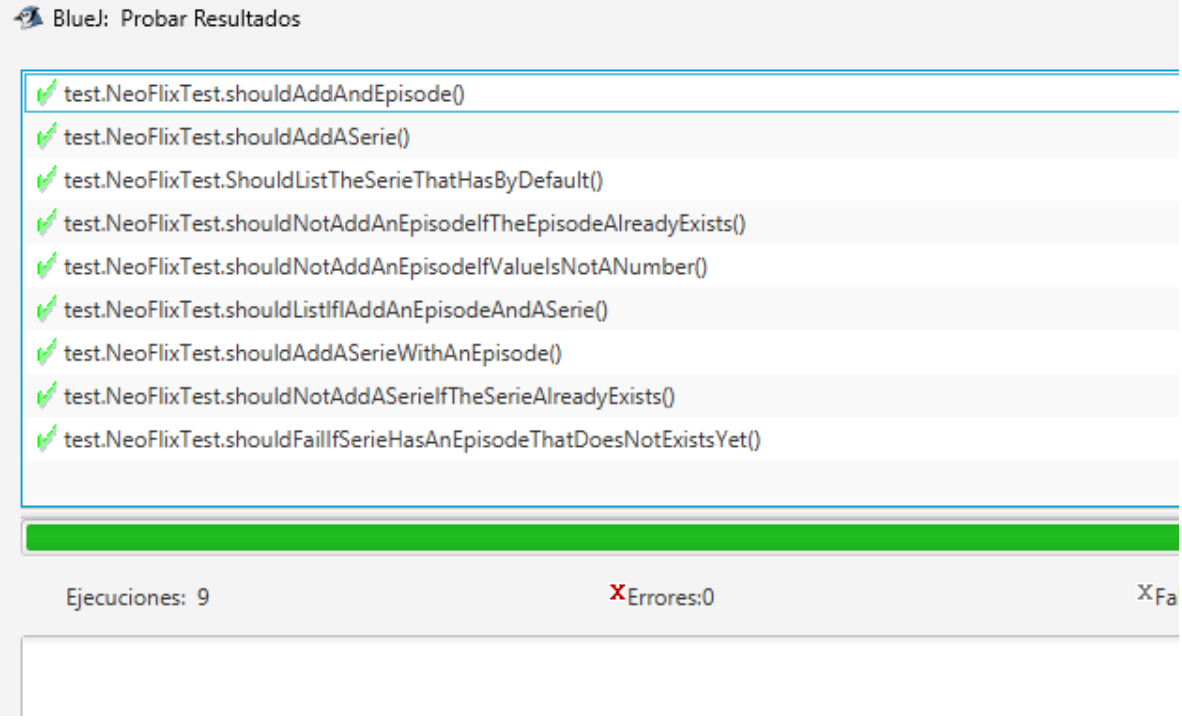


Entonces sale este error:



Además de que se creó una prueba unitaria:

```java
/**
 * Should not add an episode if some numeric value is not a number.
 */
@Test
public void shouldNotAddAnEpisodeIfValueIsNotANumber(){
    NeoFlix neoFlix = new NeoFlix();
    try{
        neoFlix.addEpisode("Wilson'sStart", "?", "100", "ninguno:c", "1", "0");
        fail("It didn't throw an exception");
    }
    catch(NeoFlixException e){
        assertEquals(NeoFlixException.NOT_NUMBER, e.getMessage());
    }
}
```



(iii) ¿Y si los valores no son los esperados? (suma de votos>10*votos o terminados>intentos)

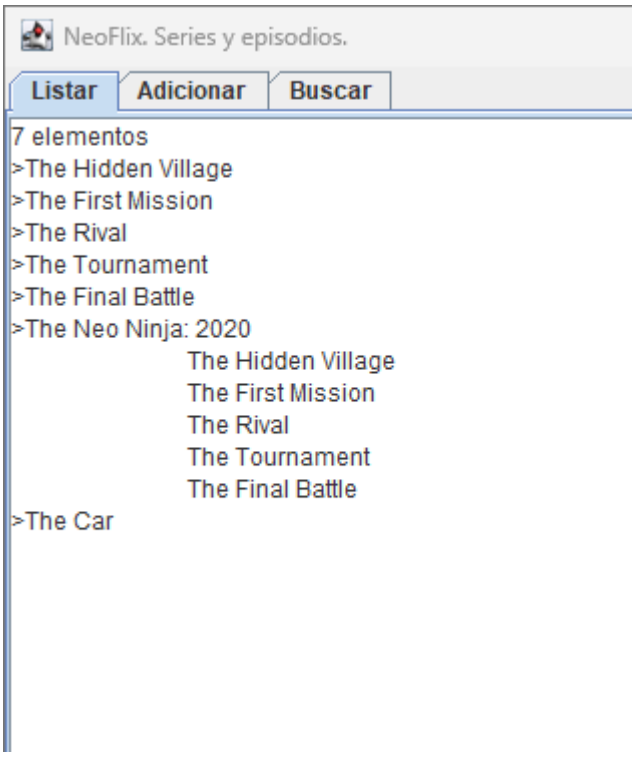
1. Propongan una prueba de aceptación que genere el fallo.

La prueba de aceptación es la siguiente:

Se quiere adicionar el siguiente capitulo



Y al listar:



(a) ¿Se generó alguna excepción?

No

(b) ¿Por qué?

Porque, no existe alguna excepción que evite el ingreso de ese tipo de datos.

2. Analicen el diseño dinámico (diagrama de secuencia). Para hacer la funcionalidad robusta:

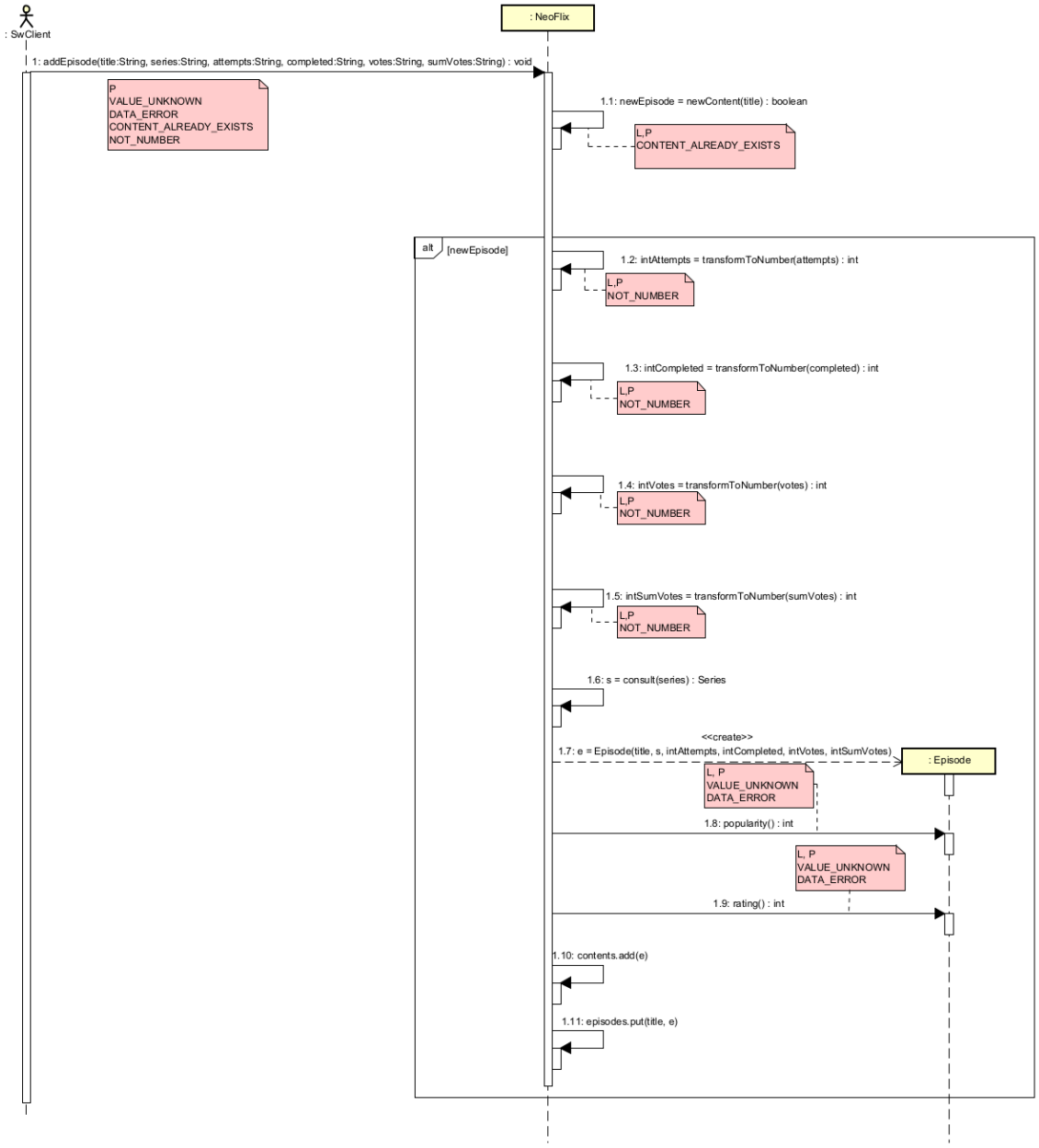
(a) ¿Qué método debería lanzar la excepción?

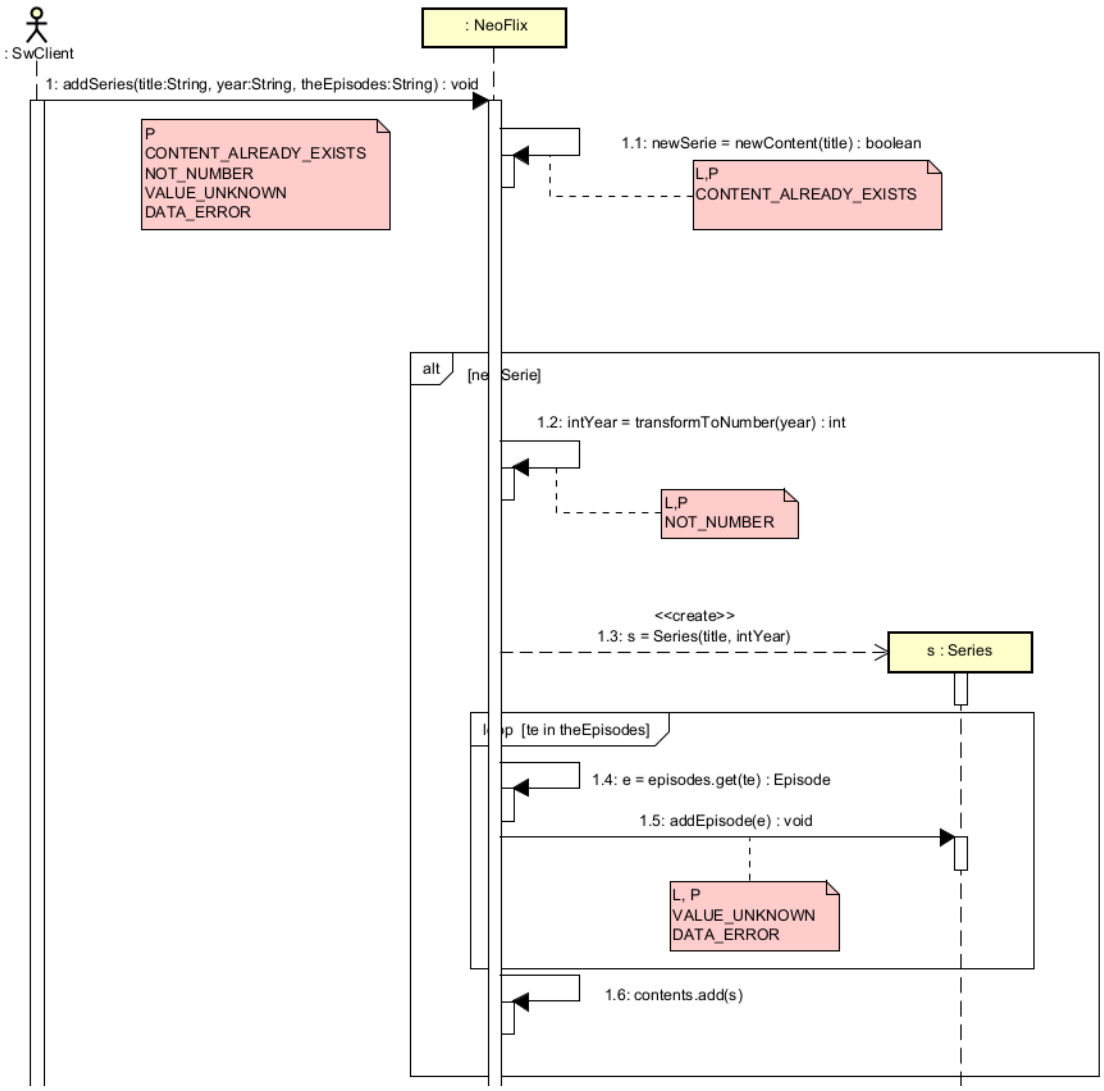
Debería lanzarla rating() y popularity(), comprobando si se puede calcular la popularidad y el rating.

(b) ¿Qué métodos deberían propagarla?
Deberían propagarla addEpisode y addSerie, además de que se puede aprovechar la función de popularity() y rating() que viene desde Audiovisual.

(c) ¿Qué método debería atenderla? Expliquen claramente.
Dentro del domain no sería necesario atenderla, puesto que solo sería necesario propagarla. La idea es que, dentro del GUI se mostrará alguna advertencia o algo referente a que no se puede añadir por x o y cosa.

3. Construya la solución propuesta. Incluyan las excepciones en el diseño y presente evidencia de las pruebas de unidad: una ejecución normal y una para cada excepción definida.


: SwClient
: NeoFlix
1: addEpisode(title:String, series:String, attempts:String, completed:String, votes:String, sumVotes:String) : void
P
VALUE_UNKNOWN
DATA_ERROR
CONTENT_ALREADY_EXISTS
NOT_NUMBER
1.1: newEpisode = newContent(title) : boolean
L,P
CONTENT_ALREADY_EXISTS
alt
[newEpisode]
1.2: intAttempts = transformToNumber(attempts) : int
L,P
NOT_NUMBER
1.3: intCompleted = transformToNumber(completed) : int
L,P
NOT_NUMBER
1.4: intVotes = transformToNumber(votes) : int
L,P
NOT_NUMBER
1.5: intSumVotes = transformToNumber(sumVotes) : int
L,P
NOT_NUMBER
1.6: s = consult(series) : Series
<<create>>
1.7: e = Episode(title, s, intAttempts, intCompleted, intVotes, intSumVotes)
: Episode
L, P
VALUE_UNKNOWN
DATA_ERROR
1.8: popularity() : int
L, P
VALUE_UNKNOWN
DATA_ERROR
1.9: rating() : int
1.10: contents.add(e)
1.11: episodes.put(title, e)



Para addEpisode:

```java
/**
* Adds a new episode.
* @param title
* @param series
* @param attempts
* @param completed
* @param votes
* @param sumVotes
* @throws CONTENT_ALREADY_EXISTS, if content already exists
* @throws NOT_NUMBER if some of the content that must be a number isn't
* @throws VALUE_UNKNOWN if some of the values are unknown.
* @throws DATA_ERROR if the data presents an error
*/
public void addEpisode(String title, String series, String attempts, String completed, String votes, String sumVotes) throws NeoFlixException{
    boolean newEpisode = newContent(title);
    int intAttempts = transformToNumber(attempts);
    int intCompleted = transformToNumber(completed);
    int intVotes = transformToNumber(votes);
    int intSumVotes = transformToNumber(sumVotes);
    Series s = (Series) consult(series);
    Episode e=new Episode(title,s, intAttempts, intCompleted, intVotes, intSumVotes);
    e.popularity();
    e.rating();
    contents.add(e);
    episodes.put(title.toUpperCase(),e);

}
```

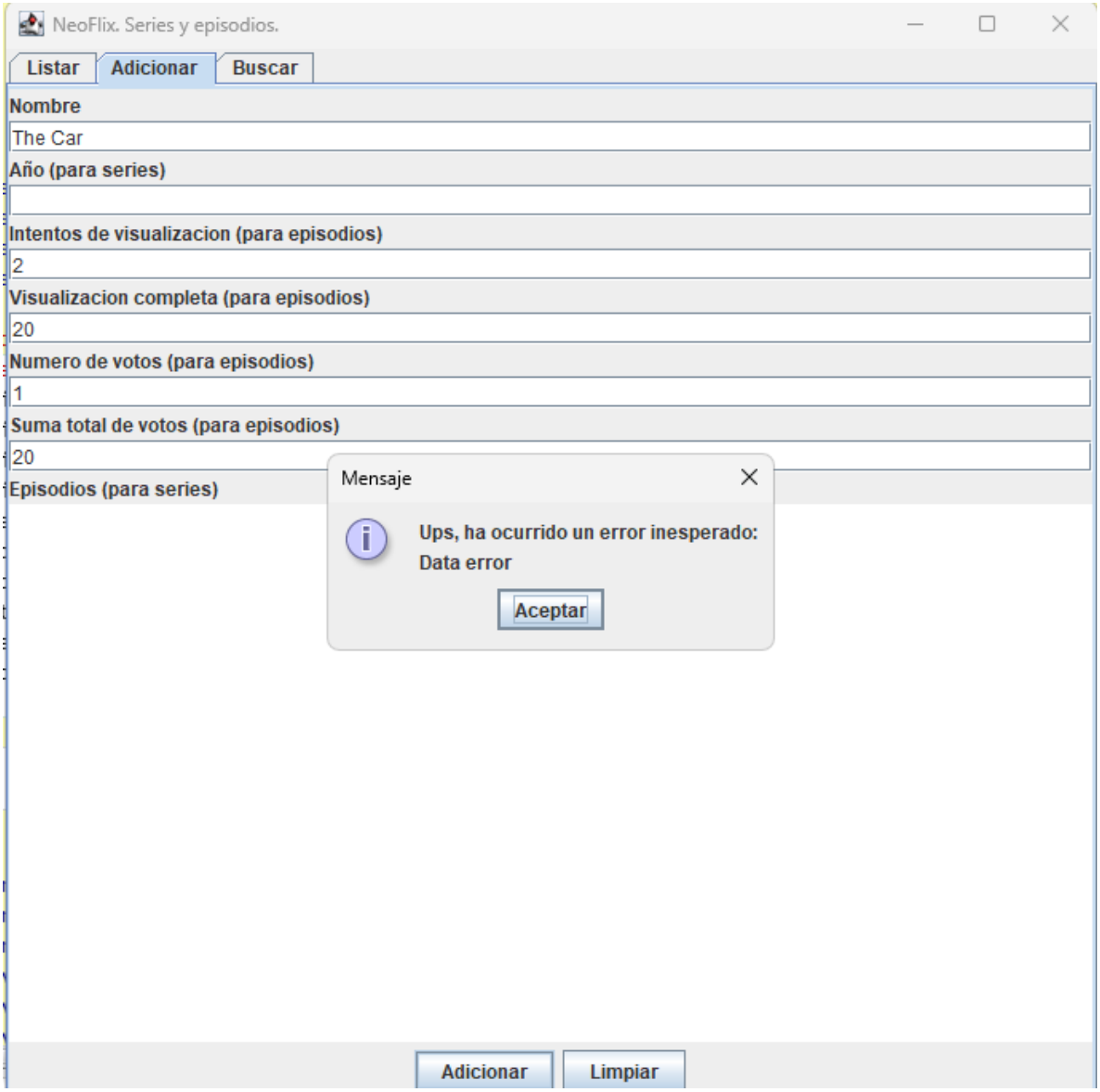
Para addSeries:

```java
/**
 * Add a new series
 * @param title
 * @param year
 * @param theEpisodes the titles of the episodes separated by newlines
 * @throws CONTENT_ALREADY_EXISTS, if content already exists
 * @throws NOT_NUMBER if some of the content that must be a number isn't
 * @throws VALUE_UNKNOWN if some of the values are unknown.
 * @throws DATA_ERROR if the data presents an error
*/
public void addSeries(String title, String year, String theEpisodes) throws NeoFlixException{
    boolean newSerie = false;
    newSerie = newContent(title);
    int intYear = transformToNumber(year);
    Series s = new Series(title,intYear);
    String [] aEpisodes= theEpisodes.split("\n");
    for (String te : aEpisodes){
        s.addEpisode(episodes.get(te.toUpperCase()));
    }
    contents.add(s);
}
```

4. Ejecuten nuevamente la aplicación con el caso de aceptación propuesto en 1.

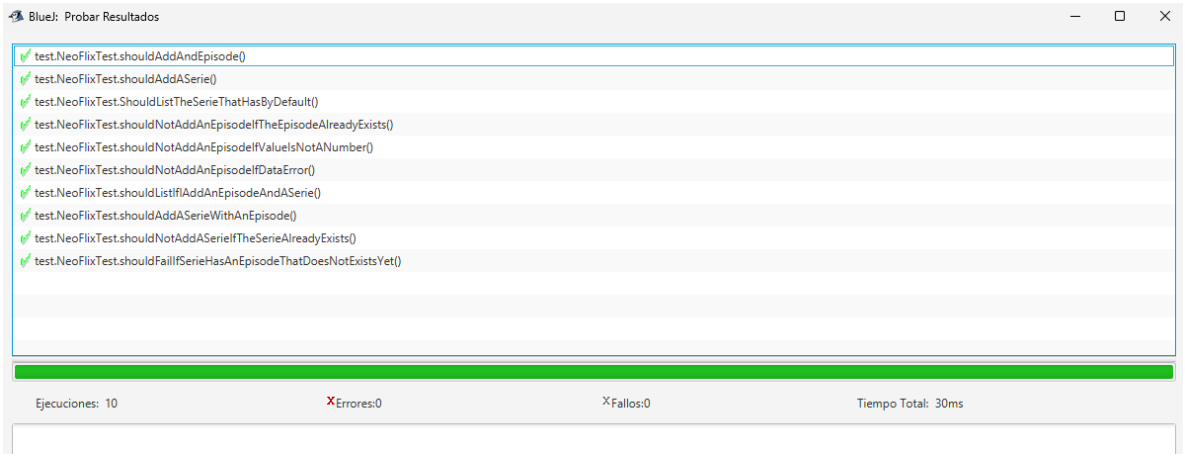
(a) ¿Qué sucede ahora? Capturen la pantalla.

Ahora:



Por último, se creó una prueba unitaria que representa este escenario:

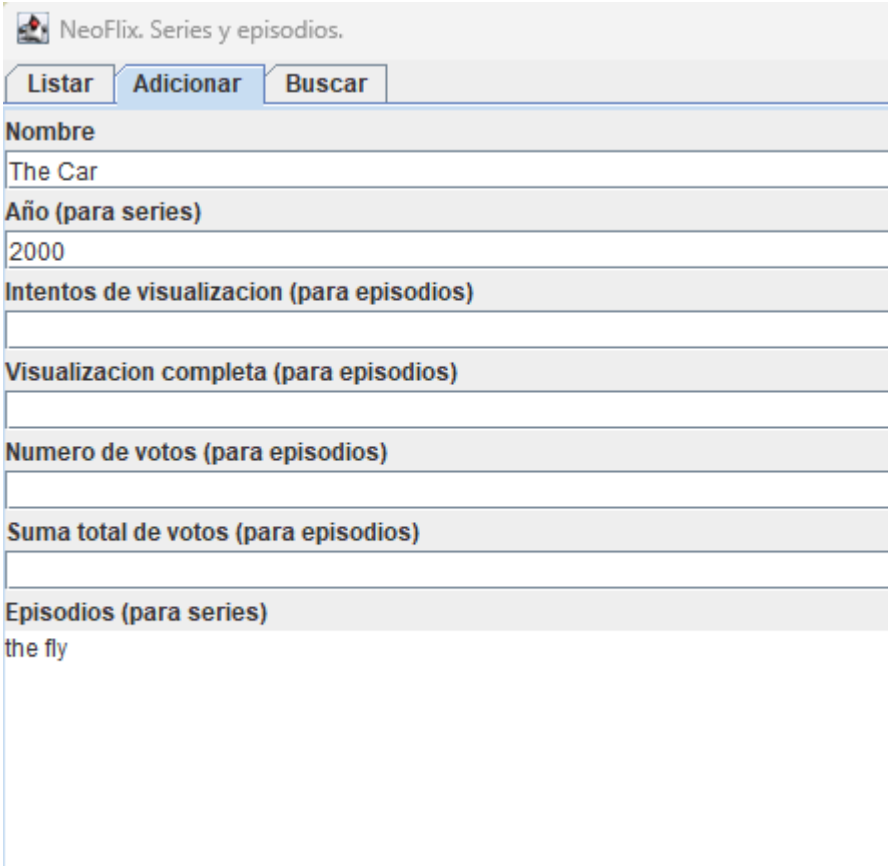
```java
/**
 * Should not add an episode if completed is greater than attempts or sumVotes is greater than 10*votes
 */
@Test
public void shouldNotAddAnEpisodeIfDataError(){
    NeoFlix neoFlix = new NeoFlix();
    try{
        neoFlix.addEpisode("Wilson'sStart", "?", "100", "200", "50", "1000");
        fail("It didn't throw an exception");
    }
    catch(NeoFlixException e){
        assertEquals(NeoFlixException.DATA_ERROR, e.getMessage());
    }
}
```



(iv) Proponga una nueva condición

1. Propongan una prueba de aceptación que genere el fallo.

La prueba que hemos visto que genera fallos es que, si uno intenta añadir un serie con episodios que no existen, tira un fallo, por ejemplo:



Hasta ahí, normal, te deja añadirla, pero a la hora de listarla:

```
Can only enter input while your program is running
Exception in thread "AWT-EventQueue-0" java.lang.NullPointerException: Cannot invoke "domain.Episode.data(boolean)" because "e" is null
        at domain.Series.data(Series.java:143)
        at domain.NeoFlix.data(NeoFlix.java:151)
        at domain.NeoFlix.toString(NeoFlix.java:176)
        at presentation.NeoFlixGUI.actionList(NeoFlixGUI.java:220)
        at presentation.NeoFlixGUI$2.actionPerformed(NeoFlixGUI.java:173)
        at java.desktop/javax.swing.AbstractButton.fireActionPerformed(AbstractButton.java:1972)
        at java.desktop/javax.swing.AbstractButton$Handler.actionPerformed(AbstractButton.java:2314)
```

(a) ¿Se generó alguna excepción?

Si, una NullPointerException

(b) ¿Por qué?

Porque llama una función a un capítulo que no existe. De hecho pasa lo mismo si no añades ningún capítulo a una serie.

2. Analicen el diseño dinámico (diagrama de secuencia). Para hacer la funcionalidad robusta:

(a) ¿Qué método debería lanzar la excepción?

Debería lanzarlo el de addSeries.

(b) ¿Qué métodos deberían propagarla?

Ese mismo método debería propagarla.

(c) ¿Qué método debería atenderla? Expliquen claramente.

Debería atenderla NeoFlixGUI para que el usuario entienda que el episodio no existe.

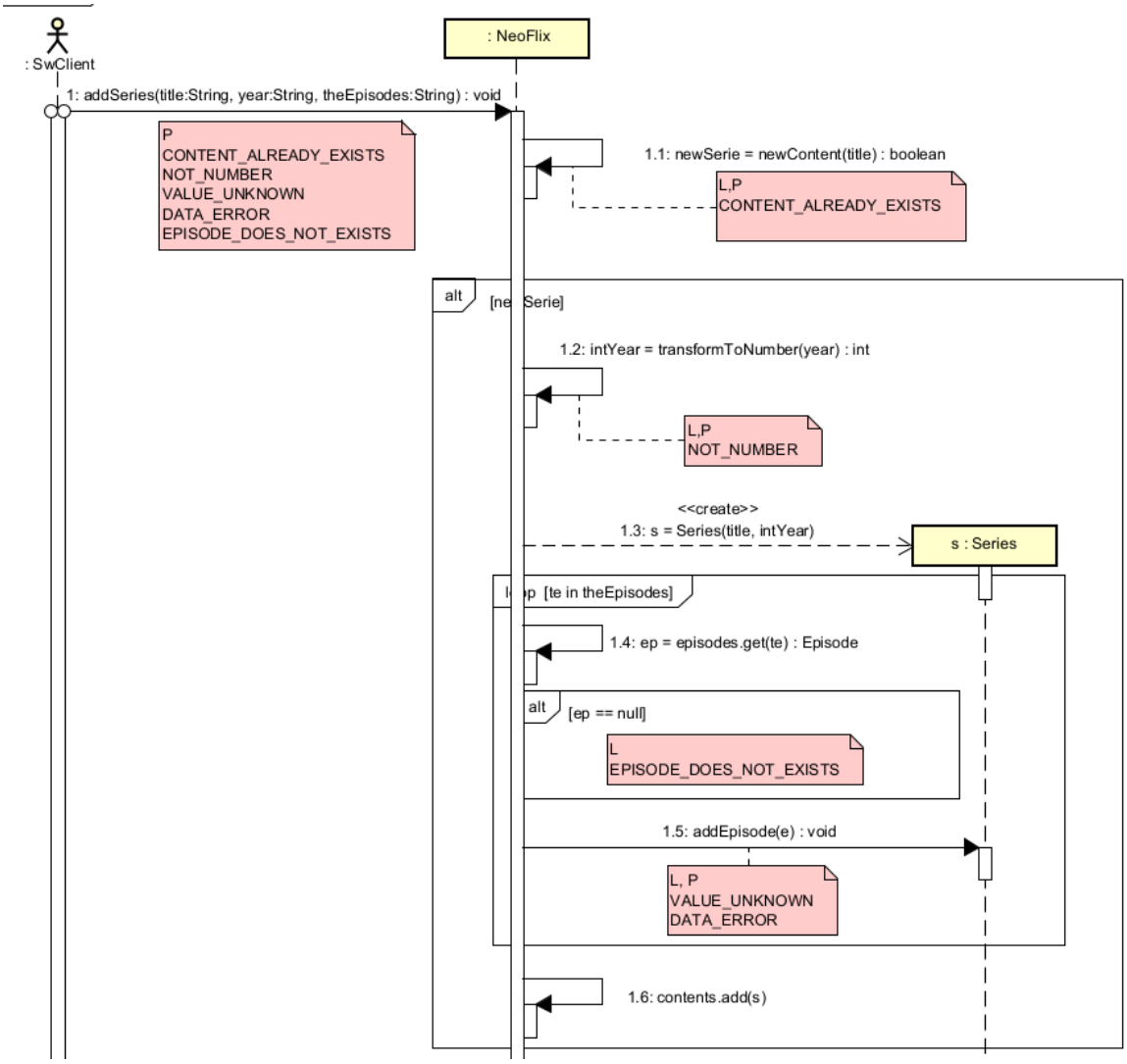
3. Construya la solución propuesta. Incluyan las excepciones en el diseño y presente evidencia de las pruebas de unidad: una ejecución normal y una para cada excepción definida.

Primero fue necesario añadir ese nuevo mensaje para las excepciones, por si se llega a dar:

```java
/**
 * Represent the NeoFlixException Class
 *
 * @author Wilson Mendivelso - David Garzon
 * @version 2/10/2026
 */
public class NeoFlixException extends Exception
{
    public static String TO_BE_IMPLEMENTED = "To be implemented";
    public static String VALUE_UNKNOWN = "Value unknown";
    public static String DATA_ERROR = "Data error";
    public static String CONTENT_EMPTY="Content empty";
    public static String CONTENT_ALREADY_EXISTS =  "Content already exists";
    public static String NOT_NUMBER = "Some of the content isn't a number";
    public static String EPISODE_DOES_NOT_EXISTS = "The episode doesn't exists";

    /**
     * Constructor for objects of class NeoFlixException
     * @param message of Exception
     */
    public NeoFlixException(String message)
    {
        super(message);
    }
}
```

Por otro lado fue necesario modificar el astah:



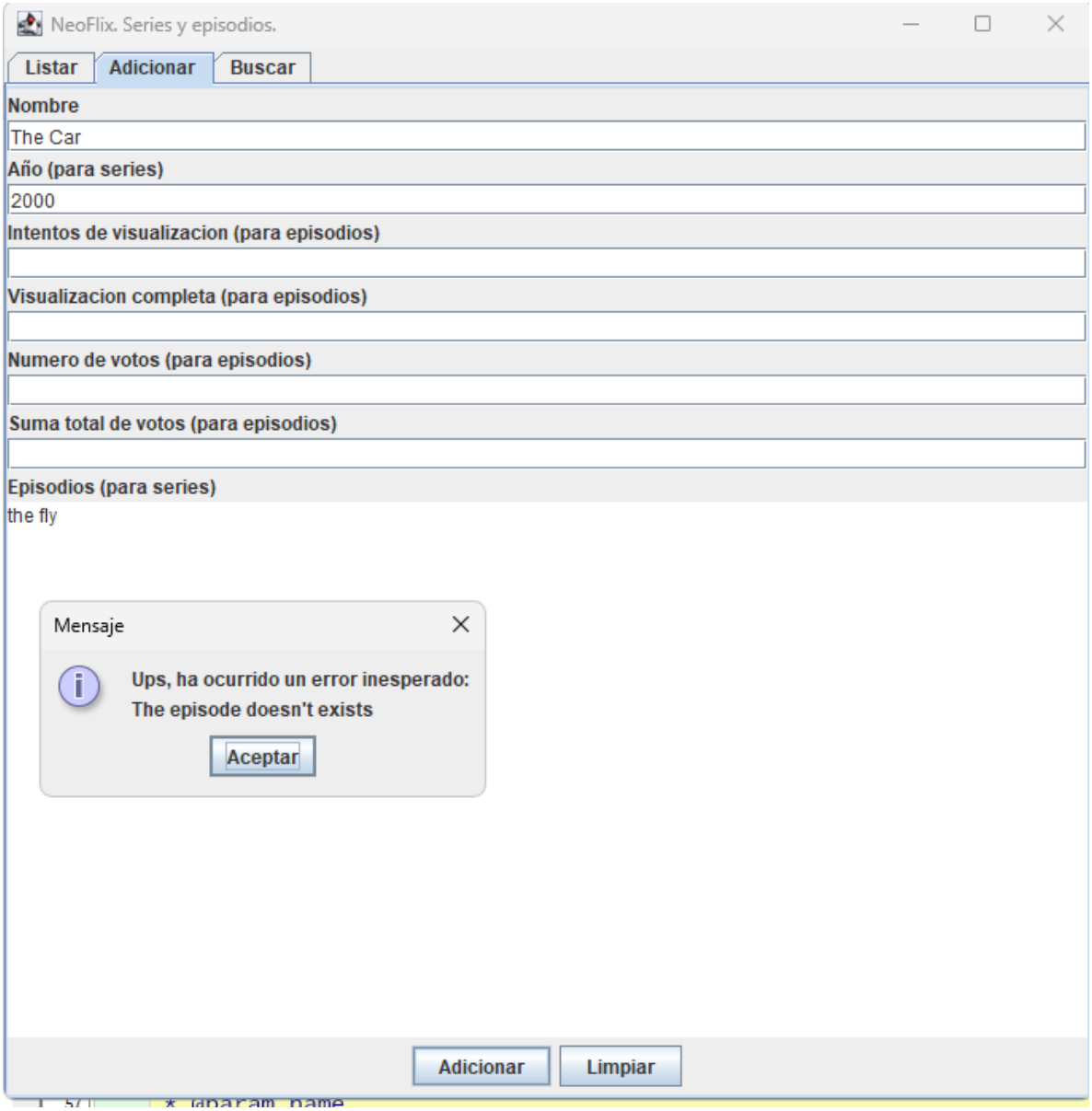
Para luego mirar el código:

```java
/**
 * Add a new series
 * @param title
 * @param year
 * @param theEpisodes the titles of the episodes separated by newlines
 * @throws CONTENT_ALREADY_EXISTS, if content already exists
 * @throws NOT_NUMBER if some of the content that must be a number isn't
 * @throws VALUE_UNKNOWN if some of the values are unknown.
 * @throws DATA_ERROR if the data presents an error
 * @throws EPISODE_DOES_NOT_EXISTS if the episode doesn't exists
*/
public void addSeries(String title, String year, String theEpisodes) throws NeoFlixException{
    boolean newSerie = false;
    newSerie = newContent(title);
    int intYear = transformToNumber(year);
    Series s = new Series(title,intYear);
    String [] aEpisodes= theEpisodes.split("\n");
    for (String te : aEpisodes){
        Episode ep = episodes.get(te.toUpperCase());
        if (ep == null){
            throw new NeoFlixException(NeoFlixException.EPISODE_DOES_NOT_EXISTS);
        }
        s.addEpisode(episodes.get(te.toUpperCase()));
    }
    contents.add(s);
}
```

4. Ejecuten nuevamente la aplicación con el caso de aceptación propuesto en 1.

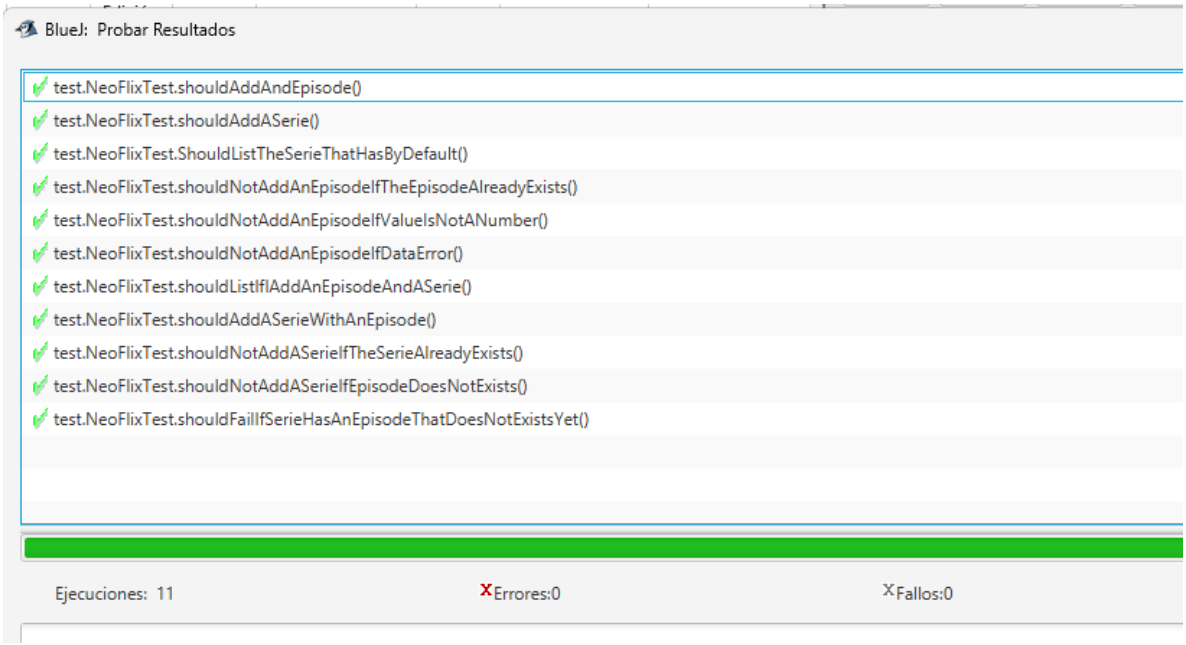
(a) ¿Qué sucede ahora? Capturen la pantalla.


NeoFlix. Series y episodios.
Listar
Adicionar
Buscar
Nombre
The Car
Año (para series)
2000
Intentos de visualizacion (para episodios)
Visualizacion completa (para episodios)
Numero de votos (para episodios)
Suma total de votos (para episodios)
Episodios (para series)
the fly
Adicionar
Limpiar



Además se creó una prueba unitaria:

```java
/**
 * Should not add a serie if an episode does not exists.
 */
@Test
public void shouldNotAddASerieIfEpisodeDoesNotExists(){
    NeoFlix neoFlix = new NeoFlix();
    try{
        neoFlix.addSeries("Wilson's Adventure", "2026", "Wilson's Start");
        fail("It didn't throw an exception");
    }
    catch(NeoFlixException e){
        assertEquals(NeoFlixException.EPISODE_DOES_NOT_EXISTS, e.getMessage());
    }
}
```



D Registrando errores. Consultar por patrones. ¡Error!

El objetivo es aprender a manejar errores no esperados.

1. Consulten en un contenido que inicia por T.

a) ¿Qué sucede?
Tira un error.

b) ¿Qué error existe? Capturen el resultado.
Tira el siguiente error: java.lang.IndexOutOfBoundsException:

```
Exception in thread "AWT-EventQueue-0" java.lang.IndexOutOfBoundsException: Index 6 out of bounds for length 6
```

c) ¿Quién debe conocer y quien NO debe conocer esta información?
El desarrollador debe conocer esta información, el usuario final no debe conocerla.

2. Revisen el código asociado a consultar en la capa de presentación y la capa de dominio.

a) ¿Qué método es responsable en la capa de presentación?
En la capa de presentación se encarga:

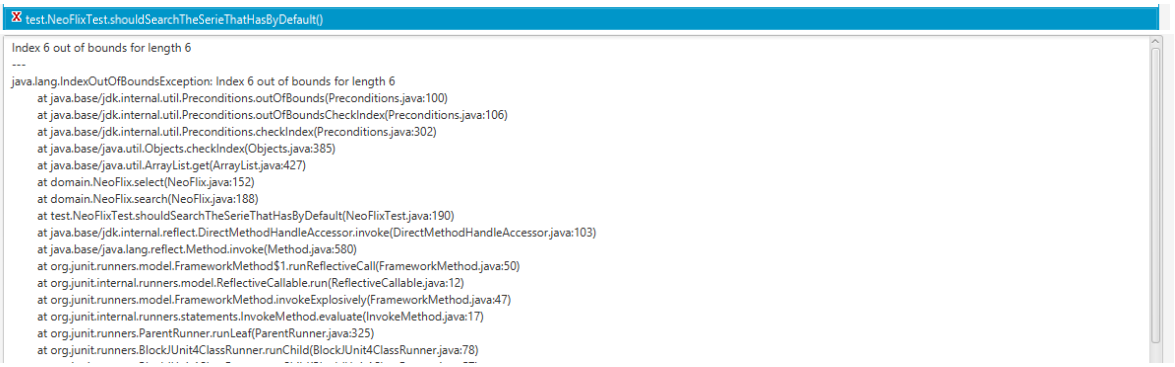
```
private void actionSearch(){
    String patronBusqueda=textSearch.getText();
    String answer = "";
    if(patronBusqueda.length() > 0) {
        answer = neoFlix.search(patronBusqueda);
    }
    textResults.setText(answer);
}
```

b) ¿Qué método en la capa de dominio?
En la capa de dominio se hace con:

```
/**
 * Return the data of episodes with a prefix
 * @param prefix
 * @return
 */
public String search(String prefix){
    return data(select(prefix),true);
}
```

3. Realicen ingeniería reversa para la capa de dominio para consultar. No modifiquen el código. Capturen los resultados de las pruebas de unidad.

```java
/**
 * Should search the serie that has by default.
 */
@Test
public void shouldSearchTheSerieThatHasByDefault(){
    NeoFlix neoFlix = new NeoFlix();
    String esperado = "6 elementos\n"+
                      ">The Hidden Village ( 60 - 8 )\n"+
                      ">The First Mission*** Indicadores incompletos\n"+
                      ">The Rival ( 60 - 8 )\n"+
                      ">The Tournament*** Indicadores incompletos\n"+
                      ">The Final Battle*** Indicadores incompletos\n"+
                      ">The Neo Ninja: 2020 ( 24 - 8 )"+
                          "\n\tThe Hidden Village ( 60 - 8 )"+
                          "\n\tThe First Mission*** Indicadores incompletos"+
                          "\n\tThe Rival ( 60 - 8 )"+
                          "\n\tThe Tournament*** Indicadores incompletos"+
                          "\n\tThe Final Battle*** Indicadores incompletos\n";
    assertEquals(esperado, neoFlix.search("T"));
}
```



4. Exploren el método record de la clase Log

```java
public static void record(Exception e){
    try{
        Logger logger=Logger.getLogger(name);
        logger.setUseParentHandlers(false);
        FileHandler file=new FileHandler(name+".log",true);
        file.setFormatter(new SimpleFormatter());
        logger.addHandler(file);
        logger.log(Level.SEVERE,e.toString(),e);
        file.close();
    }catch (Exception oe){
        oe.printStackTrace();
        System.exit(0);
    }
}
```

(a) ¿Qué servicio presta?

Por lo explicado en el laboratorio se encarga de registrar los errores en un archivo de texto, el cual se busca que esté oculto para el usuario, teniendo el error obtenido como parámetro.

5. Analicen el punto adecuado para que EN ESTE CASO se presente un mensaje especial de alerta al usuario, se guarde la información del error en el registro y continúe la ejecución. Expliquen y construyan la solución.

   Primero que todo, es necesario pensar donde se atender este tipo de errores, puesto que se pueden atender desde NeoFlix, pero es mucho mejor atenderlos desde la parte del GUI, porque, si se quiere mostrar una alerta al usuario, allí es el lugar apropiado para hacerlo. Por tanto:

```
private void actionSearch(){
    String patronBusqueda=textSearch.getText();
    String answer = "";
    try{
        if(patronBusqueda.length() > 0) {
            answer = neoFlix.search(patronBusqueda);
        }
        textResults.setText(answer);
    }
    catch (Exception e){
        Log.record(e);

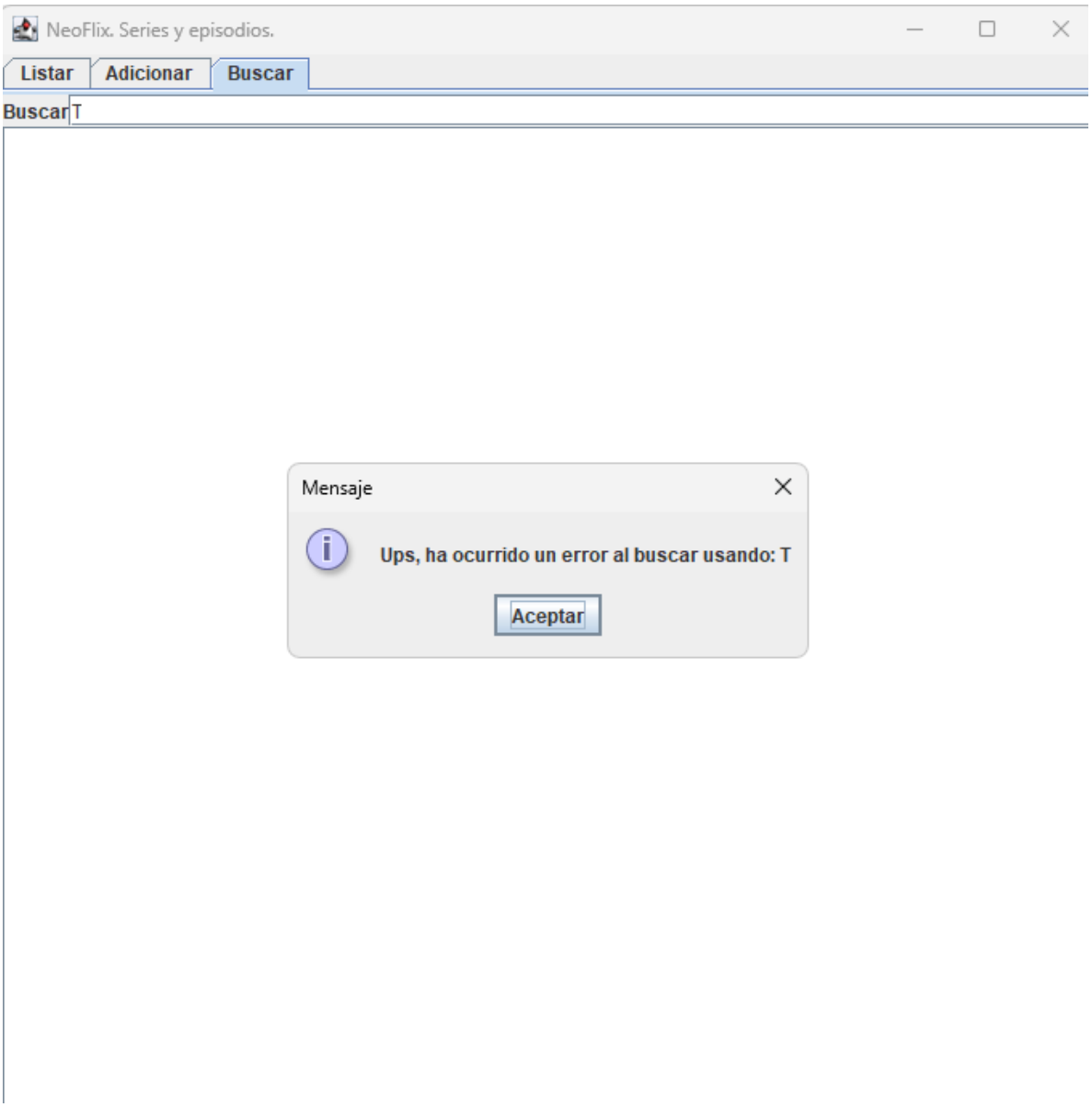
        JOptionPane.showMessageDialog(this, "Ups, ha ocurrido un error al buscar usando: " + patronBusqueda);

        textResults.setText("");
    }

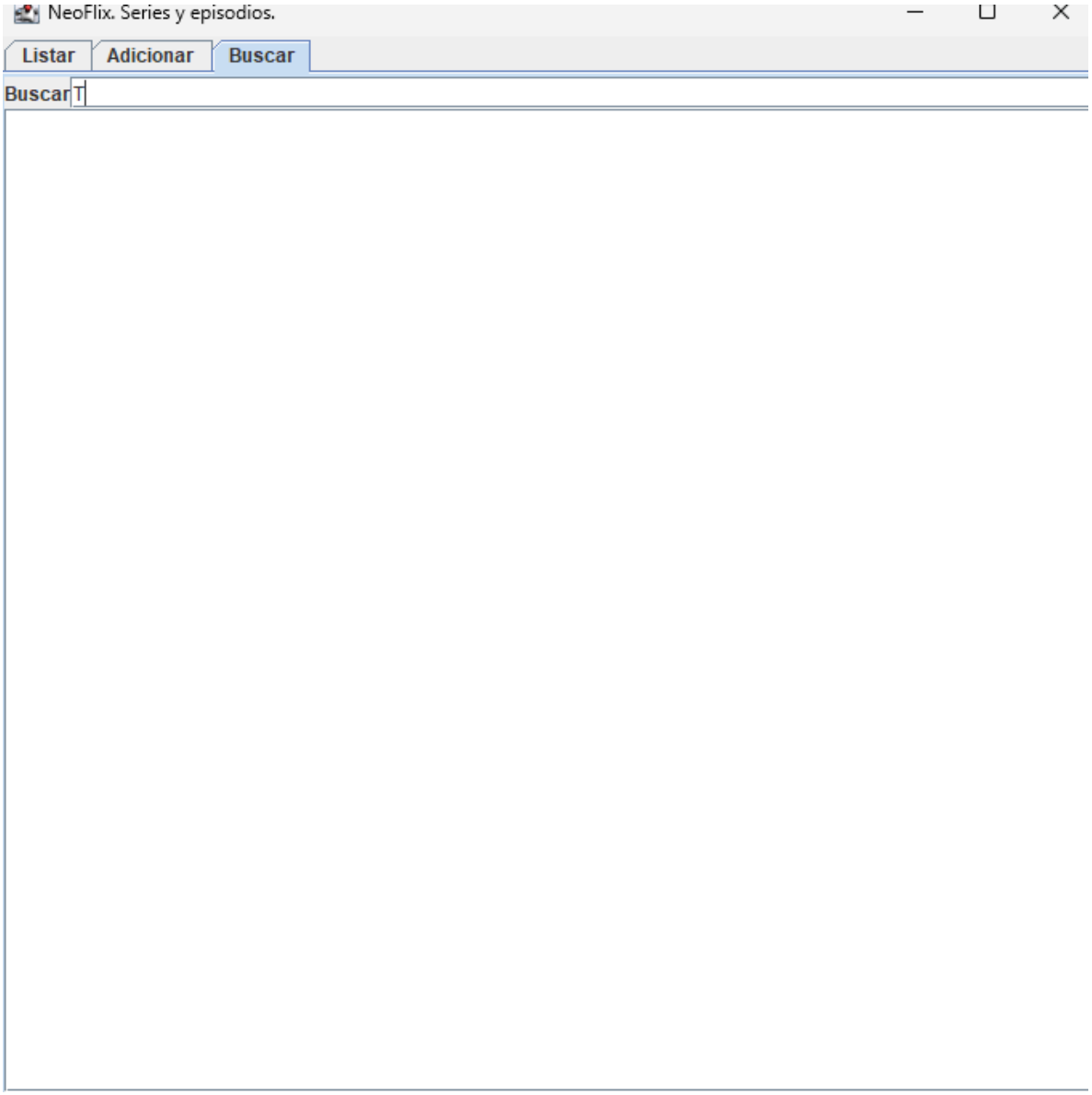
}
```

   El código se resume en:
   Intentar de buscar (como ya se hacia originalmente), pero si algo sale mal, lo atrapa el catch, se hace un record con el nombre de la excepción, se manda un mensaje al usuario diciéndole que ha ocurrido un error, y se pone en el texto una cadena vacia.

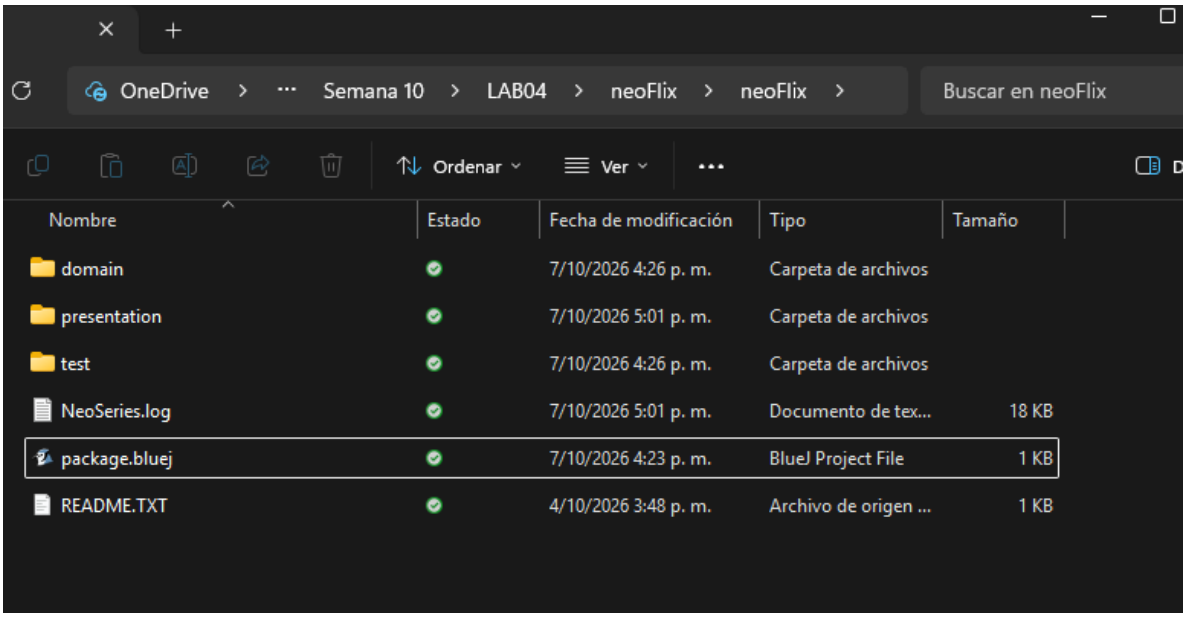
6. Ejecuten nuevamente la aplicación con el caso propuesto en 1
   a) ¿Qué mensaje salió en pantalla?



b) ¿La aplicación termina?
No, la aplicación continua:



c) ¿Qué información tiene el archivo de errores?




```
oct 07, 2026 5:01:54 P. M. domain.Log record
GRAVE: java.lang.IndexOutOfBoundsException: Index 6 out of bounds for length 6
java.lang.IndexOutOfBoundsException: Index 6 out of bounds for length 6
        at java.base/jdk.internal.util.Preconditions.outOfBounds(Preconditions.java:100)
        at java.base/jdk.internal.util.Preconditions.outOfBoundsCheckIndex(Preconditions.java:106)
        at java.base/jdk.internal.util.Preconditions.checkIndex(Preconditions.java:302)
        at java.base/java.util.Objects.checkIndex(Objects.java:385)
        at java.base/java.util.ArrayList.get(ArrayList.java:427)
        at domain.NeoFlix.select(NeoFlix.java:152)
        at domain.NeoFlix.search(NeoFlix.java:188)
        at presentation.NeoFlixGUI.actionSearch(NeoFlixGUI.java:236)
        at presentation.NeoFlixGUI$6.insertUpdate(NeoFlixGUI.java:209)
        at java.desktop/javax.swing.text.AbstractDocument.fireInsertUpdate(AbstractDocument.java:227)
        at java.desktop/javax.swing.text.AbstractDocument.handleInsertString(AbstractDocument.java:781)
        at java.desktop/javax.swing.text.AbstractDocument.insertString(AbstractDocument.java:740)
        at java.desktop/javax.swing.text.PlainDocument.insertString(PlainDocument.java:131)
        at java.desktop/javax.swing.text.AbstractDocument.replace(AbstractDocument.java:699)
        at java.desktop/javax.swing.text.JTextComponent.replaceSelection(JTextComponent.java:1335)
        at java.desktop/javax.swing.text.DefaultEditorKit$DefaultKeyTypedAction.actionPerformed(DefaultEditorKit.java:906)
        at java.desktop/javax.swing.SwingUtilities.notifyAction(SwingUtilities.java:1810)
        at java.desktop/javax.swing.JComponent.processKeyBinding(JComponent.java:2956)
        at java.desktop/javax.swing.JComponent.processKeyBindings(JComponent.java:3004)
        at java.desktop/javax.swing.JComponent.processKeyEvent(JComponent.java:2918)
        at java.desktop/java.awt.Component.processEvent(Component.java:6398)
        at java.desktop/java.awt.Container.processEvent(Container.java:2266)
        at java.desktop/java.awt.Component.dispatchEventImpl(Component.java:4996)
        at java.desktop/java.awt.Container.dispatchEventImpl(Container.java:2324)
        at java.desktop/java.awt.Component.dispatchEvent(Component.java:4828)
        at java.desktop/java.awt.KeyboardFocusManager.redispatchEvent(KeyboardFocusManager.java:1952)
        at java.desktop/java.awt.DefaultKeyboardFocusManager.dispatchKeyEvent(DefaultKeyboardFocusManager.java:883)
        at java.desktop/java.awt.DefaultKeyboardFocusManager.preDispatchKeyEvent(DefaultKeyboardFocusManager.java:1146)
        at java.desktop/java.awt.DefaultKeyboardFocusManager.typeAheadAssertions(DefaultKeyboardFocusManager.java:1020)
        at java.desktop/java.awt.DefaultKeyboardFocusManager.dispatchEvent(DefaultKeyboardFocusManager.java:848)
        at java.desktop/java.awt.Component.dispatchEventImpl(Component.java:4877)
        at java.desktop/java.awt.Container.dispatchEventImpl(Container.java:2324)
        at java.desktop/java.awt.Window.dispatchEventImpl(Window.java:2780)
        at java.desktop/java.awt.Component.dispatchEvent(Component.java:4828)
        at java.desktop/java.awt.EventQueue.dispatchEventImpl(EventQueue.java:775)
        at java.desktop/java.awt.EventQueue$4.run(EventQueue.java:720)
        at java.desktop/java.awt.EventQueue$4.run(EventQueue.java:714)
        at java.base/java.security.AccessController.doPrivileged(AccessController.java:400)
        at java.base/java.security.ProtectionDomain$JavaSecurityAccessImpl.doIntersectionPrivilege(ProtectionDomain.java:87)
        at java.base/java.security.ProtectionDomain$JavaSecurityAccessImpl.doIntersectionPrivilege(ProtectionDomain.java:98)
        at java.desktop/java.awt.EventQueue$5.run(EventQueue.java:747)
        at java.desktop/java.awt.EventQueue$5.run(EventQueue.java:745)
        at java.base/java.security.AccessController.doPrivileged(AccessController.java:400)
        at java.base/java.security.ProtectionDomain$JavaSecurityAccessImpl.doIntersectionPrivilege(ProtectionDomain.java:87)
        at java.desktop/java.awt.EventQueue.dispatchEvent(EventQueue.java:744)
        at java.desktop/java.awt.EventDispatchThread.pumpOneEventForFilters(EventDispatchThread.java:203)
        at java.desktop/java.awt.EventDispatchThread.pumpEventsForFilter(EventDispatchThread.java:124)
        at java.desktop/java.awt.EventDispatchThread.pumpEventsForHierarchy(EventDispatchThread.java:113)
```

Tiene información de la fecha, el error y de donde sale el error.

7. Revisen el código
   a) ¿Qué otro caso puede generar un error?

```
public ArrayList<Content> select(String prefix){
    ArrayList <Content> answers=new ArrayList<Content>();
    prefix=prefix.toUpperCase();
    for(int i=0;i<contents.size();i++){
        if(contents.get(i+1).getTitle().toUpperCase().startsWith(prefix)){
            answers.add(contents.get(i));
        }
    }
    return answers;
}
```

   Como originalmente tiene i +1, es necesario cambiarlo para que no se desborde y que dejen de ocurrir esos errores que se tenían desde antes.

   b) ¿Cómo lo solucionarían?
   Para ese solo es necesario quitarle el + 1.

```
/**
 * Returns the contents whose title starts with the given prefix
 * @param prefix
 * @return
 */
public ArrayList<Content> select(String prefix){
    ArrayList <Content> answers=new ArrayList<Content>();
    prefix=prefix.toUpperCase();
    for(int i=0;i<contents.size();i++){
        if(contents.get(i).getTitle().toUpperCase().startsWith(prefix)){
            answers.add(contents.get(i));
        }
    }
    return answers;
}
```

8.
   a) ¿Es adecuado que la aplicación continúe su ejecución después de sufrir un incidente como este?
   Si, porque en este caso solo falló uno de los tres componentes que tiene, o sea, la consulta, por tanto puede seguir modificando y añadiendo cosas, puesto que esa consulta no hizo que los datos se dañaran o algo por el estilo. Además, al atrapar la excepción se registra el detalle técnico en el Log y se le dice al usuario que pasó.

   b) ¿De qué dependería continuar o parar?
   Depende de si, a la hora de ingresar datos, si algún dato está corrupto y entra, entonces ya no se debería poder continuar, además si hay un error critico en el sistema se tiene que parar obligatoriamente, por ejemplo, el típico de error de OutOfMemoryError por si se el equipo se queda sin memoria.

9. Modifiquen la aplicación para garantizar que SIEMPRE que haya un error no esperado en presentación se maneje de la forma adecuada.
   a) ¿Cuál fue la solución seleccionada?

Pues en este caso fue implementar el try-catch en las acciones para encargarnos de cualquier excepción inesperada y que esta sea comentada en el Log:

```
private void actionList(){
    try {
        textDetails.setText(neoFlix.toString());
    } catch (Exception e) {
        Log.record(e);
        JOptionPane.showMessageDialog(this, "Ups, ha ocurrido un error inesperado al listar.");
    }
}

private void  actionAdd(){
    try{
        if (episodes.getText().trim().equals("")){
            neoFlix.addEpisode (title.getText(), "", atttemps.getText(), completed.getText(),votes.getText(),sumVotes.getText());
        }
        else{
            neoFlix.addSeries (title.getText(),year.getText(),episodes.getText());
        }
    }catch(NeoFlixException e){
        JOptionPane.showMessageDialog(this, "Ups, ha ocurrido un error inesperado:\n" + e.getMessage());
    }catch(Exception e){
        JOptionPane.showMessageDialog(this, "Ups, ha ocurrido un error inesperado al adicionar.");
    }
}

private void actionSearch(){
    String patronBusqueda=textSearch.getText();
    String answer = "";
    try{
        if(patronBusqueda.length() > 0) {
            answer = neoFlix.search(patronBusqueda);
        }
        textResults.setText(answer);
    }
    catch (Exception e){
        Log.record(e);

        JOptionPane.showMessageDialog(this, "Ups, ha ocurrido un error al buscar usando: " + patronBusqueda);

        textResults.setText("");
    }
```

PARTE III. DE BLUEJ A CONSOLA

En esta sección vamos a aprender a usar java desde consola. Para esto vamos a trabajar con el proyecto neoFlix.

A. Comandos básicos del sistema operativo:

Antes de iniciar debemos repasar los comandos básicos del manejo de la consola.

1. Investiguen los comandos para moverse en la estructura de directorios: crear, borrar, listar su contenido y copiar o eliminar un archivo.

Para crear directorios:

Syntax

```
mkdir [<drive>:]<path>
```

Siendo que si estoy en la carpeta la forma más sencilla es como:

mkdir <nombre_directorio>

Para borrar directorios:

Syntax

```
rmdir [<drive>:]<path> [/s [/q]]
```

Siendo su forma más sencilla rmdir <nombre_directorio>

Listar contenidos:

Syntax

```
dir [<drive>:][<path>][<filename>] [...] [/p] [/q] [/w] [/d]
```

En los casos generales se usa dir.

Copiar:

Syntax

```
copy [/d] [/v] [/n] [/y | /-y] [/z] [/a | /b] <source>
```

En términos generales copy<origen><destino>

Borrar:

Syntax

Copy

```
del [/p] [/f] [/s] [/q] [/a[:]<attributes>] <names>
erase [/p] [/f] [/s] [/q] [/a[:]<attributes>] <names>
```

En forma general es como: del nombreArchivo

Moverse:

Syntax

Copy

```
cd [/d] [<drive>:][<path>]
cd [..]
chdir [/d] [<drive>:][<path>]
chdir [..]
```

En términos generales es como cd Carpeta o cd <path>

2. Organicen un nuevo directorio con la estructura propuesta para probar desde allí los comandos de consola. Consulten y capturen el contenido de su directorio.

- neoFlix

-src

-domain

-presentation

-test

Primero es importante irnos moviendo por las carpetas usando cd nombre_carpeta. Hasta llegar a neoFlix, donde hacemos:

```
C:\Users\wilson.mendivelso-s\Downloads\LAB04-DOPO\neoFlix>mkdir src
```

Para crear la carpeta src:



Y luego nos movemos a esa carpeta:

```
C:\Users\wilson.mendivelso-s\Downloads\LAB04-DOPO\neoFlix>cd src

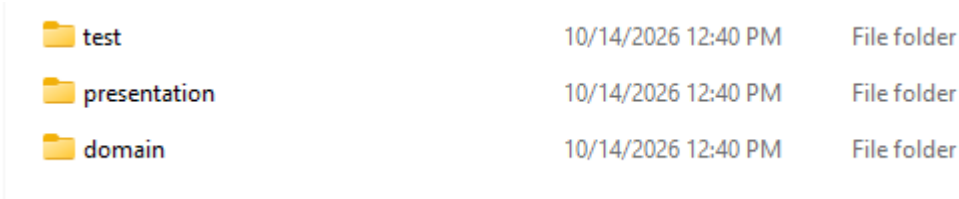
C:\Users\wilson.mendivelso-s\Downloads\LAB04-DOPO\neoFlix\src>
```

Posteriormente creamos tres nuevas carpetas ahí dentro:

```
C:\Users\wilson.mendivelso-s\Downloads\LAB04-DOPO\neoFlix\src>mkdir domain

C:\Users\wilson.mendivelso-s\Downloads\LAB04-DOPO\neoFlix\src>mkdir presentation

C:\Users\wilson.mendivelso-s\Downloads\LAB04-DOPO\neoFlix\src>mkdir test
```



3. En el directorio src/domain copien únicamente el archivo Serie.java del paquete de dominio. Consulten y capturen el contenido de src/domain.

```
C:\Users\wilson.mendivelso-s\Downloads\LAB04-DOPO\neoFlix\neoFlix\domain>copy Series.java ..\..\src\domain
        1 file(s) copied.
```



En este caso me ubiqué donde tenia originalmente Series.java, luego en el lugar de destino, me devuelvo dos carpetas, y me meto a src y luego a domain.

B Estructura de proyectos java

En java los proyectos se estructuran considerando tres directorios básicos:

neoFlix

src

bin

doc

lib

1. Investiguen los archivos que deben quedar en cada una de esas carpetas y la organización interna de ellas

- src: Es el diminutivo de Source, que es donde se ubica toda la parte del código. O sea, normalmente se ve como:
  src
  domain
  presentation
  test
  En pocas palabras es todo aquello que termina en .java, además de que se encuentran ahí los paquetes.
- bin: Es el diminutivo de Binary, en este normalmente n ose escribe nada, puesto que va a tener todos los archivos compilados. En plan, si tengo un .java, este va a tener su contraparte .class. Es importante, porque evita tener el código fuente y el código compilado.
- doc: Es el diminutivo de Documentation, que es donde va a ir toda la documentación que java genera automáticamente, o sea todos esos pdf y demás.
- lib: Es el diminutivo de Libraries, que son todas las librerías externas, o sea, todo ese código que otros escribieron y yo uso, los cuales normalmente suelen ser archivos .jar.

2. a) ¿Qué archivos de neoFlix deberían estar en el directorio src?

Los archivos que deberían estar son todos aquellos que sean .java.

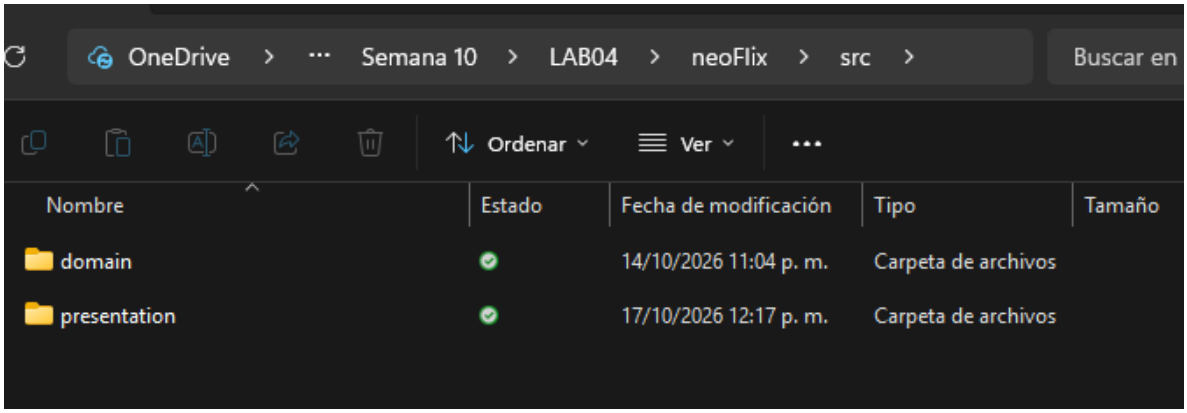
b) ¿Cómo deberían estar organizados?

Deberían estar organizados por las carpetas de domain, presentation y tests. Puesto que nos interesa tener aún diferenciado cada una de esas partes del código.

3. Copien a src únicamente los archivos de los paquetes de presentación y dominio. Consulten y capturen el contenido de los directorios que se modificaron.

   Para esta parte es primero necesario crear la carpeta que nos falta de presentación:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix\src>mkdir presentation
```



Posteriormente ya estamos en src, nos movemos a través de las carpetas y vamos copiando a la carpeta que tenemos ahí en src de domain:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix\src>copy ..\neoFlix\domain\NeoFlixException.java doma
        1 archivo(s) copiado(s).

C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix\src>copy ..\neoFlix\domain\NeoFlix.java domain
        1 archivo(s) copiado(s).

C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix\src>copy ..\neoFlix\domain\Audiovisual.java domain
        1 archivo(s) copiado(s).

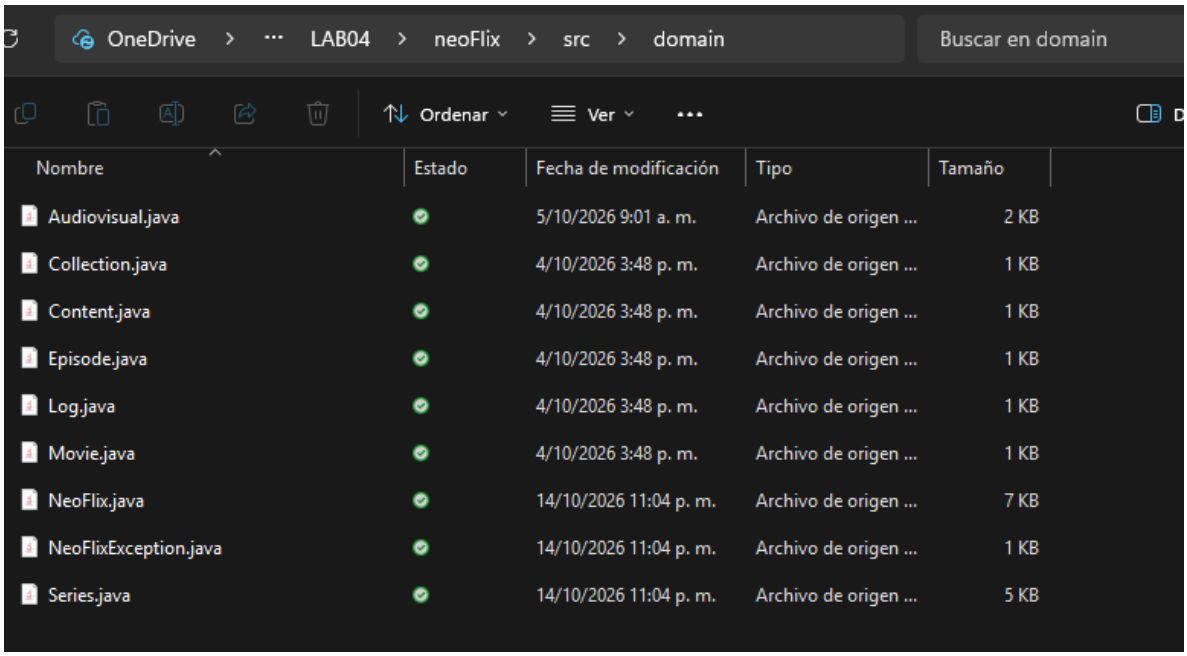
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix\src>copy ..\neoFlix\domain\Log.java domain
        1 archivo(s) copiado(s).

C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix\src>copy ..\neoFlix\domain\Movie.java domain
        1 archivo(s) copiado(s).

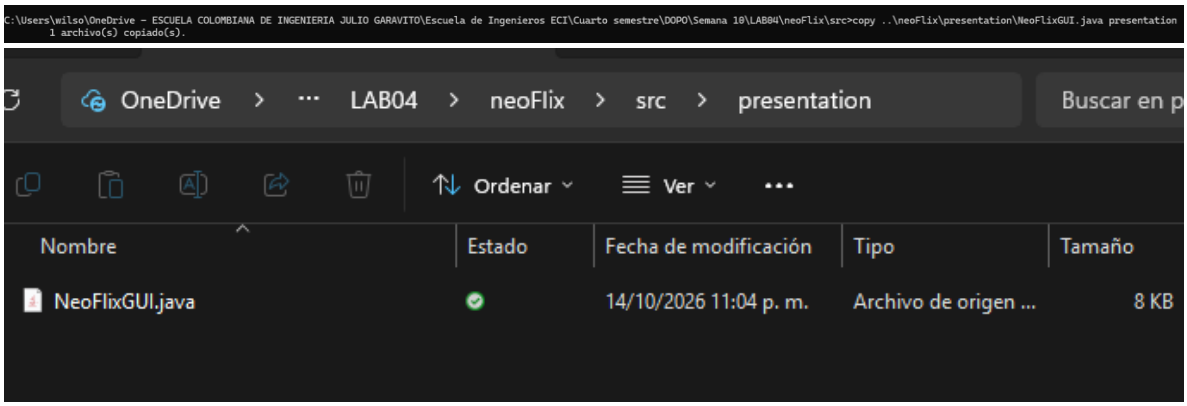
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix\src>copy ..\neoFlix\domain\Episode.java domain
        1 archivo(s) copiado(s).

C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix\src>copy ..\neoFlix\domain\Content.java domain
        1 archivo(s) copiado(s).

C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix\src>copy ..\neoFlix\domain\Collection.java domain
        1 archivo(s) copiado(s).
```



Ahora para la presentation:



C Comandos de java

1. Consulten para que sirven cada uno de los siguiente comandos:

    Javac

    Java

    Javadoc

    Jar

javac: Este se encarga de compilar el código fuente de Java, o sea, los .java, para luego generar los archivos compilados .class

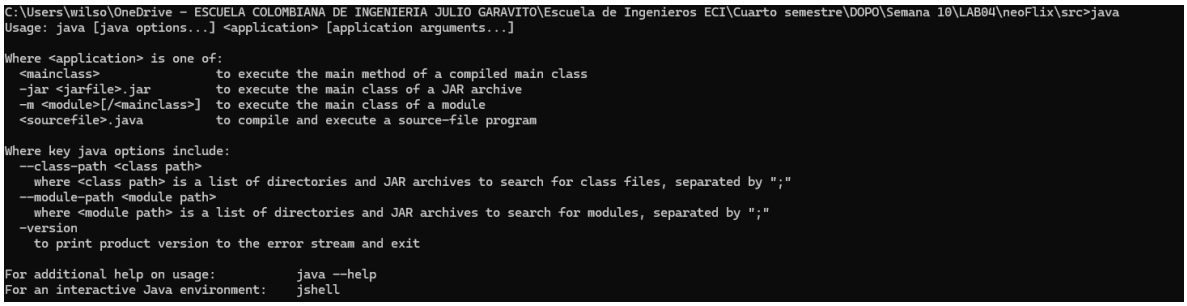
java: Se encarga de ejecutar u n programa ya compilado que sea .class

javadoc: Este se encarga generar automáticamente la documentación del código de Java en formato HTML con los comentarios. Por ejemplo si hago:

> javadoc -d doc src/domain/*.java me genera una carpeta con páginas HTML que documentan clases, métodos y atributos.

jar: Empaqueta archivos (.class, recursos, etc…) en un archivo JAR, que facilita la distribución y ejecución de una aplicación Java.

2. Creen una sesión de consola y consulten en línea las opciones de los comandos java y javac. Capturen pantallas:
   Al ejecutar java:



   Al ejecutar javac:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix\src>javac
Usage: javac <options> <source files>
where possible options include:
  @<filename>                  Read options and filenames from file
  -Akey[=value]                Options to pass to annotation processors
  --add-modules <module>(,<module>)*
        Root modules to resolve in addition to the initial modules,
        or all modules on the module path if <module> is ALL-MODULE-PATH.
  --boot-class-path <path>, -bootclasspath <path>
        Override location of bootstrap class files
  --class-path <path>, -classpath <path>, -cp <path>
        Specify where to find user class files and annotation processors
  -d <directory>               Specify where to place generated class files
  -deprecation
        Output source locations where deprecated APIs are used
  --enable-preview
        Enable preview language features.
        To be used in conjunction with either -source or --release.
  -encoding <encoding>         Specify character encoding used by source files
  -endorseddirs <dirs>         Override location of endorsed standards path
  -extdirs <dirs>              Override location of installed extensions
  -g                           Generate all debugging info
  -g:{lines,vars,source}       Generate only some debugging info
  -g:none                      Generate no debugging info
  -h <directory>
        Specify where to place generated native header files
  --help, -help, -?            Print this help message
  --help-extra, -X             Print help on extra options
  -implicit:{none,class}
        Specify whether to generate class files for implicitly referenced files
  -J<flag>                     Pass <flag> directly to the runtime system
  --limit-modules <module>(,<module>)*
        Limit the universe of observable modules
  --module <module>(,<module>)*, -m <module>(,<module>)*
        Compile only the specified module(s), check timestamps
  --module-path <path>, -p <path>
        Specify where to find application modules
  --module-source-path <module-source-path>
        Specify where to find input source files for multiple modules
  --module-version <version>
        Specify version of modules that are being compiled
  -nowarn                      Generate only mandatory warnings
  -parameters
        Generate metadata for reflection on method parameters
  -proc:{none,only,full}
        Control whether annotation processing and/or compilation is done.
```

3. Busquen la opción que sirve para conocer a la versión a que corresponden estos dos comandos. Documenten el resultado.
   Para ver la versión de java se hace de la siguiente manera:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix\src>java -version
java version "25.0.3" 2026-04-21 LTS
Java(TM) SE Runtime Environment (build 25.0.3+9-LTS-195)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.3+9-LTS-195, mixed mode, sharing)
```

   Para ver la versión de javac se hace de una manera parecida:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix\src>javac -version
javac 25.0.3
```

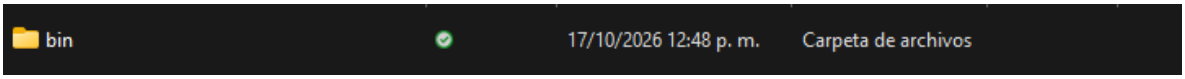
D Compilando:

1. Utilizando el comando javac, desde el directorio raíz (desde neoFlix con una sola instrucción), compile el proyecto.
   a) ¿Qué instrucción completa tuvo que dar a la consola para compilar TODO el proyecto? Tenga presenta que se pide un único comando y que los archivos compilados deben quedar en los directorios respectivos:

   Primero es necesario crear la carpeta bin y dentro de bin crear las carpetas domain y presentation.

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>mkdir bin
```

Y se crea la carpeta de bin:



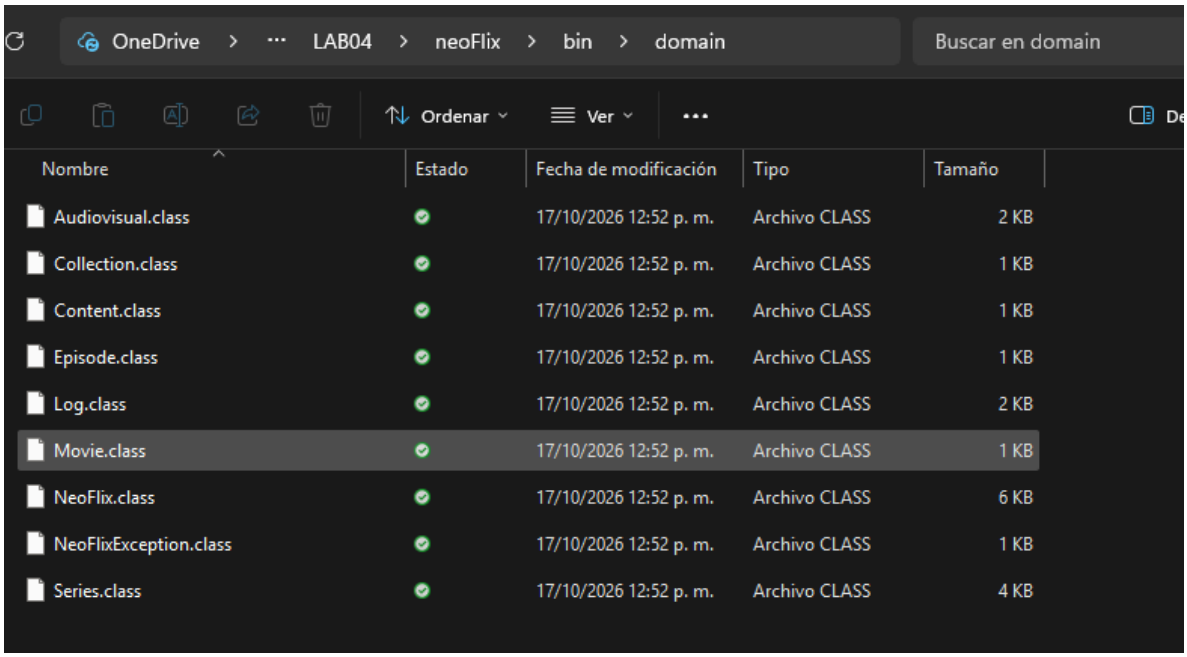
Ahora el comando es el siguiente:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>javac -d bin src\domain\*.java src\presentation\*.java
```

En pocas palabras, el -d bin, es el que coloca los .class en la carpeta bin respetando los paquetes, y pues ya ahí compila cada una de las partes del código de los .java tanto de domain como de presentation. Además, -d bin, crea automáticamente los subdirectorios domain y presentation dentro de bin según los paquetes que se declararon.

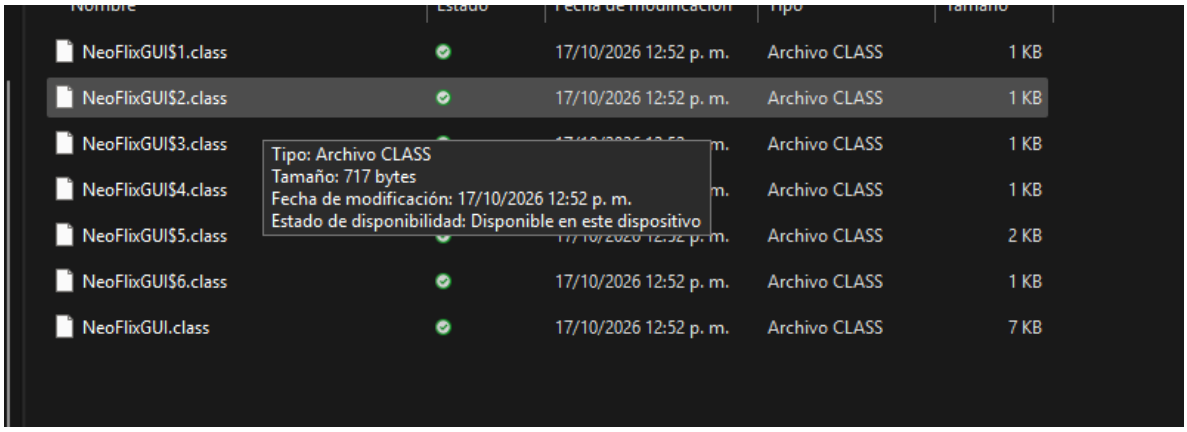
2. Revise de nuevo el contenido del directorio de trabajo y sus subdirectorios.
   a) ¿Cuáles nuevos archivos aparecen ahora y dónde se ubican? Capturen las pantallas con los contenidos de los directorios de neoFlix/bin

Ahora se ve en domain:



Que son los 9 archivos en .class, correspondientes a los del paquete domain de src.

Por otro lado, en presentation:



Acá se ve el NeoFlixGUI.class que es el compilamiento del NeoFlixGUI.java normal, además de que están las clases anónimas que ahí se definieron con todo el tema de los ActionListener y demás.

E Documentando:

1. Utilizando el comando javadoc, desde el directorio raíz, genere la documentación API en formato html en este directorio.
   a) ¿Cuál es el comando completo para generar esta documentación?
   Primero es necesario crear la carpeta donde se almacenarán los documentos:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>mkdir doc
```



   El comando para generar la documentación es el siguiente:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>javadoc -d doc src\domain\*.java src\presentation\*.java
```

2. a) ¿Cuál archivo hay qué abrir para empezar a navegar por la documentación? Ábranlo y capturen la pantalla.
   El que toca abrir es:





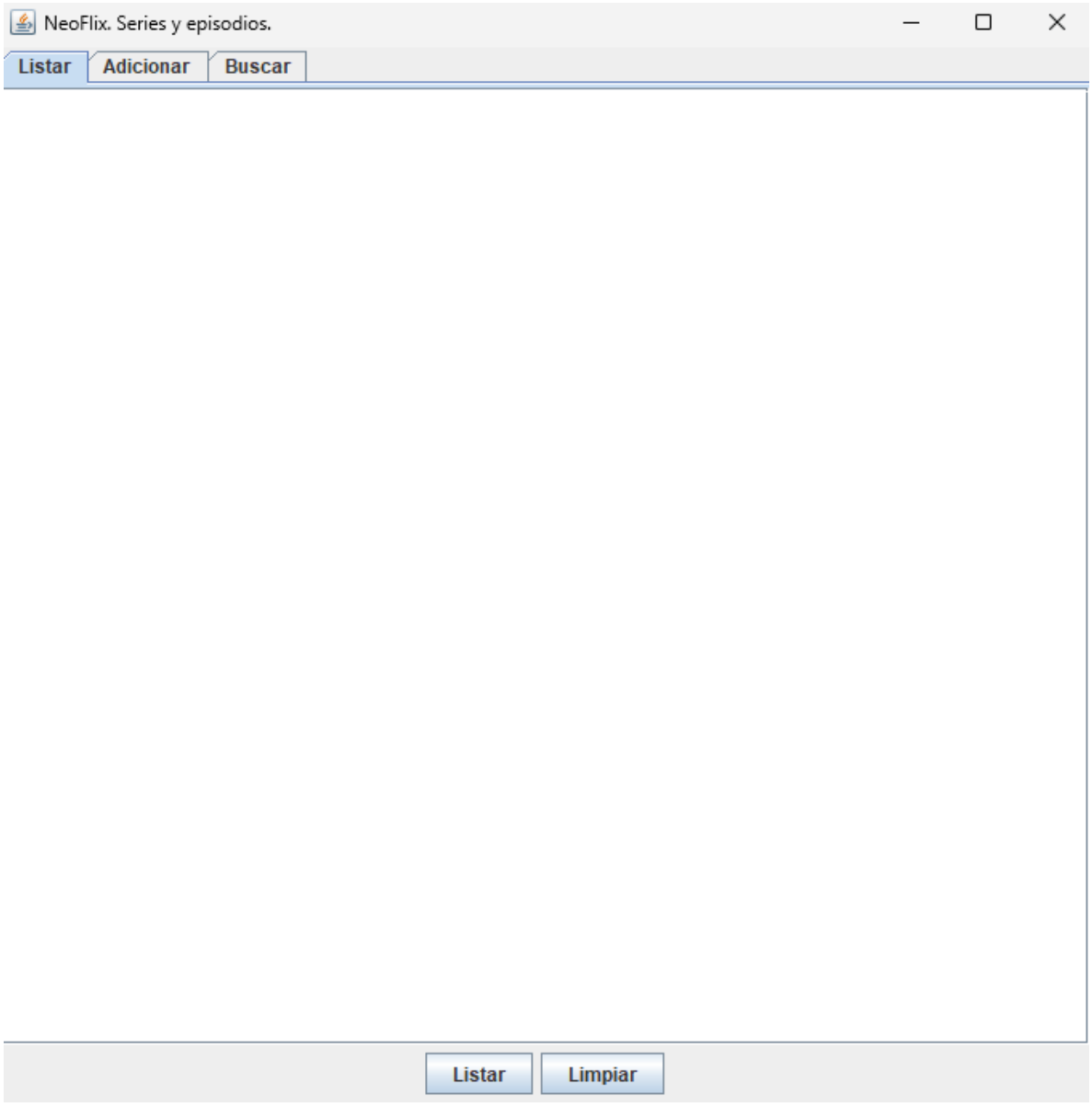
F Ejecutando:

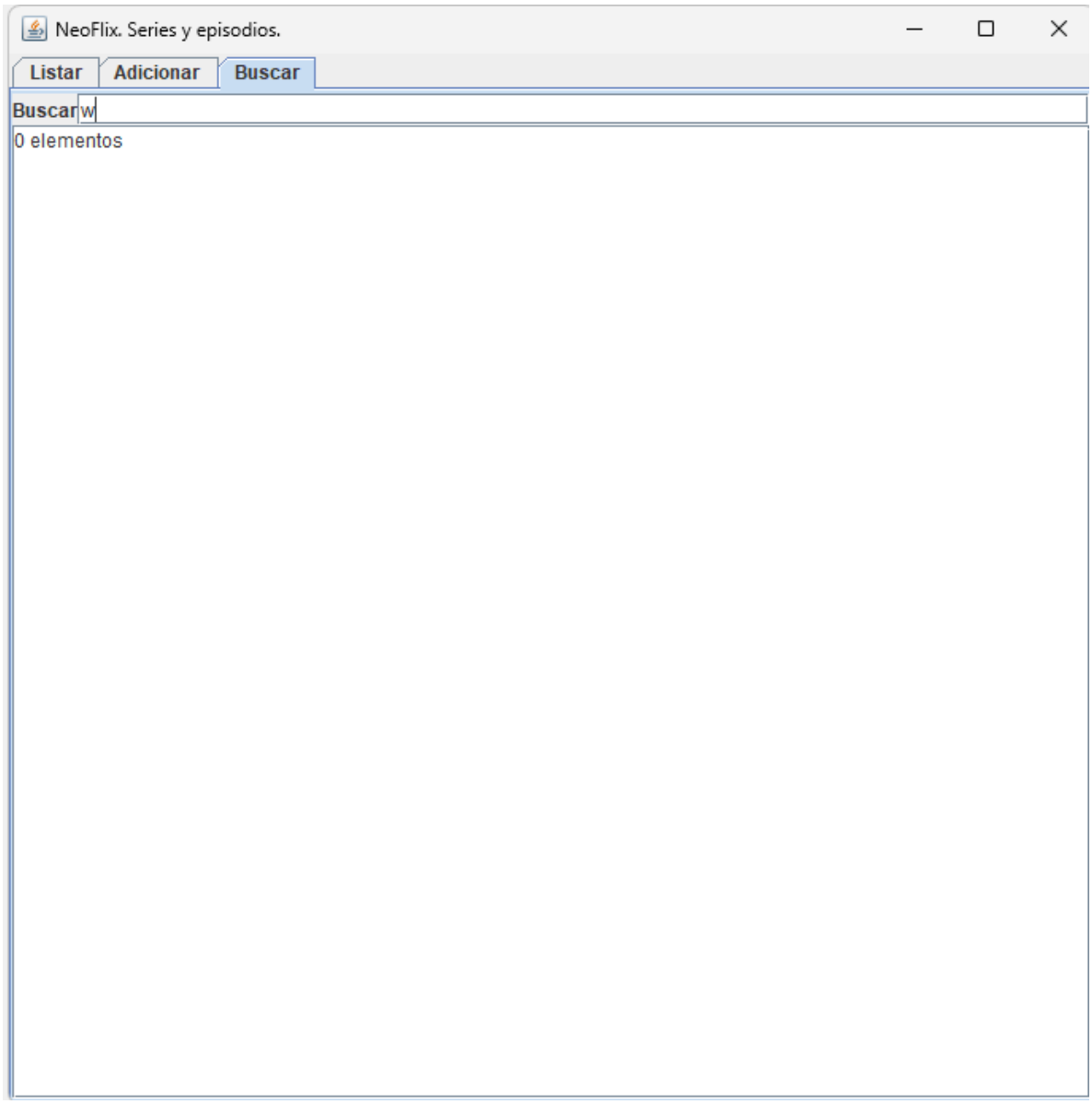
1. Empleando el comando java, desde el directorio raíz, ejecute el programa.
   a) ¿Cómo utilizó este comando?
   El comando es el siguiente:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>java -cp bin presentation.NeoFlixGUI
```

   Al ejecutarlo sale:

   El comando se divide en varias partes:
   - java es el que permite que se ejecute el Java
   - cp bin le dice que busque en las clases compiladas de la carpeta bin
   - presentation.NeoFlixGUI busca al que tenga el main
2. Capturen la pantalla del programa en ejecución:


NeoFlix. Series y episodios.
Listar
Adicionar
Buscar
Listar
Limpiar



Ahí se ve como el programa se ejecuta.

G Probando

1. Adicione ahora los archivos del paquete de pruebas y trate de compilar nuevamente el programa.
   a) ¿Qué pasa?

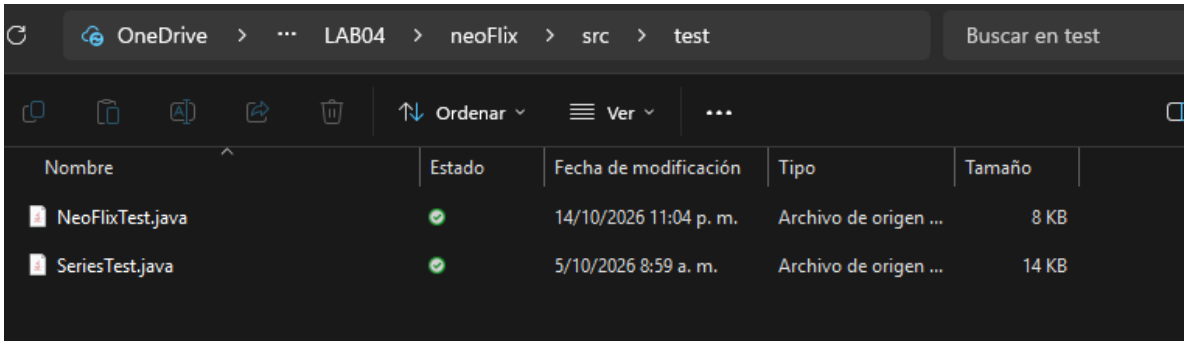
   Primero que todo es necesario crear dentro del src la carpeta para los tests:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix\src>mkdir test
```




```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>copy neoFlix\test\NeoFlixTest.java src\test
        1 archivo(s) copiado(s).
```

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>copy neoFlix\test\SeriesTest.java src\test
        1 archivo(s) copiado(s).
```



Para compilar:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO\Semana 10\LAB04\neoFlix>javac -d bin src\test\*.java
src\test\NeoFlixTest.java:5: error: package org.junit does not exist
import static org.junit.Assert.*;
                ^
src\test\NeoFlixTest.java:6: error: package org.junit does not exist
import org.junit.After;
               ^
src\test\NeoFlixTest.java:7: error: package org.junit does not exist
import org.junit.Before;
               ^
src\test\NeoFlixTest.java:8: error: package org.junit does not exist
import org.junit.Test;
               ^
src\test\NeoFlixTest.java:2: error: package domain does not exist
import domain.*;
       ^
src\test\SeriesTest.java:5: error: package org.junit does not exist
import static org.junit.Assert.*;
                ^
src\test\SeriesTest.java:6: error: package org.junit does not exist
import org.junit.After;
               ^
src\test\SeriesTest.java:7: error: package org.junit does not exist
import org.junit.Before;
               ^
src\test\SeriesTest.java:8: error: package org.junit does not exist
import org.junit.Test;
               ^
src\test\SeriesTest.java:2: error: package domain does not exist
import domain.*;
       ^
src\test\NeoFlixTest.java:16: error: cannot find symbol
    @Test
     ^
```

Pero al hacerlo salen muchos errores relacionados al tema de que faltan librerías y demás.

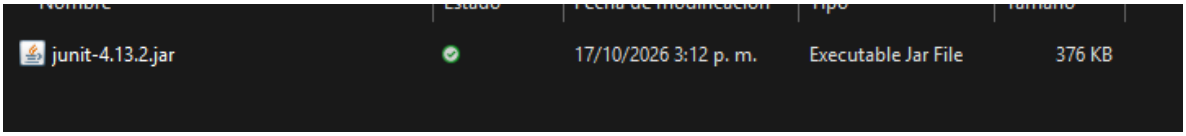
2. Tenga en cuenta que estas clases requieren la librería junit.
   (a) ¿Dónde se incluye una librería en la estructura de archivos?

   Se incluye en la carpeta lib:

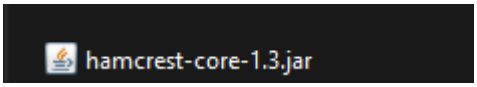
```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>mkdir lib
```



Para incluir la librería es necesario descargar Junit 4.13.2:



Además de descargar la de hamcrest-1.3.jar



b) ¿Cómo se compila incluyendo un .jar?

Para compilarlo toca tener un -cp, o sea un classpath, donde se especifica las clases de mi programa en bin y las clases que voy a usar como el Junit y el hamcrest que son .jar.

c) ¿Qué instrucción completa tuvo que dar a la consola para compilar?
El comando fue el siguiente:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>javac -cp "bin;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" -d bin src\test\*.java
```

3. Ejecute desde consola las pruebas.
   a) ¿Cómo utilizó el comando java?
   El comando java en este caso será de dos formas:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>java -cp "bin;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.runner.JUnitCore test.N
eoFlixTest
JUnit version 4.13.2
.........
Time: 0,053

OK (10 tests)
```

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>java -cp "bin;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.runner.JUnitCore test.S
eriesTest
JUnit version 4.13.2
....................
Time: 0,031

OK (20 tests)


C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>
```

En resumidas cuentas, habían dos tests diferentes NeoFlixTest y SeriesTest, el comando funciona, porque se ejecuta en este caso las pruebas, buscando dentro de bin las pruebas, además de que se dice que se tendrán en cuenta esas dos librerías .jar, además se dice que se

usará el JUnitCore dentro de junit como el que corra las pruebas, por últimos, si dice test.Algo, es porque test es el paquete dentro bin y ese Algo es la clase.

4. Pegue en su documento el resultado de las pruebas

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>java -cp "bin;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.runner.JUnitCore test.N
eoFlixTest
JUnit version 4.13.2
..........
Time: 0,053

OK (10 tests)
```

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>java -cp "bin;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.runner.JUnitCore test.S
eriesTest
JUnit version 4.13.2
....................
Time: 0,031

OK (20 tests)


C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\LAB04\neoFlix>
```

H Empaquetando

1. Consulten como utilizar desde consola el comando jar para empaquetar su programa entregable en un archivo .jar, que contenga los archivos bytecode necesario (no las fuentes ni las clases de prueba) ¿Cómo empaquetó jar?

Para empaquetar se puede hacer de la siguiente manera, desde el neoFlix:

```
C:\Users\wilso\OneDrive - ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO\Escuela de Ingenieros ECI\Cuarto semestre\DOPO
\Semana 10\Garzon - Mendivelso\neoFlix>jar cfe neoFlix.jar presentation.NeoFlixGUI -C bin domain -C bin presentation
```

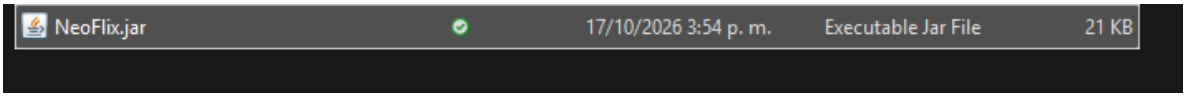
El comando inicia con jar, luego tiene cfe, que significan:

c -> Create, crea el nuevo jar

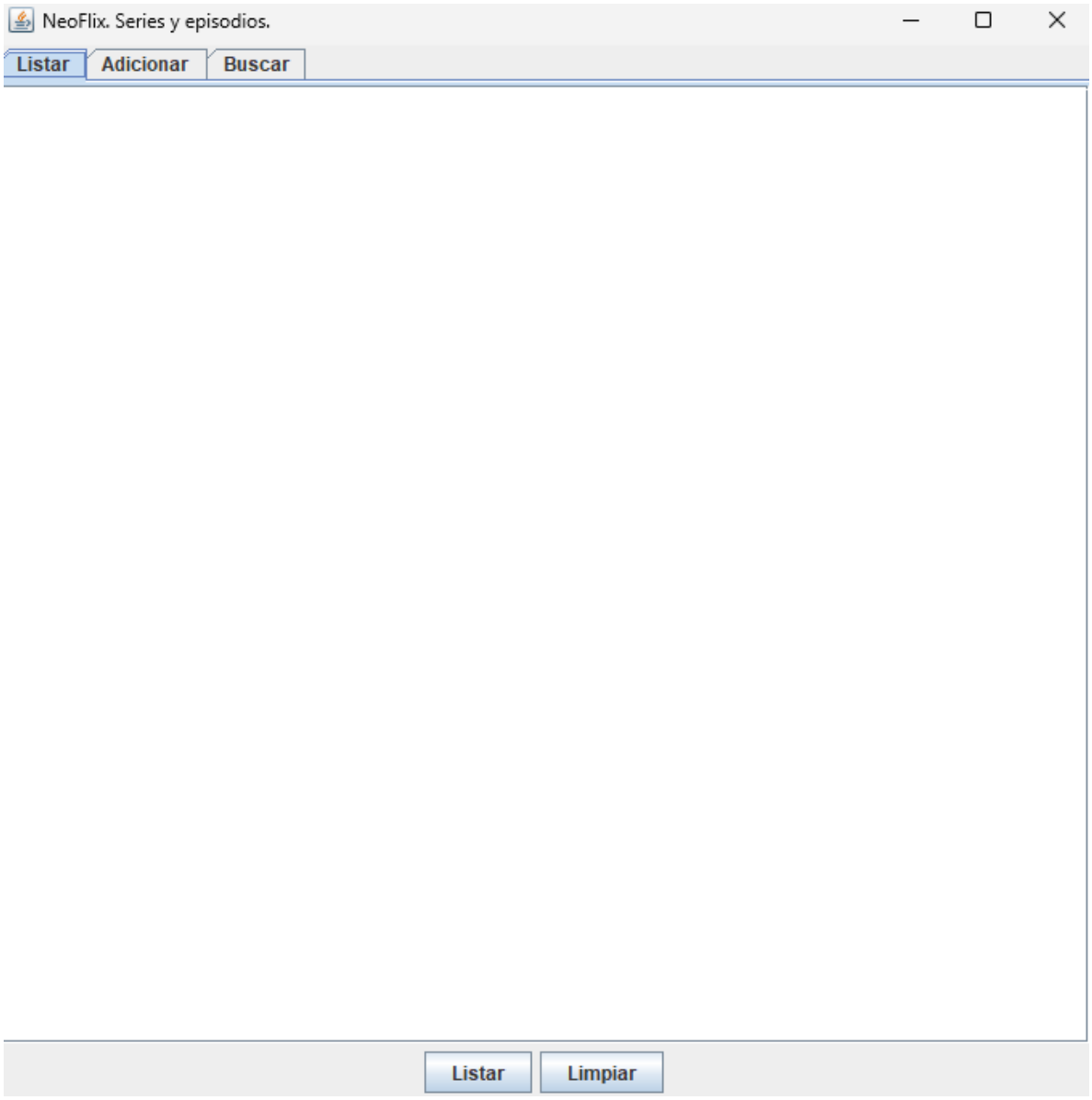
f -> Dice que el siguiente argumento será el nombre del archivo

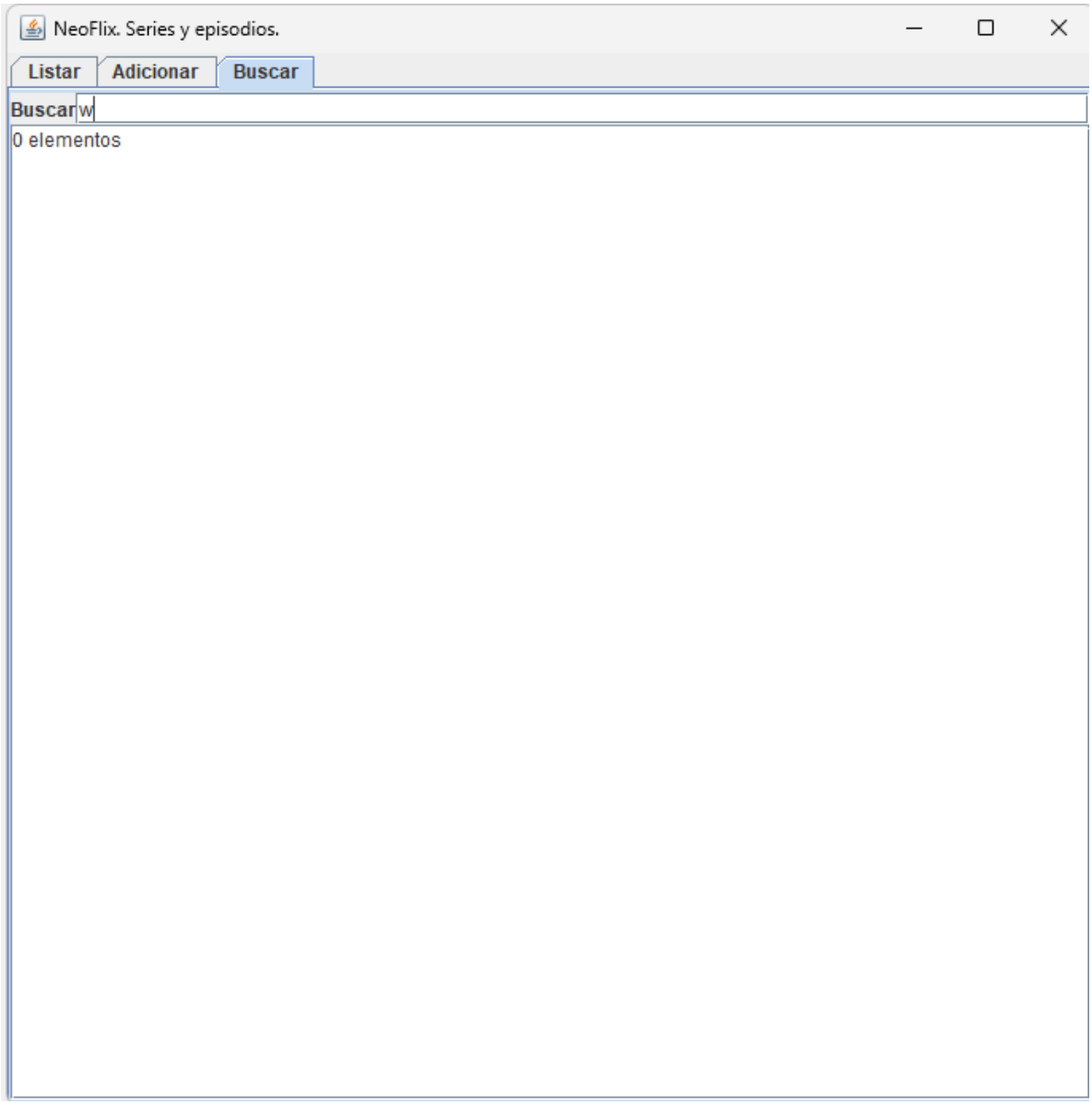
e- > Especifica cual es la clase principal

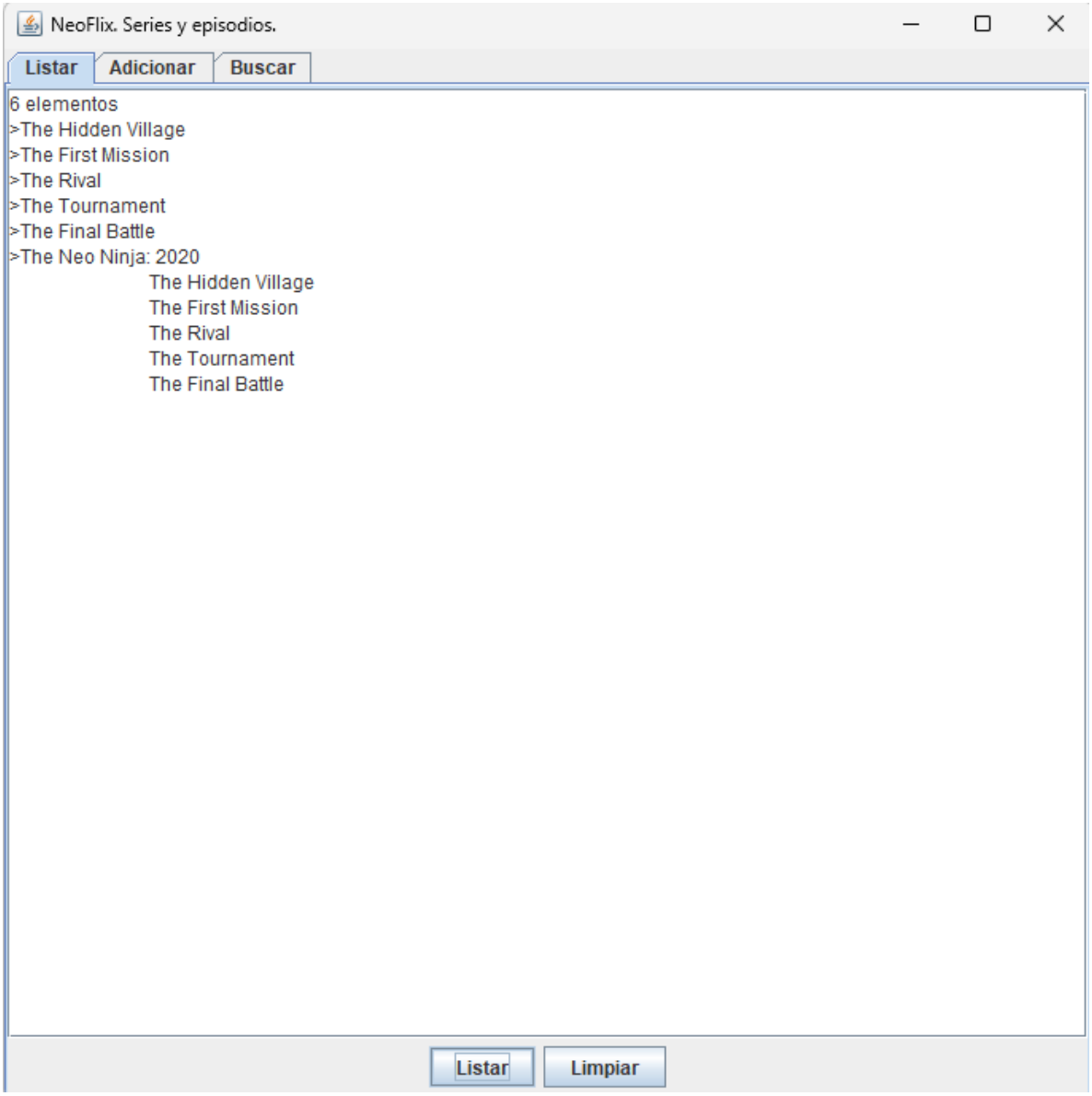
La parte de -C bin ., indica que entrará a bin y meterá todo su contenido en JAR

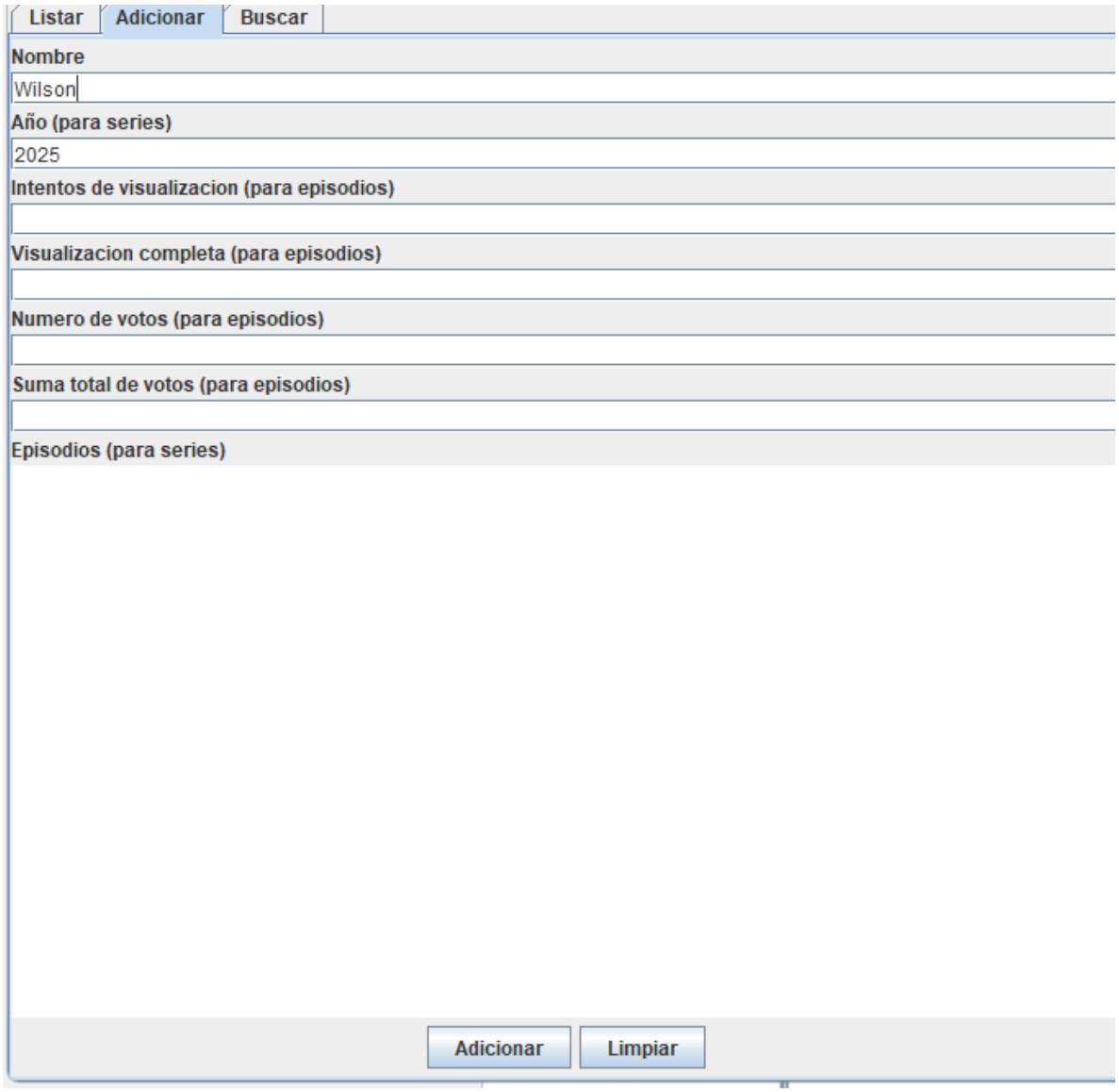


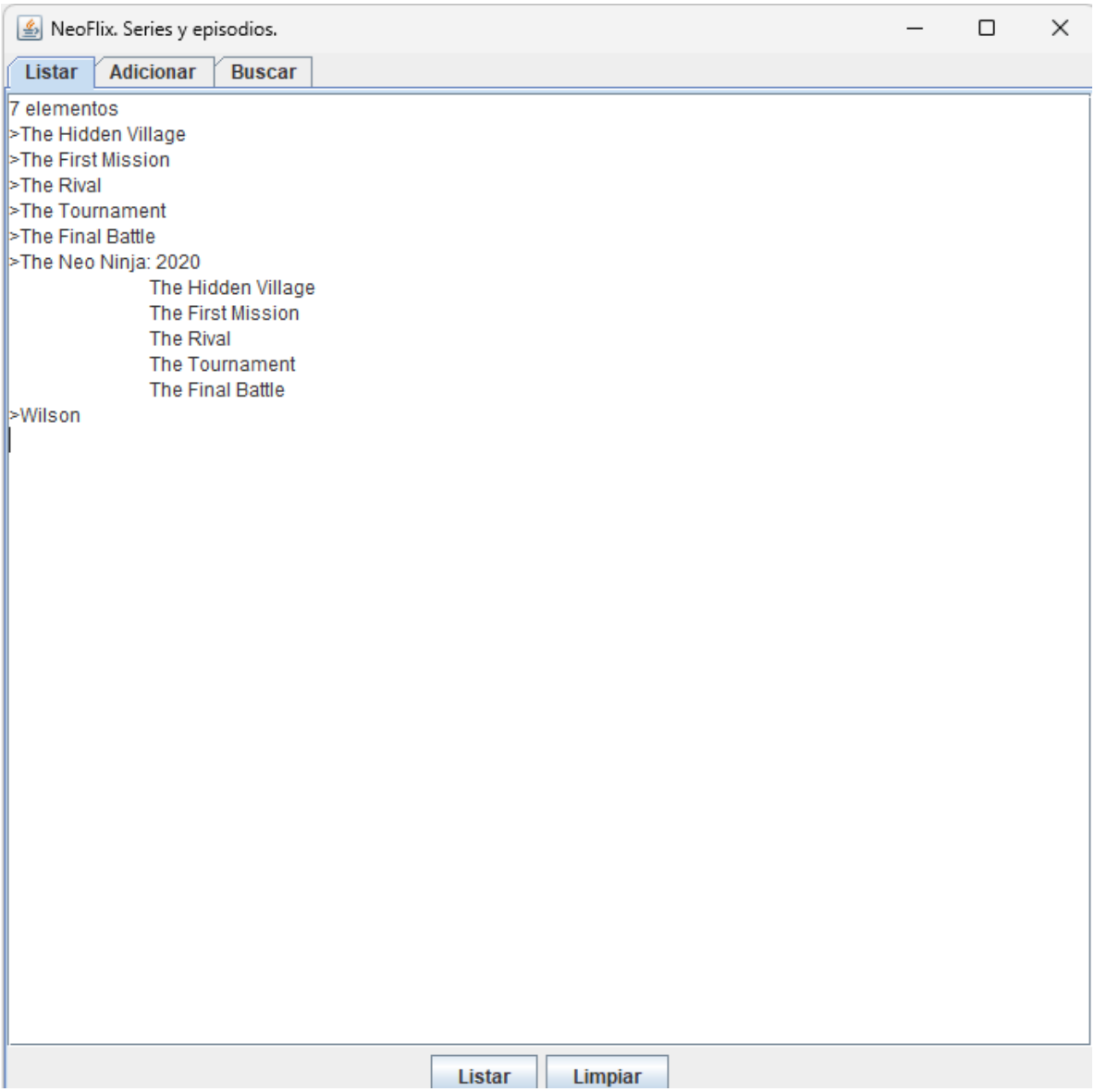
2. A) ¿Cómo se ejecuta el proyecto empaquetado? Capturen la pantalla del programa en ejecución:




NeoFlix. Series y episodios.
Listar
Adicionar
Buscar
Buscar
w
0 elementos


NeoFlix. Series y episodios.
Listar
Adicionar
Buscar
6 elementos
>The Hidden Village
>The First Mission
>The Rival
>The Tournament
>The Final Battle
>The Neo Ninja: 2020
The Hidden Village
The First Mission
The Rival
The Tournament
The Final Battle
Listar
Limpiar


Listar
Adicionar
Buscar
Nombre
Wilson
Año (para series)
2025
Intentos de visualizacion (para episodios)
Visualizacion completa (para episodios)
Numero de votos (para episodios)
Suma total de votos (para episodios)
Episodios (para series)
Adicionar
Limpiar



Retrospectiva:

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)

Cada uno invirtió aproximadamente 16 horas.

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

Terminado, hicimos todo lo solicitado

3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?

Pair programming, porque fuimos codificando al tiempo y corrigiendo errores que de pronto el otro no se fijaba.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Aprender a usar la consola de comandos, porque siempre ha sido algo bien interesante.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

Entender como usar la consola de comandos. Simplemente buscamos ejemplos y demás.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

El pair programming, nos comprometemos a hacer desarrollos más rápidos.

7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Usamos mayormente el learn Microsoft para el tema de los comandos de la consola de comandos, aunque también de usaron documentos de Oracle o páginas externas para saber el orden y definición de algunos conceptos nuevos.

Referencias:

https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/cmd

https://stackoverflow.com/questions/41905863/difference-between-src-and-bin

https://docs.oracle.com/en/java/javase/19/docs/specs/man/javac.html